virtual memory. The cache manager bridges the gap between these two file representations, ensuring consistency in file data between the two views.[330]

When a file is read from a disk, its contents are stored in the system cache, which is a portion of the system address space. Then, a user-mode process copies the data from the cache into its own address space. A thread can set a flag that overrides this default behavior; in this case, the transfer occurs directly between the disk and the process's address space. Similarly, instead of writing directly to the disk, a process writes new data to the cache entry (unless a flag is set).[331] Fulfilling an I/O request without generating an IRP or accessing a device is called **fast I/O.**[332]

Dirty cache entries can be written to disk in several ways. The memory manager might need to page a dirty cache page out of memory to make room for another memory page. When this occurs, the contents of the cache page are queued to be written to disk. If a dirty cache page is not paged out of memory by the memory manager, the cache manager must flush the page back to disk. It accomplishes this by using a lazy writer thread. The lazy writer is responsible for, once per second, choosing one-eighth of the dirty pages to be flushed to disk. It constantly reevaluates how often dirty pages are being created and adjusts the amount of pages it queues to be flushed accordingly. The lazy writer chooses pages based on how long a page has been dirty and the last time the data on that page was accessed for a read operation. Additionally, threads can force a specific file to be flushed. Threads can also specify write-through caching (see Section 12.8, Caching and Buffering) by setting a flag when creating the file object.[333]

## 21.10 Interprocess Communication

Windows XP provides many interprocess communication (IPC) mechanisms to allow processes to exchange data and cooperate to complete tasks. It implements many traditional UNIX IPC mechanisms, such as pipes, message queues (called mailslots by Windows XP)[334] and shared memory. In addition to these "data-oriented" IPCs, Windows XP allows processes to communicate through "procedure-oriented" or "object-oriented" techniques, using such tools as remote procedure calls or Microsoft's Component Object Model. Users on a Windows XP system also can initiate IPC with familiar features such as the clipboard and drag-and-drop capabilities. In each of Windows XP's IPC mechanisms, a server process makes some communication object available. A client process can contact the server process via this communication object to place a request. The request might be for the server to execute a function or to return data or objects. The remainder of this section describes these IPC tools. Section 21.11, Networking, introduces techniques and tools, such as sockets, that processes can use to communicate across a network.

### 21.10.1 Pipes

Windows XP provides pipes for direct communication between two processes.[335] If communicating processes have access to the same memory bank, pipes use shared

memory. Thread can manipulate pipes using standard file system routines (e.g., read, write and open). A process that creates the pipe is called the **pipe server;** processes that connect to a pipe are referred to as **pipe clients.**[336] Each pipe client communicates solely with the pipe server (i.e., not with other clients using the same pipe) because the pipe server uses a unique instance of the pipe for each client process.[337] The pipe server specifies a mode (read, write or duplex) when the pipe is created. In read mode, the pipe server receives data from clients; in write mode, it sends data to pipe clients; in duplex mode, it can both send and receive data via the pipe.[338] Windows XP provides two types of pipes: **anonymous pipes** and **named pipes.**

*Anonymous Pipes*

Anonymous pipes are used for unidirectional communication (i.e., duplex mode is not allowed) and can be used only among local processes. Local processes are processes that can communicate with each other without sending messages over a network.[339] A process that creates an anonymous pipe receives both a read handle and a write handle for the pipe. A process can read from the pipe by passing the read handle as an argument in a read function call, and a process can write to the pipe by passing the write handle as an argument in a write function call (for more information on handles see Section 21.5.2, Object Manager). To communicate with another process, the pipe server must pass one of these handles to another process. Typically, this is accomplished through inheritance (i.e., the parent process allows the child process to inherit one of the pipe handles). Alternatively, the pipe server can send a handle for the pipe to another, unrelated process via some IPC mechanism.[340]

Anonymous pipes support only synchronous communication. When a thread attempts to read from the pipe, it waits until the requested amount of data has been read from the pipe. If there is no data in the pipe, the thread waits until the writing process places the data in the pipe. Similarly, if a thread attempts to write to a pipe containing a full buffer (the buffer size is specified at creation time), it waits until the reading process removes enough data from the pipe to make room for the new data. A thread that is waiting to read data via a pipe can stop waiting if all write handles to the pipe are closed or if some error occurs. Similarly, a thread waiting to write data to a pipe can stop waiting if all read handles to the pipe are closed or if some error occurs.[341]

***Named Pipes***

Named pipes support several features absent in anonymous pipes. Named pipes can be bidirectional, be shared between remote processes[342] and support asynchronous communication. However, named pipes introduce additional overhead. When the pipe server creates a pipe, the pipe server specifies its mode, its name and the maximum number of instances of the pipe. A pipe client can obtain a handle for the pipe by specifying the pipe's name when attempting to connect to the pipe. If the number of pipe instances is less than the maximum, the client process connects to a new instance of the pipe. If there are no available instances, the pipe client can wait for an instance to become available.[343]

Named pipes permit two pipe writing formats. The system can transmit data either as a stream of bytes or as a series of messages. All instances of a pipe must use the same write format (recall that a single pipe server can communicate with multiple clients by creating a unique instance of the pipe for each client). Although sending data as a series of bytes is faster, messages are simpler for the receiver to process because they package related data.[344]

Windows XP permits a process to enable **write-through mode.** When the write-through flag is set, write operations do not complete until the data reaches the receiving process's buffer. In the default mode, once the data enters the writer's buffer, the write method returns. Write-through mode increases fault tolerance and permits greater synchronization between communicating processes because the sender knows whether a message successfully reaches its destination. However, write-through mode degrades performance, because the writing process must wait for the data to be transferred across a network from the writer's end of the pipe to the reader's end.[345] The system always performs writes using the write-through method when a process is using messages to ensure that each message remains a discrete unit.[346]

Asynchronous I/O capability increases named pipes' flexibility. Threads can accomplish asynchronous communication via named pipes in several ways. A thread can use an event object to perform an asynchronous read or write request. In this case, when the function completes, the event object enters its *signaled* state (see Section 21.6.3, Thread Synchronization). The thread can continue processing, then wait for one or more of these event objects to enter *signaled* states to complete a communication request. Alternatively, a thread can specify an I/O completion routine. The system queues this routine as an APC when the function completes, and the thread executes the routine the next time the thread enters an alertable *wait* state.[347] In addition, a pipe can be associated with an I/O completion port because pipes are manipulated like files.

### 21.10.2 Mailslots

Windows XP provides **mailslots** for unidirectional communication between a server and clients.[348] The process that creates the mailslot is the **mailslot server,** whereas the processes that send messages to the mailslot are **mailslot clients.** Mailslots act as repositories for messages sent by mailslot clients to a mailslot server. Mailslot clients can send messages to either local or remote mailslots; in either case, there is no confirmation of receipt. Windows XP implements mailslots as files in the sense that a mailslot resides in memory and can be manipulated with standard file functions (e.g., read, write and open). Mailslots are temporary files (the Windows XP documentation refers to them as "pseudofiles"); the object manager deletes a mailslot when no process holds a handle to it.[349]

Mailslot messages can be transmitted in two forms. Small messages are sent as datagrams, which are discussed in Section 16.6.2, User Datagram Protocol (UDP). Datagram messages can be broadcast to several mailslots within a particular **domain.** A domain is a set of computers that share common resources such as print-

ers.[350, 351] Larger messages are sent via a Server Message Block (SMB) connection.[352] SMB is a network file sharing protocol used in Windows operating systems.[353] Both SMB and datagram messages can be sent from a mailslot client to a mailslot server by specifying the server name in a write operation.[354]

### 21.10.3 Shared Memory

Processes can communicate by mapping the same file to their respective address spaces, a technique known as shared memory. The Windows XP documentation calls this mechanism **file mapping.** A process can create a **file mapping object** that maps any file, including a pagefile, to memory The process passes a handle for the file mapping object to other processes, either by name or through inheritance. The process communicates by writing data into these shared memory regions, which other processes can access.

A process can use the handle to create a **file view** that maps all or part of the file to the process's virtual address space. A process may own multiple file views of the same file. This allows a process to access the beginning and end of a large file without mapping all of the file to memory, which would reduce its available virtual address space. Because identical file views map to the same memory frames, all processes see a consistent view of a memory-mapped file. If one process writes to its file view, the change will be reflected in all file views of that portion of the file.[355,356]

File mapping objects accelerate data access. The system maps file views independently to each process's virtual address space, but the data is located in the same frames in main memory. This minimizes the necessity of reading data from the disk. The VMM treats file mappings the same way it treats copy-on-write pages (see Section 21.7.1, Memory Organization). Because the VMM has been optimized to deal with paging, file access is efficient. Due to its efficient implementation, file mapping has uses beyond communication. Processes can use file mapping to perform random and sequential I/O quickly. File mapping is also helpful for databases that need to overwrite small sections of large files.[357,358]

File mapping objects do not provide synchronization mechanisms to protect the files they map. Two processes can overwrite the same portion of the same file simultaneously. To prevent race conditions, processes must use synchronization mechanisms, such as mutexes and semaphores, that are provided by Windows XP and described in Section 21.6.3, Thread Synchronization.[359]

### 21.10.4 Local and Remote Procedure Calls

Many IPC mechanisms facilitate data exchange between two processes. **Local procedure calls (LPCs)** and remote procedure calls (RPCs), which we described in Section 17.3.2, provide a more procedural form of IPC and largely hide the underlying network programming. LPCs and RPCs allow a process to communicate with another process by calling functions executed by the other process.[360] The process that calls the function is the client process, and the one that executes the function is the server process.

When client and server processes reside on the same physical machine, processes execute LPCs; when they reside on separate machines, processes execute RPCs.[361] A designer employs LPCs and RPCs the same as procedure calls within a process. However, in some cases, the programmer takes a few extra steps to establish a connection (these are described shortly).

Note that Microsoft does not publicly document the LPC interface, and user-mode threads cannot expose LPCs. The system reserves LPCs for use by the Windows XP kernel. For example, when a user thread calls a function in an environment subsystem's API, the kernel might convert the call into an LPC. Also, Windows XP converts RPCs between two processes on the same machine (called **local remote procedure calls** or **LRPCs)** into LPCs, because LPCs are more efficient.[362] LPCs have less overhead than RPCs, because all data transfers using LPCs involve direct memory copying rather than network transport.[363] Although the name "local remote procedure call" might sound like an oxymoron, it is simply a descriptive name for a procedure call made with the RPC interface between local processes.

Figure 21.16 describes the path through which data travels during an LPC or RPC call. In both an RPC and an LPC, the client process calls a procedure that maps to a stub (1). This stub marshals the necessary function arguments (i.e., gathers the function arguments into a message to send to the server) and converts them to **Network Data Representation (NDR),** which is a standard network data format, described in The Open Group's Distributed Computing Environment (DCE) standard (2).[364, 365] Next, the stub calls the appropriate functions from a runtime library to send the request to the server process. The request traverses the network (in the case of an RPC) to reach the server-side runtime library (3), which passes the request to

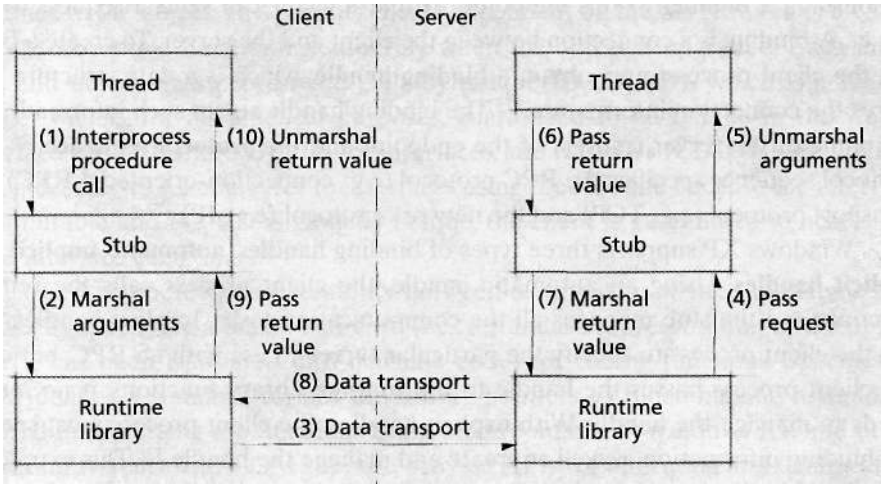


*Figure 21.16 | Flow of an LPC or RPC call.*

the server stub (4). The server stub unmarshals the data (i.e., converts the client's message into a call to the intended procedure) for the server function (5). The server process executes the requested function and sends any return values to the client via the reverse path (6 through 10).[366] Developers can employ RPCs using many different transport and network protocols such as TCP/IP and Novell Netware's Internetworking Packet eXchange/Sequenced Packet eXchange (IPX/SPX).[367] Both client processes and server processes can execute RPCs either synchronously or asynchronously, and the communicating processes need not use the same method.[368]

### *Cliend and Server Communication in an RPC*

To expose a procedure as an RPC, the server process must create an **Interface Definition Language (IDL) file.** This file specifies the interfaces that the RPC server presents to other processes.[369] The developer writes the interface in **Microsoft IDL (MIDL),** which is Microsoft's extension of IDL—The Open Group's Distributed Computing Environment (DCE) standard for RPC interoperability.[370] The IDL file consists of a header and an interface body. The header describes information global to all interfaces defined in the body such as the **universally unique identifier (UUID)** for the interface and an RPC version number.[371] The body contains all variable and function prototype declarations. The MIDL compiler builds the client and server stubs from the IDL file.[372] The server and client also can create **application configuration files** (ACFs), which specify platform-specific attributes, such as the manner in which data should be marshalled or unmarshalled.[373]

The physical communication in an RPC is accomplished through an endpoint that specifies the network-specific address of the server process and is typically a hardware port or named pipe. The server creates an endpoint before exposing the RPC to other processes.[374] Client-side runtime library functions are responsible for establishing a **binding** to this endpoint, so that a client can send a request to the server. A binding is a connection between the client and the server. To create a binding, the client process must obtain a **binding handle,** which is a data structure that stores the connection information.[375] The binding handle stores such information as the name of the server, address of the endpoint and the **protocol sequence.**[376] The protocol sequence specifies the RPC protocol (e.g., connection-oriented, LRPC), the transport protocol (e.g., TCP) and the network protocol (e.g., IP).[377]

Windows XP supports three types of binding handles: **automatic, implicit** and **explicit handles.** Using an automatic handle, the client process calls the remote function, and the stub manages all the communication tasks. Implicit handles permit the client process to specify the particular server to use with an RPC, but once the client process passes the handle to the runtime library functions, it no longer needs to manage the handle. With explicit handles, the client process must specify the binding information as well as create and manage the handle.[378] This extra control permits client processes using explicit handles to connect to more than one server process and simultaneously execute multiple RPCs.[379]

Communication via an LPC is similar to an RPC, although some steps are omitted. The server must still create an IDL file, which is compiled by a MIDL

compiler. The client process must include the client-side runtime library with the file that calls the LPC. The stub handles the communication, and the client and server processes communicate over a port for procedure calls that involve small data transfers. Procedure calls in which processes transfer a large amount of data must use shared memory. In this case, the sender and receiver place message data into a section of shared memory.[380]

### 21.10.5 Component Object Model (COM)

Microsoft's **Component Object Model (COM)** provides a software architecture allowing interoperability between diverse software components. In the COM architecture, the relative location of two communicating components is transparent to the programmer and can be in process, cross process or cross network. COM is not a programming language, but a standard designed to promote interoperability between components written in different programming languages. COM also is implemented on some flavors of UNIX and Apple Macintosh operating systems. Developers use COM to facilitate cooperation and communication between separate components in large applications.[381]

COM objects (a COM object is the same as a COM component) interact indirectly and exclusively through interfaces.[382] A COM interface, which is written in MIDL, is similar to a Java interface; it contains function prototypes that describe function arguments and return values, but does not include the actual implementation of the functions. As with Java, a COM object must implement all of the methods described in the interface and can implement other methods as well.[383] Once created, a COM interface is immutable (i.e., it cannot change). A COM object can have more than one interface; developers augment COM objects by adding a new interface to the object. This permits clients dependent on the old features of a COM object to continue functioning smoothly when the object is upgraded. Each interface and object class possesses a **globally unique ID (GUID),** which is a 128-bit integer that is, for all practical purposes, guaranteed to be unique in the world. **Interface IDs (IIDs)** are GUIDs for interfaces, and **class IDs (CLSIDs)** are GUIDs for object classes. Clients refer to interfaces using the IID, and because the interface is immutable and the IID is globally unique, the client is guaranteed to access the same interface each time.[384]

COM promotes interoperability between components written in diverse languages by specifying a binary standard (i.e., a standard representation of the object after it has been translated into machine code) for calling functions. Specifically, COM defines a standard format for storing pointers to functions and a standard method for accessing the functions using these pointers.[385] Windows XP, and other operating systems with COM support, provide APIs for allocating and deallocating memory using this standard. The COM library in Windows XP provides a rich variety of support functions that hide most of the COM implementations.[386]

COM servers can register their objects with the Windows XP registry. Clients can query the registry using the COM object's CLSID to obtain a pointer to the

COM object.[387] A client can also obtain a pointer to a COM object's interface from another COM object or by creating the object. A client can use a pointer to any interface for a COM object to find a pointer to any other interface that the COM object exposes.[388]

Once the client obtains a pointer to the desired interface, the client can execute a function call to any procedure exposed by this interface (assuming the client has appropriate access rights). For in-process procedure calls, COM executes the function directly, adding essentially no overhead. For cross-process procedure calls in which both processes reside on the same computer, COM uses LRPCs (see Section 21.10.4, Local and Remote Procedure Calls), and Distributed COM (DCOM) supports cross-network function calls.[389] COM marshals and unmarshals all data, creates the stubs and proxies (proxies are client-side stubs) and transports the data (via direct execution, LRPCs or DCOM).[390]

COM supports several threading models a process can employ to maintain its COM objects. In the **apartment model,** only one thread acts as a server for each COM object. In the **free thread model,** many threads can act as the server for a single COM object (each thread operates on one or more instances of the object). A process can also use the **mixed model** in which some of its COM objects reside in single apartments and others can be accessed by free threads. In an apartment model, COM provides synchronization by placing function calls in the thread's window message queue. COM objects that can be accessed by free threads must maintain their own synchronization (see Section 21.6.3, Thread Synchronization, for information on thread synchronization).[391]

Microsoft has built several technologies on top of COM to further facilitate cooperation in component software design. **COM+** extends COM to handle advanced resource management tasks; e.g., it provides support for transaction processing and uses thread pools and object pools, which move some of the responsibility for resource management from the component developer to the system.[392] COM+ also adds support for Web services and optimizes COM scalability.[393] Distributed COM (DCOM) provides a transparent extension to basic COM services for cross-network interactions and includes protocols for finding DCOM objects on remote services.[394] Object Linking and Embedding (OLE), discussed in the next section, builds on COM to provide standard interfaces for applications to share data and objects. **ActiveX Controls** are self-registering COM objects (i.e., they insert entries in the registry upon creation). Typically, ActiveX Controls support many of the same embedding interfaces as OLE objects; however, ActiveX Controls do not need to support all of them. This makes ActiveX Controls ideal for embedding in Web pages because they have less overhead.[395]

### *21.10.6 Drag-and-Drop and Compound Documents*

Windows XP provides several techniques that enable users to initiate IPC. A familiar example is permitting users to select text from a Web page, copy this text and paste it into a text editor. Users can also embed one document inside another; e.g., a

user could place a picture created in a graphical design program into a word processing document. In both of these cases, the two applications represent different processes that exchange information. Windows XP provides two primary techniques for this type of data transport: the **clipboard** and **Object Linking and Embedding (OLE).**

The clipboard is a central repository of data, accessible to all processes.[396] A process can add data to the clipboard when the user invokes either the copy or the cut command. The selected data is stored in the clipboard along with the data's format.[397] Windows XP defines several standard data formats, including text, bitmap and wave. Processes can register new clipboard formats, create private clipboard formats and synthesize one or more of the existing formats.[398] Any process can retrieve data from the clipboard when the user invokes the paste command.[399] Conceptually, the clipboard acts as a small area of globally shared memory.

OLE builds on COM technology by defining a standard method for processes to exchange data. All OLE objects implement several standard interfaces that describe how to store and retrieve data for an OLE object, how to access the object and how to manipulate the object. Users can create compound documents—documents with objects from more than one application—either by linking outside objects or embedding them. When a document links an object from another source, it does not maintain the object inside its document container (the document container provides storage for the document's objects and methods for manipulating and viewing these objects). Rather, the document maintains a reference to the original object. The linked object reflects updates made by the server process to the object. Embedding an object places it, along with the interfaces for manipulating the object, inside the client document. The linked or embedded object is a COM component complete with a set of interfaces that the client can use to manipulate the object.[400]

## *21.11 Networking*

Windows XP supports many networking protocols and services. Developers employ networking services to accomplish IPC with remote clients and make services and information available to users. Users exploit networks to retrieve information and access resources available on remote computers. This section investigates various aspects of Windows XP's networking model. We examine network I/O and the driver architecture employed by Windows XP. We consider the network, transport and application protocols supported by Windows XP. Finally, we describe the network services that Windows XP provides, such as the Active Directory and .NET.

### *21.11.1 Network Input/Output*

Windows XP provides a transparent I/O programming interface. In particular, programmers use the same functions regardless of where the data reside. However, the Windows XP system must handle network I/O differently than local I/O.[401]

Figure 21.17 illustrates how Windows XP handles a network I/O request. The client contains a driver called a **redirector** (or **network redirector).**[402] A redirector is a file system driver that directs network I/O requests to appropriate devices over a network.[403] First, the client process passes an I/O request to an environment subsystem (1). The environment subsystem then sends this request to the I/O manager (2), which packages the request as an IRP, specifying the file's location in **Uniform Naming Convention (UNC) format.** UNC format specifies the file's pathname, including on which server and in which directory on that server the file is located. The I/O manager sends the **IRP** to the **Multiple UNC Provider (MUP),** which is a file system driver that determines the appropriate redirector to which to send the request (3). Later in this section we see that Windows XP ships with two file sharing protocol, CIFS and WebDAV, which use different redirectors. After the redirector receives the IRP (4), the redirector sends the request over the network to the appropriate server file system driver (5). The server driver determines whether the client has sufficient access rights for the file, then communicates the request to the target device's driver stack via an IRP (6). The server driver receives the result of the I/O request (7). If there is an error processing the request, the server driver informs the client. Otherwise, the server driver passes the data back to the redirector (8), and this data is passed to the client process (9).[404,405]

The **Common Internet File System (CIFS)** is Windows XP's native file sharing protocol. CIFS is used for the application layer of the TCP/IP stack (see Section 16.4,

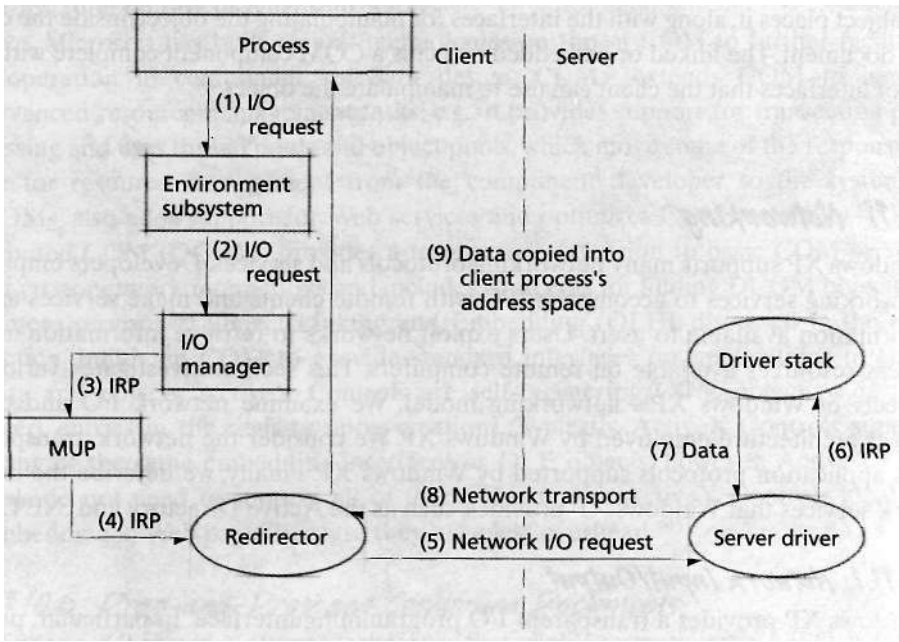


***Figure 21.17*** | *Handling & network I/O request.*

TCP/IP Protocol Stack). VMS and several varieties of UNIX also support CIFS. Developers often pair CIFS with NetBIOS (discussed later in this section) over TCP/IP for the network and transport layers, but CIFS can use other network and transport protocols as well.[406] CIFS is meant to complement HTTP and replace older file sharing protocols such as FTP.[407] Windows XP also supports other network file sharing protocols such as **Web-based Distributed Authoring and Versioning (WebDAV).** WebDAV allows users to write data directly to HTTP servers and is designed to support collaborative authoring between groups in remote locations.[408]

To share files using CIFS, the client and server must first establish a connection. Before a client can access a file, the client and server connect, and the server authenticates the client (see Section 19.3, Authentication) by evaluating the username and password sent by the client. The redirectors interact to set up the session and transfer data.[409] CIFS provides several types of **opportunistic locks (oplocks)** to enhance I/O performance. A client uses an oplock to secure exclusive access to a remote file. Without one of these oplocks, a client cannot cache network data locally, because other clients also might be accessing the data. If two clients both cache data locally and write to this data, their caches will not be coherent (see Section 15.5, Multiprocessor Memory Sharing). Therefore, a client obtains an oplock to ensure that it is the only client accessing the data.[410] When a second client attempts to gain access to the locked file (by attempting to open it), the server may break (i.e., invalidate) the first client's oplock. The server allows the client enough time to flush its cache before granting access to the second client.[411]

### *21.11.2 Network Driver Architecture*

Windows XP uses a driver stack to communicate network requests and transmit network data. This driver stack is divided into several layers, which promotes modularity and facilitates support for multiple network protocols. This subsection describes the low-level driver architecture employed in Windows XP.

Microsoft and 3Com developed the **Network Driver Interface Specification (NDIS),** which specifies a standard interface between lower-level drivers in the network driver stack. NDIS drivers provide the functionality of the link layer and some functionality of the network layer of the TCP/IP protocol stack (see Section 16.4, TCP/IP Protocol Stack).[412] NDIS drivers communicate through functions provided in Windows XP's NDIS library. The NDIS library functions translate one driver's function call into a call to a function exposed by another driver. This standard interface increases the portability of NDIS drivers.[413] NDIS allows the same programming interface (e.g., Windows sockets, see Section 21.11.3, Network Protocols) to establish connections using different network protocols, such as IP and Internet Packet eXchange (IPX).[414]

Figure 21.18 illustrates the architecture that Windows XP employs to process network data. Windows XP divides the network architecture into three layers that map to layers of the TCP/IP protocol stack. The physical network hardware, such as the network interface card (NIC) and the network cables, form the physical layer.

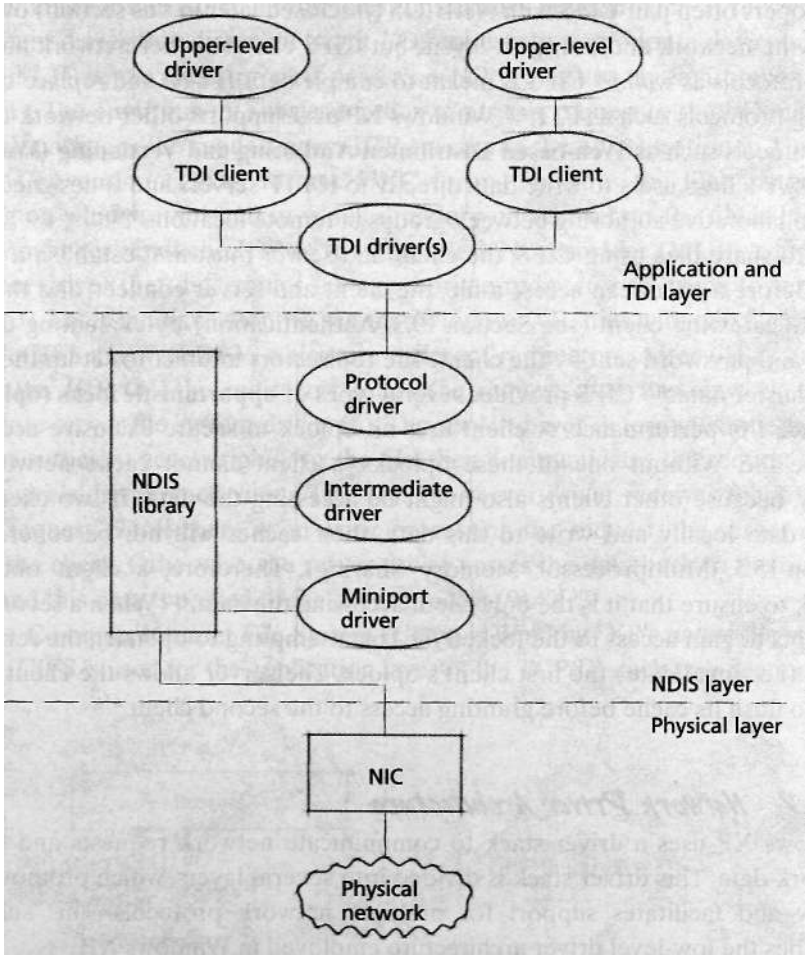


***Figure 21.18*** | *Networking driver architecture.*

NDIS drivers (described in the following paragraphs) manage the data link and network layers. **Transport Driver Interface (TDI) drivers** and TDI clients compose the top layer. The TDI drivers provide a transport interface between NDIS drivers and TDI clients. TDI clients are low-level application drivers and might interact with other application-layer drivers.

Designers implement the NDIS layer with several drivers. An **NDIS miniport driver** manages the NIC and transmits data between the NIC and higher-level drivers. The miniport driver interfaces with the drivers above it in the driver stack to transmit outgoing network data to the NIC. An upper-level driver invokes an NDIS function that passes the packet to the miniport driver, and the miniport driver passes the data to the NIC. The miniport driver also processes the NIC's interrupts and passes data that the NIC receives up the driver stack.[415]

**NDIS intermediate drivers** are optional drivers that reside between a miniport driver and a higher-level driver. Intermediate drivers translate packets between different communication media, filter packets or provide load balancing across several NICs.[416]

**NDIS protocol drivers** are the highest-level NDIS drivers. Protocol drivers place data into packets and pass these packets to lower-level drivers (such as intermediate drivers or miniport drivers). Protocol drivers provide an interface between the transport drivers and other NDIS drivers and can be used as the lowest layer in the implementation of a transport protocol stack (such as TCP/IP).[417]

The NDIS protocol driver interacts with TDI drivers. A TDI driver exports a network interface to upper-level kernel-mode clients. The redirectors described in the previous subsection are TDI clients. Winsock applications (described in the next subsection) also are TDI clients. TDI drivers implement transport protocols such as UDP and TCP; NDIS protocol drivers often implement the network protocol (e.g., IP). Alternatively, a TDI transport driver can implement both the network and transport protocols. Upper-level kernel-mode drivers send network requests to TDI drivers.[418] These drivers implement the application layer of the TCP/IP protocol stack. The next subsections describe the various protocols and network services available to Windows XP applications.

### 21.11.3 Network Protocols

Windows XP supports several different protocols for the network, transport and application layers of the TCP/IP stack. Windows XP supports some of these protocols to provide compatibility with popular legacy programs or other network clients.

#### *Network and Transport Protocols*

IP is installed by default on Windows XP systems; IP provides routing for packets traversing a network and is the Internet's network protocol. Users can install other network protocols, such as the **Internetwork Packet exchange (IPX)** protocol, which is used by Novell's Netware.[419] IPX provides services similar to IP; IPX routes packets between different locations in a network, but is designed for LANs.[420] Because the NDIS library supports a standard interface between protocol drivers and NICs, Windows XP can use the same NIC to operate both protocols.[421]

Windows XP also supports several transport protocols. TCP, by far the most commonly used transport protocol, is installed by default. TCP provides a connection-oriented transport; i.e., the participants of a network communication must create a session before communicating.[422] For connectionless transport, Windows XP supports UDP.[423] Users can also install support for **Sequenced Packet eXchange (SPX),** which offers connection-oriented services for IPX packets. Windows XP provides SPX for interoperability with clients and servers using IPX/SPX protocols, such as Netware,[424]

Before the explosion of interest in the Internet and World Wide Web, the DOS and Windows operating systems supported **NetBIOS Extended User Inter-**

**face (NetBEUI)** as the native network and transport protocols. **Network Basic Input/Output System (NetBIOS)** is the programming API that supports NetBEUI. NetBEUI is a fairly primitive protocol most applicable for small networks. It has several limitations; for example, it does not provide for network connection routing outside a LAN.[425] Windows XP does not provide support for NetBEUI, although support can be manually installed by a user.

However, **NetBIOS over TCP/IP (NBT)** is still supported by Windows XP to provide compatibility with old applications that employ the NetBIOS API.[426] NetBIOS layered over TCP and IP provides connection-oriented communication using NBT. Alternatively, developers can layer NetBIOS over UDP and IP for connectionless communication using NBT. CIFS is commonly used with NBT as the underlying network and transport protocols. However, in Windows XP, CIFS can execute over TCP/IP without NetBIOS encapsulation.[427]

### *Application Layer Protocols*

At the application layer, Windows XP supports HTTP for Web browsing and CIFS for file and print sharing. For HTTP support, Windows XP provides **WinHTTP** and **WinlNet.** WinHTTP supports communication between clients and servers over an HTTP session. It provides SSL security and is integrated with Microsoft Passport (see Section 19.3.4, Single Sign-On). WinHTTP also provides other HTTP services, such as tracing and Kerberos authentication (see Section 19.3.3, Kerberos) and allows synchronous and asynchronous communication.[428] WinlNet is an older API that allows applications to interact with the FTP, HTTP and Gopher protocols to access resources over the Internet.[429] WinlNET is designed for client applications, whereas WinHTTP is ideal for servers.[430]

CIFS, described in the previous section, is an extension of the **Server Message Block (SMB)** file sharing protocol. SMB is the native file sharing protocol of older Windows operating systems such as Windows NT and Windows 95. SMB uses NetBIOS to communicate with other computers in a virtual LAN. A virtual LAN consists of computers that share the same NetBIOS namespace; the computers can be anywhere in the world. To share files with SMB, two computers on the same virtual LAN set up an SMB session; this includes acknowledging the connection, authenticating the identity of both parties and determining the access rights of the client. CIFS extends SMB, adding a suite of protocols that improve access control and handle announcing and naming of network resources. The familiar My Network Places program included with Windows XP allows users to share files through CIFS.[431]

### *Winsock*

Windows XP supports network communication through sockets. **Windows sockets 2 (Winsock 2)** is an application of BSD sockets (see Section 20.10.3, Sockets) to the Windows environment. Sockets create communication endpoints that a process can use to send and receive data over a network.[432] Sockets are not a transport protocol; consequently, they need not be used at both ends of a communication connection. Developers can layer Winsock 2 on top of any transport and network protocols (e.g.

TCP/IP or IPX/SPX). Users can port code written for UNIX-style sockets to Winsock applications with minimal changes. However, Winsock 2 includes extended functionality that might make porting an application from Windows XP to a UNIX-based operating system implementing Berkeley sockets difficult.[433, 434]

The Winsock specification implements many of the same functions that Berkeley sockets implement to provide compatibility with Berkeley socket applications. As with Berkeley sockets, a Windows socket can be either a stream socket or a datagram socket. Recall that stream sockets provide reliability and packet ordering guarantees.[435] Winsock adds extra functionality to sockets, such as asynchronous I/O, protocol transparency (i.e., developers can choose the network and transport protocols to use with each socket) and quality-of-service (QoS) capabilities.[436] The latter provide to processes information about network conditions and the success or failure of socket communication attempts.[437]

### *21.11.4 Network Services*

Windows XP includes various network services that allow users to share information, resources and objects across a network. **Active Directory** provides directory services for shared objects (e.g., files, printers, services, users, etc.) on a network.[438] Its features include location transparency (i.e., users do not know the address of an object), hierarchical storage of object data, rich and flexible security infrastructure and the ability to locate objects based on different object properties.[439]

**Lightweight Directory Access Protocol (LDAP)** is a protocol for accessing, searching and modifying Internet directories. An Internet directory contains a listing of computers or resources accessible over the Internet. LDAP employs a client/server model.[440] LDAP clients can access an Active Directory.[441]

Windows XP provides **Remote Access Service (RAS),** which allows users to remotely connect to a LAN. The user must be connected to the network through a WAN or a VPN. Once the user connects to the LAN through RAS, the user operates as though directly on the LAN.[442]

Support for distributed computing on Windows XP is provided through the Distributed Component Object Model (DCOM) and .NET. DCOM is an extension of COM, which we described in Section 21.10.5, Component Object Model (COM), for processes that share objects across a network. Constructing and accessing DCOM objects is done just as with COM objects, but the underlying communication protocols support network transport. For information on using DCOM for distributed computing see Section 17.3.5, DCOM (Distributed Component Object Model).[443] The next section introduces .NET.

## *21.11.5 .NET*

Microsoft has developed a technology called .NET, which supplants DCOM for distributed computing in newer applications. .NET is aimed at transforming computing from an environment where users simply execute applications on a single computer to a fully distributed environment. In this computing model, local applications com-

municate with remote applications to execute user requests. This contrasts with the traditional view of a computer as merely fulfilling a single user's requests. .NET is a middleware technology, which provides a platform- and language-neutral development environment for components that can easily intemperate.[444, 445]

Web services are the foundation of .NET; they encompass a set of XML-based standards that define how data should be transported between applications over HTTP. Because of XML's large presence in this technology, .NET Web services are often called XML Web services. Developers use Web services to build applications that can be accessed over the Internet. Web service components can be exposed over the Internet and used in many different applications (i.e., Web services are modular). Because XML messages (which are sent and retrieved by Web services) are simply text messages, any application can retrieve and read XML messages. Web services make .NET interoperable and portable between different platforms and languages.[446]

The .NET programming model is called the **.NET framework.** This framework provides an API which expands on the Windows API to include support for Web services. The .NET framework supports many languages, including Visual Basic .NET, Visual C++ .NET and C#. The .NET framework facilitates development of .NET Web services.[447]

.NET server components include Web sites and .NET enterprise servers, such as Windows Server 2003. Many Web service components that home users access are exposed on these two platforms. Although desktop operating systems, such as Windows XP, or server systems running Windows XP can expose Web services, Windows XP systems are mainly consumers (i.e., clients) of .NET services.[448]-[449]

## 21.12 Scalability

To increase Windows XP's flexibility, Microsoft developed several editions. For example, Windows XP Home Edition is tailored for home users, whereas Windows XP Professional is more applicable in a corporate environment. Microsoft has also produced editions of Windows XP to address scalability concerns. Although all editions of Windows XP support symmetric multiprocessing (SMP), Windows XP 64-Bit Edition is specifically designed for high-performance desktop systems, which are often constructed as SMP systems (see Section 15.4.1, Uniform Memory Access). Additionally, Microsoft has developed an embedded operating system derived from Windows XP, called Windows XP Embedded. An embedded system consists of tightly coupled hardware and software designed for a specific electronic device, such as a cellular phone or an assembly-line machine. This section describes the features included in Windows XP to facilitate scaling up to SMP systems and down to embedded systems.

### 21.12.1 Symmetric Multiprocessing (SMP)

Windows XP scales to symmetric multiprocessor (SMP) systems. Windows XP supports multiprocessor thread scheduling, locking the kernel, a 64-bit address space

(in Windows XP 64-Bit Edition) and API features that support programming server applications.

Windows XP can schedule threads efficiently on multiprocessor systems, as described in Section 21.6.2, Thread Scheduling. Windows XP attempts to schedule a thread on the same processor on which it recently executed to exploit data cached on a processor's private cache. The system also uses the ideal processor attribute to increase the likelihood that a process's threads execute concurrently, thus increasing performance. Alternatively, developers can override this default and use the ideal processor attribute to increase the likelihood that a process's threads execute on the same processor to share cached data.[450]

Windows XP implements spin locks (described in Section 21.6.3, Thread Synchronization), which are kernel locks designed for multiprocessor systems. Specifically, the queued spin lock decreases traffic on the processor-to-memory bus, increasing Windows XP's scalability.[451] In addition, as the original NT kernel has evolved (recall that Windows XP is NT 5.1), designers have reduced the size of critical sections, which reduces the amount of time a processor is busy waiting to obtain a lock. Sometimes, a processor cannot perform other work when waiting for a spin lock, such as when a processor needs to execute thread scheduling code. Reducing the size of this critical section increases Windows XP's scalability.[452] Additionally, Windows XP provides a number of other fine-grained locks (e.g., dispatcher objects and executive resource locks), so that developers need only lock specific sections of the kernel, allowing other components to execute without disruption.[453]

Memory support for large SMP systems emanates from Windows XP 64-Bit Edition. 64-bit addressing provides $2^{64}$ bytes (or 16 exabytes) of virtual address space; this contrasts with $2^{32}$ bytes (or 4 gigabytes) for 32-bit versions of Windows XP. In practice, Windows XP 64-Bit Edition provides processes with 7152GB of virtual memory address range.[454] Windows XP 64-Bit Edition currently supports up to 1 terabyte of cache memory, 128 gigabytes of paged system memory and 128 gigabytes of nonpaged pool. Windows XP 64-Bit Edition has been ported to Intel Itanium II processors, which are described in Section 14.8.4, Explicitly Parallel Instruction Computing (EPIC)[455]. Soon (as of this writing), Microsoft plans to release Windows XP 64-Bit Edition for Extended Systems to support AMD Opteron and Athlon 64 processors.[456] The 64-bit editions of Windows also offer greater performance and precision when manipulating floating point numbers, because these systems store floating point numbers using 64 bits.[457]

Finally, Windows XP has added several programming features tailored for high-performance systems, which is an environment in which many SMP systems are employed. The job object (described in Section 21.6.1, Process and Thread Organization) allows servers to handle resource allocation to a group of processes. In this way, a server can manage the resources it commits to a client request.[458] Also, I/O completion ports (described in Section 21.9.2, Input/Output Processing) facilitate the processing of large numbers of asynchronous I/O requests. In a large server program, many threads might issue asynchronous I/O requests; I/O completion ports

provide a central area where dedicated threads can wait and process incoming I/O completion notifications.[459] Additionally, thread pools (described in Section 21.6.1, Process and Thread Organization) permit server processes to handle incoming client requests without the costly overhead of creating and deleting threads for each new request. Instead, these requests can be queued to the thread pool.[460]

### *21.12.2 Windows XP Embedded*

Microsoft has created an operating system called **Windows XP Embedded,** which scales Windows XP for embedded devices. Microsoft divides Windows XP into clearly defined functional units called **components.** Components range from user-mode applications such as Notepad to portions of kernel space such as the CD-ROM interface and power management tools. The designer of a Windows XP Embedded system chooses from over 10,000 components to customize a system for a particular device.[461] Components for Windows XP Embedded contain the same code as Windows XP Professional, providing full compatibility with Windows XP. Windows XP Embedded also contains the core Windows XP microkernel.[462]

Windows XP Embedded is designed for more robust embedded devices such as printers, routers, point-of-sale systems and digital set-top boxes. Microsoft provides a different operating system, Windows CE, for lighter devices such as cellular phones and PDAs. The goal of Windows XP Embedded is to allow devices with limited resources to take advantage of the Windows XP computing environment.[463]

Devices such as medical devices, communication equipment and manufacturing equipment would benefit from Windows XP Embedded, but require hard real-time guarantees. Recall from Section 8.9, Real-Time Scheduling, the difference between soft real-time scheduling (like that provided by Windows XP) and hard real-time scheduling. Soft real-time scheduling assigns higher priority to real-time threads, whereas hard real-time scheduling provides guarantees that, given a start time, a thread will finish by a certain deadline. Venturcom[464] has designed a Real-Time Extension (RTX) for Windows XP Embedded and Windows XP. RTX includes a new threading model (implemented as a device driver), a collection of libraries and an extended HAL. Venturcom's RTX achieves hard real-time goals by masking all non-real-time interrupts when real-time threads execute, never masking real-time interrupts (recall that interrupt masking is accomplished through the HAL) and enforcing strict thread priorities. For example, threads waiting for synchronization objects, such as mutexes and semaphores, are placed in the *ready* state in priority order (as opposed to FIFO order). RTX implements 128 priority levels to increase scheduling flexibility.

## *21.13 Security*

Any operating system aimed at the corporate market must offer substantial security capabilities. Windows XP Professional provides a wide variety of security services to its users, whereas Windows XP Home Edition has a slightly more limited range and is targeted at consumers with simple security needs. The following subsections

describe security in Windows XP Professional. Much of the information, especially non-networking related material, applies to Windows XP Home Edition as well.

Windows XP provides users with a range of authentication and authorization options that permit access a physical terminal, a local network or the Internet. A database of users' credentials helps provide users with single sign-on; once a user logs onto Windows XP, the system takes care of authenticating the user to other domains and networks without prompting for credentials. The operating system includes a built-in firewall (see Section 19.6.1, Firewalls) that protects the system from malicious access.

### 21.13.1 Authentication

Windows XP has a flexible authentication policy. Users provide the system with credentials which consists of their identity (e.g. username) and proof of identity (e.g. password). For local access, the system authenticates a user's credentials against the **Security Account Manager (SAM)** database. A designated computer, called the **domain controller,** handles remote authentication. The native authentication system for network access is Kerberos V5 (see Section 19.3.3, Kerberos). Windows XP also supports the NT LanMan (NTLM) authentication protocol for backward compatibility with domain controllers running NT 4.O.[465]

Authentication begins at a logon screen; this can be either the standard logon program (called MSGINA) or a third-party **Graphical Identification and Authentication (GINA)** DLL.[466] GINA passes the user's username and password to the Winlogon program, which passes the credentials to the Local Security Authority (LSA). The LSA is responsible for handling all authentication and authorization for users physically accessing that particular machine.[467]

If the user is logging on only to the local machine, the LSA verifies the user's credentials with the local SAM database. Otherwise, the LSA passes the credentials over the network to the domain controller (the computer responsible for handling authentication) via the **Security Support Provider Interface (SSPI).** This is a standard interface for obtaining security services for authentication that is compatible with both Kerberos V5 and NTLM. The domain controller first attempts to authenticate using Kerberos, which verifies credentials using the Active Directory. If Kerberos fails, GINA resends the username and password. This time the domain controller attempts to authenticate via the NTLM protocol, which relies on a SAM database stored on the domain controller. If both Kerberos and NTLM fail, GINA prompts the user to reenter the username and password.[468, 469] Figure 21.19 illustrates the authentication process.

Computers, especially mobile ones such as laptops, often use different credentials to connect to different domains. This is done for security reasons; if one domain is compromised, accounts on the other domains remain secure.(Windows XP lets users store domain-specific credentials in Stored User Names and Passwords. Upon an authentication error, the LSA first attempts to authenticate using a saved credential for that domain. Upon failure, the LSA activates the GINA screen

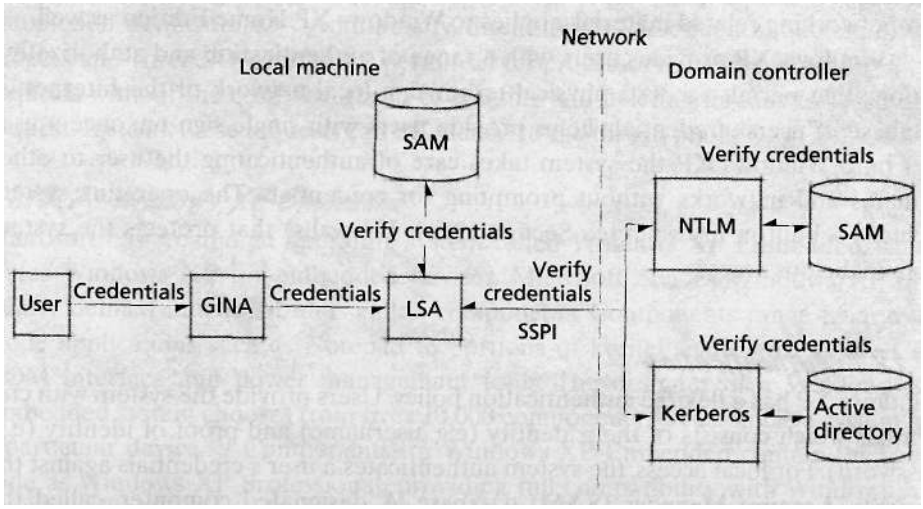


*Figure 21.19 | Authentication process in a Windows XP network.*

to prompt the user for a credential. The screen lets the user select a stored credential from a drop-down menu. The system records every credential that a user enters to a GINA (as long as the user selects a box giving the system permission to do so). In addition, users may use a special GUI to add, edit and remove credentials.[470]

## 21.13.2 Authorization

Any user, group, service or computer that performs an action in a Windows XP system is considered a **security principal.** A user can set security permissions for any security principal.[471] It is often advantageous to place users with the same security privileges into a security group, then set security permissions based on the security group. Security groups can be nested and grouped to form larger security groups. Using security groups simplifies managing the security policy for a large system.[472]

The system assigns each security principal a unique **security identifier (SID).** When a security principal logs on, the system uses the security principal's SID to assign an **access token.** The access token stores security information about the security principal, including its SID, the SID of all groups to which the security principal belongs and the default access control list (see Section 19.4.3, Access Control Mechanisms) to assign to all objects the security principal creates. The system attaches a copy of the access token to every process and thread that the security principal runs. The system uses a thread's access token to verify that the thread is authorized to perform a desired action.[473]

Windows XP uses access tokens to implement **fast user switching**—enabling a new user to log on without logging off the current user. (Fast user switching works only on computers that are not members of a domain). Often, especially in home environments, a user may start a long-running process, then leave the computer to

execute unattended. Fast user switching permits another person to log on, run programs, check e-mail, play a game, and so on. The first user's processes continue executing in the background; however, this is transparent to the new user. An access token stores a session ID that is unique to each logon session. The system uses the access token attached to each thread and process to determine to which user a process or thread belongs.[474, 475]

Every resource, such as a file, folder or device, has a **security descriptor** that contains the resource's security information. A descriptor contains the resource owner's SID and a **discretionary access control list (DACL)** that determines which security principals may access the resource. The DACL is an ordered list of **access control entries (ACEs);** each ACE contains the SID of a security principal and the type of access (if any) the security principal has to that particular resource.[476]

When a thread attempts to access a resource, Windows XP uses the thread's access token and the resource's security descriptor to determine whether to permit access. The system traverses the DACL and stops when it finds an ACE with a SID that matches one of the SIDs in the access token. The thread is permitted to perform the requested operation only if the first matching ACE authorizes the action. If the security descriptor has no DACL, access is granted; if the DACL has no matching ACE, access is denied.[477]

Windows XP offers users flexibility in specifying the security policy of their system. The main role of the operating system is to enforce the policy that its users set. However, with so many options to choose from, an administrator might inadvertently create an insecure security policy. To prevent this, Windows XP includes many useful features to help administrators maintain system security.

Recall the notion of security groups mentioned at the start of the section. Windows XP defines three basic security groups: Everyone (all authenticated security principals), Anonymous (all unauthenticated security principals) and Guest (a general-purpose account). System administrators can use these three security groups as the building blocks of their security policy. In addition, Windows XP includes several default security policies that administrators can employ as a starting point for creating their own policy. In general, each default policy uses the highest level of security, and system administrators are responsible for increasing user privileges. For example, by default, users who log on from a remote machine are forced to use Guest privileges. Although this might be an annoyance, this policy is preferable to inadvertently overlooking a security hole.[478]

### *21.13.3 Internet Connection Firewall*

Windows XP has a built-in packet filtering firewall (see 18.6.1, Firewalls) called the **Internet Connection Firewall (ICF).** This firewall can be used to protect either an entire network or a single computer.[479] ICF filters out all unsolicited inbound traffic, with the exception of packets sent to ports explicitly designated as open. The firewall stores information about all outgoing packets in a flow table. It permits an incoming packet to enter the system only if there is an entry in the flow table corre-

sponding to an outgoing packet sent to the home address of the incoming packet.[480] ICF does not filter outgoing packets; it assumes that only legitimate processes would send an outgoing packet. (This can cause problems if a Trojan Horse program gains access to the system.)[481]

ICF supports common Internet communication protocols such as FTP and LDAP. To permit applications to communicate with a Windows XP system via these protocols, ICF leaves certain ports open for all incoming traffic. Also, many applications such as MSN Messenger and online games require an open port to run successfully. Users and applications with administrative privileges can designate a particular port open for all incoming traffic; this process is called **port mapping.**[482, 483]

### 21.13.4 Other Features

Windows XP comes with many features that help keep the system secure. For example, users can encrypt files and folders using the Encrypting File System (see Section 21.8.2, NTFS, for more information). Users can control which Web sites may store cookies (small temporary files used by a server to store user-specific information) on their computer. Administrators can limit which applications may execute on the system; each application can be identified by its file hash, so that moving or renaming it would not bypass the restriction. Windows XP can also check whether the application is signed with a trusted certificate (i.e., a signature from an entity that the user specified as trustworthy) or comes from a trusted Internet Zone (i.e., an Internet domain that the user specified as safe).[484] Users can use Automatic Update to get notification, or even installation, of the latest security patches.[485]

## Web Resources

msdn.microsoft.com/library
The MSDN Library provides information about Microsoft products and services, including the most recent platform Software Development Kit (SDK) and Driver Development Kit (DDK). Go to the menu on the left side of the screen and click on Windows Development, followed by Windows Base Services, to research the Windows XP SDK. To research the DDK, click on Windows Development, followed by Driver Development Kit. Particularly useful information can be found by next clicking Kernel-Mode Driver Architecture, followed by Design Guide. The MSDN Library also provides networking information (click Networking and Directory Services) and COM information (click Component Development).

www.microsoft.com/windowsxp/default.asp
Microsoft's homepage for Windows XP. From here, the reader can find XP overviews, support information and gateways to other areas of Microsoft with more technical information on Windows XP.

www.microsoft.com/technet/prodtechnol/winxppro
Microsoft TechNet provides technical information on Microsoft products. This section is devoted to Windows XP Professional; particularly useful is the Resource Kit (click Resource Kits),

www.winnetmag.com
Publishes technical articles about Windows & .NET ranging from introductions for novice users to in-depth internals descriptions for advanced developers.

www.osronline.com
Features technical articles and tutorials about the internals of Windows operating systems. It offers online articles, as well as a free subscription to *NT Insider,* a technical magazine about the NT line of operating systems.

www.extremetech.com/category2/0,3971,23445,00.asp
ExtremeTech's page for operating systems. It includes news articles, some technical insight and a forum to discuss operating systems.

www.theeldergeek.com/index.htm
Describes Windows XP from the user's perspective, including excellent information about the registry, system services and networking.

www.windowsitlibrary.com
Provides free articles and sells books and articles about Windows operating systems. Some of these articles and books provide technical internals information.

www.sysinternals.com
Dedicated to providing internals information about Windows operating systems, especially the NT line.

msdn.microsoft.com/msdnmag/default.aspx
The *MSDN Magazine* (formerly know as *Microsoft Systems Journal)* contains articles about Microsoft's products, targeted for developers.

www.winsupersite.com
Provides updated information about Windows products. It evaluates them from both technical and user perspectives.

developer.intel.com
Provides technical documentation regarding Intel products, including Pentium and Itanium processors. Their system programmer guides (e.g., for the Pentium 4 processor, developer.intel.com/design/Pentium4/manuals/) provide information about how Intel processors interact with the operating system,

developer.amd.com
Provides technical documentation regarding AMD products, including Athlon and Opteron processors. Their system programmer guides (e.g., for the Opteron processor, www.amd.com/us-en/Processors/DevelopWithAMD/0,,30_2252_739_7044,00.html) provide information about how AMD processors interact with the operating system.

## *Key Terms*

**.NET**—Microsoft initiative aimed at transforming computing from an environment in which users simply execute applications on a single computer to a fully distributed environment; .NET provides a platform- and language-neutral development environment for components that can easily interoperate.

**.NET framework**—Programming model for creating XML-based Web services and applications. The .NET framework supports over 20 programming languages and facilitates application programming by providing libraries to perform common operations (e.g., input/output, string manipulation and network communication) and access data via multiple database interfaces (e.g., Oracle and SQL Server).

**abandoned mutex** (Windows XP)—Mutex that was not released before the termination of the thread that held it.

**access control entry (ACE)** (Windows XP)—Entry in a discretionary access control list (DACL) for a specific resource that contain the security identifier (SID) of a security principal and the type of access (if any) the security principal has to that particular resource.

**access token** (Windows XP)—Data structure that stores security information, such as the security identifier (SID) and group memberships, about a security principal.

**Active Directory** (Windows XP)—Network service that provides directory services for shared objects (e.g., files, printers, services, etc.) on a network.

**ActiveX Controls**—Self-registering COM objects useful for embedding in Web pages.

**affinity mask** (Windows XP)—List of processors on which a process's threads are allowed to execute.

**alertable** I/O (Windows XP)—Type of asynchronous I/O in which the system notifies the requesting thread of the completion of the I/O with an APC.

**alertable *wait* state** (Windows XP)—State in which a thread cannot execute until it is awakened either by an APC entering the thread's APC queue or by receiving a handle to the object (or objects) for which the thread is waiting.

**alternate data stream**—File attribute that stores the contents of an NTFS file that are not in the default data stream. NTFS files may have multiple alternate data streams.

**anonymous pipe** (Windows XP)—Unnamed pipe that can be used only for synchronous one-way communication between local processes.

**apartment model**—COM object threading model in which only one thread acts as a server for each COM object.

**APC IRQL** (Windows XP)-IRQL at which APCs execute and incoming asynchronous procedure calls (APCs) are masked.

**application configuration file (ACF)** (Windows XP)—File that specifies platform-specific attributes (e.g., how data should be formatted) for an RPC function call.

**asynchronous procedure call (APC)** (Windows XP)—Procedure calls that threads or the system can queue for execution by a specific thread.

**automatic binding handle** (Windows XP)—RPC binding handle in which the client process simply calls a remote function, and the stub manages all communication tasks.

**auto-reset timer object** (Windows XP)—Timer object that remains *signaled* only until one thread finishes waiting on the object.

**Bad Page List** (Windows XP)—List of page frames that have generated hardware errors; the system does not store pages on these page frames.

**balance set manager** (Windows XP)—Executive component responsible for adjusting working set minimums and maximums and trimming working sets.

**base priority class** (Windows XP)—Process attribute that determines a narrow range of base priorities the process's threads can have.

**base priority level** (Windows XP)—Thread attribute that describes the thread's base priority within a base priority class.

**binding** (Windows XP)—Connection between the client and the server used for network communication (e.g., for an RPC).

**binding handle** (Windows XP)—Data structure that stores the connection information associated with a binding between a client and a server.

**bus driver**—WDM (Windows Driver Model) driver that provides some generic functions for devices on a bus, enumerates those devices and handles Plug and Play I/O requests.

**cache manager** (Windows XP)—Executive component that handles file cache management.

**class ID (CLSID)**-Globally unique ID given to a COM object class.

**class/miniclass driver pair** (Windows XP)—Pair of device drivers that act as a function driver for a device; the class driver provides generic processing for the device's particular device class, and the miniclass driver provides processing for the specific device.

**clipboard** (Windows XP)—Central repository of data that is accessible to all processes, typically used with copy, cut and paste commands.

**clock IRQL** (Windows XP)—IRQL at which clock interrupts occur and at which APC, DPC/dispatch, DIRQL and profile-level interrupts are masked.

**cluster** (NTFS)—Basic unit of disk storage in an NTFS volume, consisting of a number of contiguous sectors; a system's cluster size can range from 512 bytes to 64KB, but is typically 2KB, 4KB or 8KB.

**COM+**—Extension of COM (Microsoft's Common Object Model) that handles advanced resource management tasks, such as providing support for transaction processing and using thread and object pools.

**commit memory** (Windows XP)—Necessary stage before a process can access memory. The VMM ensures that there is enough space in a pagefile for the memory and creates page table entries (PTEs) in main memory for the committed pages.

**Common Internet File System (CIFS)**—Native file sharing protocol of Windows XP.

**Component Object Model (COM)**—Microsoft-developed software architecture that allows interoperability between diverse components through the standard manipulation of object interfaces.

**component** (Windows XP)—Functional unit of Windows XP. Components range from user-mode applications such as Notepad to portions of kernel space such as the CD-ROM interface and power management tools. Dividing Windows XP into components facilitates operating system development for embedded systems.

**compression unit** (NTFS)—Sixteen clusters. Windows XP compresses and encrypts files one compression unit at a time.

**configuration manager** (Windows XP)—Executive component that manages the registry.

**critical section object** (Windows XP)—Synchronization object, which can be employed only by threads within a single process, that allows only one thread to own a resource at a time.

**data stream** (NTFS)—File attribute that stores the file's content or metadata; a file can have multiple data streams.

**default data stream** (NTFS)—File attribute that stores the primary contents of an NTFS file. When an NTFS file is copied to a file system that does not support multiple data streams, only the default data stream is preserved.

**deferred procedure call (DPC)** (Windows XP)—Software interrupt that executes at the DPC/dispatch IRQL and executes in the context of the currently executing thread.

**device extension** (Windows XP)—Portion of the nonpaged pool that stores information a driver needs to process I/O requests for a particular device.

**device IRQL (DIRQL)** (Windows XP)-IRQL at which devices interrupt and at which APC, DPC/dispatch and

lower DIRQLs are masked; the number of DIRQLs is architecture dependent.

**device object** (Windows XP)—Object that a device driver uses to store information about a physical or logical device.

**direct I/O**—I/O transfer method whereby data is transferred between a device and a process's address space without using a system buffer.

**directory junction** (Windows XP)—Directory that refers to another directory within the same volume, used to make navigating the file system easier.

**discretionary access control list (DACL)** (Windows XP)—Ordered list that tells Windows XP which security principals may access a particular resource and what actions those principals may perform on the resource.

**dispatcher** (Windows XP)—Thread scheduling code dispersed throughout the microkernel.

**dispatcher object** (Windows XP)—Object, such as a mutex, semaphore, event or waitable timer, that kernel and user threads can use for synchronization purposes.

**domain** (Windows XP)—Set of computers that share common resources.

**domain controller** (Windows XP)—Computer responsible for security on a network.

**DPC/dispatch IRQL** (Windows XP)-IRQL at which DPCs and the thread scheduler execute and at which APC and incoming DPC/dispatch level interrupts are masked.

**driver object** (Windows XP)—Object used to describe device drivers; it stores pointers to standard driver routines and to the device objects for the devices that the driver services.

**driver stack**—Group of related device drivers that cooperate to handle the I/O requests for a particular device.

**dynamic-linked library (DLL)** (Windows XP)—Module that provides data or functions to which processes or other DLLs link at execution time.

**dynamic priority class** (Windows XP)—Priority class that encompasses the five base priority classes within which a thread's priority can change during system execution; these classes are idle, below normal, normal, above normal and high.

**environment subsystem** (Windows XP)—User-mode component that provides a computing environment for other user-mode processes; in most cases, only environment subsystems interact directly with kernel-mode components in Windows XP.

**event object** (Windows XP)—Synchronization object that becomes *signaled* when a particular event occurs.

**executive** (Windows XP)—Portion of the Windows XP operating system that is responsible for managing the operating system's subsystems (e.g., I/O subsystem, memory subsystem and file system).

**executive resource lock** (Windows XP)—Synchronization lock available only to kernel-mode threads and that may be held either in shared mode by many threads or in exclusive mode by one thread.

**executive process (EPROCESS) block** (Windows XP)—Executive data structure that stores information about a process, such as that process's object handles and the process's ID; an EPROCESS block also stores the process's KPROCESS block.

**executive thread (ETHREAD) block** (Windows XP)—Executive data structure that stores information about a thread, such as the thread's pending I/O requests and the thread's start address; an ETHREAD block also stores the thread's KTHREAD block.

**explicit handle** (Windows XP)—Binding handle in which the client must specify all binding information and create and manage the handle.

**fast mutex** (Windows XP)—Efficient mutex variant that operates at the APC level with some restrictions (e.g., a thread cannot specify a maximum wait time to wait for a fast mutex).

**fast user switching** (Windows XP)—Ability of a new user to logon to a Windows XP machine without logging off the previous user.

**fiber** (Windows XP)—Unit of execution created by a thread and scheduled by that thread.

**fiber local storage (FLS)** (Windows XP)—Area of a process's address space where a fiber can store data that only the fiber can access.

**file mapping** (Windows XP)—Interprocess communication mechanism in which multiple processes access the same file by placing it in their virtual memory space. The different virtual addresses correspond to the same main memory addresses.

**file mapping object** (Windows XP)—Object used by processes to map any file into memory.

**file view** (Windows XP)—Portion of a file specified by a file mapping object that a process maps into its memory.

**filter driver**—WDM (Windows Driver Model) driver that modifies the behavior of a hardware device (e.g., providing mouse acceleration) or adds some extra services (e.g., security checks).

**free model**—COM (Microsoft's Component Object Model) object threading model in which many threads can act as the server for a COM object.

**Free Page List** (Windows XP)—List of page frames that are available for reclaiming; although these page frames do not contain any valid data, they may not be used until the zero-page thread sets all of their bits to zero.

**function driver**—WDM (Windows Driver Model) device driver that implements a device's main functions; it does most of the I/O processing and provides the device's interface.

**globally unique ID (GUID)**—128-bit integer that is, for all practical purposes, guaranteed to be unique in the world. COM (Microsoft's Component Object Model) uses GUIDs to uniquely identify interfaces and object classes.

**Graphical Identification and Authentication (GINA)** (Windows XP)—Graphical user interface used that prompts users for credentials, usually in the form of a username and password. Windows XP ships with its own implementation of GINA called MSGINA, but it accepts third-party DLLs.

**Hardware Abstraction Layer (HAL)** (Windows XP)—Operating system component that interacts directly with the hardware and abstracts hardware details for other system components.

**high IRQL** (Windows XP)—Highest-priority IRQL, at which machine check and bus error interrupts execute and all other interrupts are masked.

**high-level driver** (Windows XP)—Device driver that abstracts hardware specifics and passes I/O requests to low-level drivers.

**ideal processor** (Windows XP)—Thread attribute that specifies a processor on which the system should attempt to schedule the thread for execution.

**inheritance** (Windows XP)—Technique by which a child process obtains attributes (e.g., most types of handles and the current directory) from its parent process upon creation.

***initialized* state** (Windows XP)—State in which a thread is created.

**I/O completion port** (Windows XP)—Port at which threads register and block, waiting to be awakened when processing completes on an I/O request.

**I/O completion routine** (Windows XP)—Function registered by a device driver with the I/O manager in reference to an IRP; the I/O manager calls this function when processing of the IRP completes.

**I/O request packet (IRP)** (Windows XP)—Data structure that describes an I/O request.

I/O manager (Windows XP)—Executive component that interacts with device drivers to handle I/O requests.

I/O throttling (Windows XP)—Technique that increases stability when available memory is low; the VMM manages memory one page at a time during I/O throttling.

I/O status block (Windows XP)—Field in an IRP that indicates whether an I/O request completed successfully or, if not, the request's error code.

Interface Definition Language (IDL) file (Windows XP)—File that specifies the interfaces that an RPC server exposes.

interface ID (IID)—Globally unique ID for a COM interface.

interlocked singly linked list (SList) (Windows XP)—Singly linked list in which insertions and deletions are performed as atomic operations.

interlocked variable access (Windows XP)—Method of accessing variables that ensures atomic reads and writes to shared variables.

intermediate driver (Windows XP)—Device driver that can be interposed between high- and low-level drivers to filter or process I/O requests for a device.

Internet Connection Firewall (ICF)—Windows XP's packet filtering firewall.

Internetwork Packet eXchange (IPX)—Novell Netware's network protocol designed specifically for LANs.

Interrupt Dispatch Table (IDT)—Kernel data structure that maps hardware interrupts to interrupt vectors.

interrupt request level (IRQL) (Windows XP)—Measure of interrupt priority; an interrupt that occurs at an IRQL equal to or lower than the current IRQL is masked.

interrupt service routine (ISR) (Windows XP)—Function,registered by a device driver, that processes interrupts issued by the device that the driver services.

job object (Windows XP)—Object that groups several processes and allows developers to manipulate and set limits on these processes as a group.

kernel handle (Windows XP)—Object handle accessible from any process's address space, but only in kernel mode.

kernel-mode APC (Windows XP)—APC generated by a kernel-mode thread and queued to a specified user-mode thread; the user-mode thread must process the APC as soon as the user-mode thread obtains the processor.

kernel-mode driver—Device driver that executes in kernel mode.

kernel process (KPROCESS) block (Windows XP)—Kernel data structure that stores information about a process, such as that process's base priority class.

**kernel thread (KTHREAD) block** (Windows XP)-Kernel data structure that stores information about a thread, such as the objects on which that thread is waiting and the location in memory of the thread's kernel stack.

**large page** (Windows XP) — Set of pages contiguous in memory that the VMM treats as a single page.

**last processor** (Windows XP)—Thread attribute equal to the processor that most recently executed the thread.

**lazy allocation—**Policy of waiting to allocate resources, such as pages in virtual memory and page frames in main memory, until absolutely necessary.

**Lempel-Ziv compression algorithm—**Data compression algorithm that NTFS uses to compress files.

**Lightweight Directory Access Protocol (LDAP)** (Windows XP)—Protocol for accessing, searching and modifying Internet directories (e.g., Active Directory).

**localized least-recently used page replacement** (Windows XP)—Policy of moving to disk the least-recently used page of a process at its working set maximum when that process requests a page in main memory. The policy is localized by process, because Windows XP moves only the requesting process's pages; Windows XP approximates this policy using the clock algorithm.

**local procedure call (LPC)** (Windows XP)—Procedure call made by a thread in one process to a procedure exposed by another process in the same domain (i.e., set of computers that share common resources); LPCs can be created only by system components.

**local remote procedure call (LRPCs)** (Windows XP)-RPCs between two processes on the same machine.

**low-level driver** (Windows XP)—Device driver that controls a peripheral device and does not depend on any lower-level drivers.

**mailslot** (Windows XP)—Message queue, which a process can employ to receive messages from other processes.

**mailslot client** (Windows XP)—Process that sends mailslot messages to mailslot servers.

**mailslot server** (Windows XP)—Process that creates a mailslot and receives messages in it from mailslot clients.

**major function code** (Windows XP)—Field in an IRP that describes the general function (e.g., read or write) that should be performed to fulfill an I/O request.

**manual-reset timer object** (Windows XP)—Timer object that remains *signaled* until a thread specifically resets the timer.

**Master File Table (MFT)** (NTFS)-File which is structured as a table in which NTFS stores information (e.g. name, time stamp, and location) about all files in the volume.

**memory descriptor list (MDL)** (Windows XP)—Data structure used in an I/O transfer that maps a process's virtual addresses to be accessed in the transfer to physical memory addresses.

**memory map—**Data structure that stores the correspondence between a process's virtual address space and main memory locations.

**Microsoft IDL (MIDL)—**Microsoft's extension of Interface Definition Language (IDL)—The Open Group's Distributed Computing Environment (DCE) standard for RPC interoperability.

**minor function code** (Windows XP)—Field in an IRP that, together with the major function code, describes the specific function that should be performed to fulfill an I/O request (e.g., to start a device, a PnP major function code is used with a start device minor function code).

**mixed model—**COM threading model, in which some COM objects reside in single apartments and others can be accessed by free threads.

**Modified No-write Page List** (Windows XP)—List of page frames for which the VMM must write an entry to a log file before freeing.

**Modified Page List** (Windows XP)—List of page frames that the VMM must write to the pagefile before freeing.

**Multiple UNC Provider (MUP)** (Windows XP)-File system driver that determines the appropriate redirector to which to send a network I/O request.

**must-succeed request** (Windows XP) —Request for space in main memory that a process issues when it requires more main memory to continue functioning properly. Windows XP always denies must-succeed requests; previous versions of Windows always fulfilled them.

**mutex object** (Windows XP) — Synchronization object that allows at most one thread access to a protected resource at any time; it is essentially a binary semaphore.

**named pipe** (Windows XP)—Type of pipe that can provide bidirectional communication between two processes on local or remote machines and that supports both synchronous and asynchronous communication.

**native API** (Windows XP)—Programming interface exposed by the executive, which environment subsystems employ to make system calls.

**NDIS intermediate driver** (Windows XP)—NDIS driver that can be interposed between a miniport driver and a higher-level driver and adds extra functionality, such as translating packets between different communication media, filtering packets or providing load balancing across several NICs.

**NDIS miniport driver** (Windows XP)—NDIS driver that manages a NIC and sends and receives data to and from the NIC.

**NDIS protocol driver** (Windows XP)—NDIS driver that places data into packets and passes these packets to lower-level drivers. NDIS protocol drivers provide an interface between the transport drivers and other NDIS drivers and can be used as the lowest layer in the implementation of a transport protocol stack such as TCP/IP.

**neither I/O** (Windows XP)—I/O transfer technique in which a high-level driver executes in the context of the calling thread to set up a buffered I/O transfer or direct I/O transfer or to directly perform the transfer in the process's address space.

**NetBIOS Extended User Interface (NetBEUI)** (Windows XP)—Native network and transport protocol for DOS and older Windows operating systems.

**NetBIOS over TCP/IP (NBT)** (Windows XP)-Replacement protocol for NetBEUI that provides backward compatibility with older applications (by keeping the NetBIOS interface) and takes advantage of TCP/IP protocols.

**Network Basic Input/Output System (NetBIOS)** (Windows XP)—API used to support NetBEUI and now used with NBT.

**Network Data Representation (NDR)—**Standard format for network data described in the The Open Group's Distributed Computing Environment (DCE) standard.

**Network Driver Interface Specification (NDIS)** (Windows XP)—Specification that describes a standard interface between lower-level layered drivers in the network driver stack; NDIS drivers provide the functionality of the link layer in the TCP/IP stack and are used on Windows XP systems.

**network redirector—***See* redirect or (Windows XP).

**nonpaged pool—**Area of main memory that stores pages that are never moved to disk.

**nonresident attribute—**NTFS file attribute whose data does not fit inside the MFT entry and is stored elsewhere.

**object handle** (Windows XP)—Data structure that allows threads to manipulate an object.

**Object Linking and Embedding (OLE)—**Microsoft technology built on COM that defines a standard method for processes to exchange data by linking or embedding COM objects.

**object manager** (Windows XP)—Executive component that manages objects.

**object manager namespace** (Windows XP)—Group of object names in which each name is unique in the group.

**opportunistic lock (oplock)** (Windows XP)—Lock used by a CIFS client to secure exclusive access to a remote file, ensuring that the client can cache data locally and keep its cache coherent.

**overlapped I/O** (Windows XP)—Microsoft synonym for asynchronous I/O.

**page directory register—**Hardware register that stores a pointer to the current process's page directory table.

**page directory table** (Windows XP)—4KB page that contains 1024 entries that point to frames in memory.

**paged pool** (Windows XP)—Pages in memory that may be moved to the pagefile on disk.

**pagefile** (Windows XP)—File on disk that stores all pages that are mapped to a process's virtual address space but do not currently reside in main memory.

**page frame database** (Windows XP)—Array that contains the state of each page frame in main memory.

**page list** (Windows XP)—List of page frames that are in the same state. The eight state lists are: the Valid Page List, the Standby Page List, the Modified Page List, the Modified No-write Page List, the Transitional Page List, the Free Page List, the Zeroed Page List, and the Bad Page List.

**page trimming** (Windows XP)—Technique in which Windows XP takes a page belonging to a process and sets the page's PTE to invalid to determine whether the process actually needs the page. If the process does not request the page within a certain time period, the system removes it from main memory.

**passive IRQL** (Windows XP)—IRQL at which user- and kernel-mode normally execute and no interrupts are masked. Passive IRQL is the lowest-priority IRQL.

**periodic timer** (Windows XP)—Waitable timer object that reactivates after a specified interval.

**pipe client** (Windows XP)—Process that connects to an existing pipe to communicate with that pipe's server.

**pipe server** (Windows XP)—Process that creates a pipe and communicates with pipe clients that connect to the pipe.

**PnP I/O requests** (Windows XP)-IRPs generated by the PnP manager to query a driver for information about a device, to assign resources to a device or to direct a device to perform some action.

**Plug and Play (PnP) manager** (Windows XP)—Executive component (which also exists partly in user space) that dynamically recognizes when new devices are added to

the system (as long as these devices support PnP), allocates and deallocates resources to devices and interacts with device setup programs.

**port mapping** (Windows XP)—Process of explicitly allowing the Internet Connection Firewall (ICF) to accept all incoming packets to a particular port.

**power IRQL** (Windows XP)—IRQL at which power failure interrupts execute and at which all interrupts except high-IRQL interrupts are masked.

**power manager** (Windows XP)—Executive component that administers the operating system's power management policy.

**power policy** (Windows XP)—Policy of a system with regard to balancing power consumption and responsiveness of devices.

**primary thread** (Windows XP)—Thread created when a process is created.

**priority inversion**—Situation which occurs when a high-priority thread is waiting for a resource held by a low-priority thread, and the low-priority thread cannot obtain the processor because of a medium-priority thread; hence, the high-priority thread is blocked from execution by the medium-priority thread.

**process environment block (PEB)** (Windows XP)—User-space data structure that stores information about a process, such as a list of DLLs linked to the process and information about the process's heap.

**profile IRQL** (Windows XP)—IRQL at which debugger interrupts execute and at which APC, DPC/dispatch, DIRQL and incoming debugger interrupts are masked.

**protocol sequence** (Windows XP)—String used by a client in an RPC call that specifies the RPC protocol, transport protocol and network protocol for that RPC.

**Prototype Page Table Entry (PPTE)** (Windows XP)-32-bit record that points to a frame in memory that contains either a copy-on-write page or a page that is part of a process's view of a mapped file.

**queued spin lock** (Windows XP)—Spin lock in which a thread releasing the lock notifies the next thread in the queue of waiting threads.

***ready* state** (Windows XP)—State in which a thread is waiting to use a processor.

**real-time priority class** (Windows XP)—Base priority class encompassing the upper 16 priority levels; threads of this class have static priorities.

**recovery key** (NTFS)—Key that NTFS stores that can decrypt an encrypted file; administrators can use recovery keys when a user has forgotten the private key needed to decrypt the file.

**redirector** (Windows XP)—File system driver that interacts with a remote server driver to facilitate network I/O operations.

**registry** (Windows XP)—Central database in which user, system and application configuration information is stored.

**Remote Access Service (RAS)** (Windows XP)—Network service, which allows users to remotely connect to a LAN.

**reparse point** (NTFS)—File attribute containing a tag and a block of up to 16KB of data that a user associates with a file or directory; when an application accesses a reparse point, the system executes the file system filter driver specified by the reparse point's tag.

**request IRQL** (Windows XP)—IRQL at which interprocess interrupts execute and all interrupts except power-level and high-level interrupts are masked.

**reserve memory** (Windows XP)—To indicate to the VMM that a process intends to use an area of virtual memory; the VMM allocates memory in the process's virtual address space but does not allocate any page frames in main memory.

**resident attribute** (NTFS)—File attribute whose data is stored within the MFT entry.

***running* state** (Windows XP)—State in which a thread is currently in execution.

**Security Accounts Manager (SAM)** (Windows XP)—Database that administers information about all security principals in the system. It provides services such as account creation, account modification and authentication.

**security descriptor** (Windows XP)—Data structure that stores information about which security principals may access a resource and what actions they may perform on that resource. The most important element of a security descriptor is its discretionary access control list (DACL).

**security identifier (SID)** (Windows XP)—Unique identification number assigned to each security principal in the system.

**security principal** (Windows XP)—Any user, process, service or computer that can perform an action in a Windows XP system.

**Security Support Provider Interface (SSPI)** (Windows XP)—Microsoft's standardized protocol for authentication and authorization recognized by both the Kerberos and NTLM authentication services.

**semaphore object** (Windows XP)—Synchronization object that allows a resource to be owned by up to a specified number of threads; it is essentially a counting semaphore.

**Sequenced Packet eXchange (SPX)—**Novell Netware's transport protocol, which offers connection-oriented services for IPX packets.

**Server Message Block (SMB)—**Network file-sharing protocol used in Windows operating systems on top of which CIFS (Common Internet File System) is built.

**single-use timer** (Windows XP)—Waitable timer object that is used once and then discarded.

***signaled* state** (Windows XP)—State in which a synchronization object can be placed, allowing one or more threads *waiting* on this object to awaken.

**sparse file** (NTFS)—File with large blocks of regions filled with zeroes that NTFS tracks using a list of empty regions rather than explicitly recording each zero bit.

**Standby Page List** (Windows XP)—List of page frames that are consistent with their on-disk version and can be freed.

***standby* state** (Windows XP)—Thread state denoting a thread that has been selected for execution.

**system service** (Windows XP)—Process that executes in the background whether or not a user is logged into the computer and typically executes the server side of a client/server application; a system service for Windows XP is similar to a daemon for Linux.

**system worker thread** (Windows XP)—Thread controlled by the system that sleeps until a kernel-mode component queues a work item for processing.

***terminated* state** (Windows XP)—Thread state that denotes a thread is no longer available for execution.

**thread local storage (TLS)** (Windows XP)—Area of a thread's process's address space where the thread can store private data, inaccessible to other threads.

**thread environment block (TEB)** (Windows XP)—User-space data structure that stores information about a thread, such as the critical sections owned by a thread and exception-handling information.

**thread pool** (Windows XP) — Collection of worker threads that sleep until a request is queued to them, at which time one of the threads awakens and executes the queued function.

**timer-queue timer** (Windows XP)—Timer that signals a worker thread to perform a specified function at a specified time.

**transitional fault** (Windows XP)—Fault issued by the MMU when it tries to access a page that is in main memory, but whose page frame's status is set to standby, modified, or modified no-write.

**Transitional Page List** (Windows XP)—List of page frames whose data is in the process of being moved to or from disk.

***transition* state** (Windows XP)—Thread state denoting a thread that has completed a wait but is not yet ready to run because its kernel stack has been paged out of memory.

**Transport Driver Interface (TDI) driver** (Windows XP)-Network driver that exports a network interface to upper-level kernel-mode clients and interacts with low-level NDIS drivers.

**universally unique identifier (UUID)—**ID that is guaranteed, for all practical purposes, to be unique in the world. UUIDs uniquely identify an RPC interface.

**Uniform Naming Convention (UNC) format** (Windows XP) — Format that specifies a file's pathname, including on which server and in which directory on that server the file is located.

***unknown* state** (Windows XP)—Thread state denoting that the some error has occurred and the system does not know the state of the thread.

***unsignaled* state** (Windows XP)—State in which a synchronization object can be; threads *waiting* on this object do not awaken until the object transitions to the *signaled* state.

**user-mode APC** (Windows XP)—APC queued by user-mode thread and executed by the target thread when the target thread enters an alertable *wait* state.

**user-mode driver** (Windows XP)—Device driver that executes in user space.

**Valid Page List** (Windows XP)—List of page frames that are currently in a process's working set.

**Virtual Address Descriptor (VAD)** (Windows XP)—Structure in memory that contains a range of virtual addresses that a process may access.

**virtual memory manager (VMM)** (Windows XP)—Executive component that manages virtual memory.

**VMS—**Operating system for the DEC VAX computers, designed by David Cutler's team.

**waitable timer object** (Windows XP) — Synchronization object that becomes *signaled* after a specified amount of time elapses.

**wait function** (Windows XP)—Function called by a thread to wait for one or more dispatcher objects to enter a *signaled* state; calling this function places a thread in the *waiting* state.

***waiting* state** (Windows XP)—Thread state denoting that a thread is not ready for execution until it is awakened (e.g.. by obtaining a handle to the object on which it is waiting).

**Web-based Distributed Authoring and Versioning (WebDAV)** (Windows XP)—Network file-sharing protocol that allows users to write data directly to HTTP servers and is

designed to support collaborative authoring between groups in remote locations.

**Win32 environment subsystem** (Windows XP)—User-mode process interposed between the executive and the rest of user space that provides a typical 32-bit Windows environment.

**Win32 service** (Windows XP)—*See* system service (Windows XP).

**Windows Driver Model (WDM)** (Windows XP)-Standard driver model that enables source-code compatibility across all Windows platforms; each WDM driver must be written as a bus driver, function driver or filter driver and support PnP, power management and WML

**Windows Management Instrumentation (WMI)** (Windows XP)—Standard that describes how drivers provide measurement and instrumentation data to users (e.g., configuration data, diagnostic data or custom data) and allow user applications to register for WMI driver-defined events.

**Windows XP Embedded—**Embedded operating system, which uses the same binary files as Windows XP but allows designers to choose only those components applicable for a particular device.

**WinlNet** (Windows XP)—API that allows applications to interact with FTP, HTTP and Gopher protocols to access resources over the Internet.

**WinHTTP** (Windows XP)—API that supports communication between clients and servers over an HTTP session.

**Windows sockets version 2 (Winsock 2)—**Adaptation and extension of BSD sockets for the Windows environment.

**worker thread** (Windows XP)—Kernel-mode thread that the system uses to execute functions queued by user-mode or other kernel-mode threads.

**working set** (Windows XP)—All of the pages in main memory that belong to a specific process.

**working set maximum** (Windows XP)—Upper limit on the number of pages a process may have simultaneously in main memory.

**working set minimum** (Windows XP)—Number of pages in main memory the working set manager leaves a process when it executes the page-trimming algorithm.

**write-through mode** (Windows XP)—Method of writing to a pipe whereby write operations do not complete until the data being written is confirmed to be in the buffer of the receiving process.

**Zeroed Page List** (Windows XP)—List of page frames whose bits are all set to zero.

## *Exercises*

**21.1** *[Section 21.4, System Architecture]* What aspects of the Windows XP architecture make the system modular and portable to different hardware? In what ways has Microsoft sacrificed modularity and portability for performance?

**21.2** *[Section 21.5.2, Object Manager]* What are the benefits of managing all objects in a centralized object manager?

**21.3** *[Section 21.5.5, Deferred Procedure Calls (DPCs)]* Why might a routine that generates a DPC specify the processor (in a multiprocessor system) on which the DPC executes?

**21.4** *[Section 21.5.5, Deferred Procedure Calls (DPCs)]* How do DPCs increase the system's responsiveness to incoming device interrupts?

**21.5** *[Section 21.5.5, Deferred Procedure Calls (DPCs)]* Why is it that DPCs cannot access pageable data?

**21.6** *[Section 21.6.1, Process and Thread Organization]* Give an example of why it may be useful to create a job for a single process.

21.7 *[Section 21.6.1, Process and Thread Organization]* What entity schedules fibers?

21.8 *[Section 21.6.1, Process and Thread Organization]* How might thread pools introduce inefficiency?

21.9 *[Section 21.6.2, Thread Scheduling]* The dispatcher schedules each thread without regard to the process to which the thread belongs, meaning that, all else being equal, the same process implemented with more threads receives a greater share of execution time. Name a disadvantage of this strategy.

21.10 *[Section 21.6.2, Thread Scheduling]* Why does the system reset a real-time thread's quantum after preemption?

21.11 *[Section 21.6.2, Thread Scheduling]* Under what circumstances would Windows XP increase a dynamic-priority thread's priority? Under what circumstances would Windows XP decrease it?

21.12 *[Section 21.6.2, Thread Scheduling]* How can policies implemented in Windows XP lead to priority inversion? How does Windows XP handle priority inversion? In what ways is this a good policy? What are its drawbacks?

**21.13** *[Section 21.6.2, Thread Scheduling]* Why might a developer manipulate the value of a thread's ideal processor?

**21.14** *[Section 21.6.3, Thread Synchronization]* Windows XP does not explicitly detect deadlock situations for threads using dispatcher objects. How might threads using dispatcher objects create a deadlock? What mechanisms does Windows XP include to help avoid these situations?

**21.15** *[Section 21.6.3, Thread Synchronization]* In what ways are threads executing at an IRQL equal to or greater than DPC/dispatch level restricted? Why is this so?

**21.16** *[Section 21.6.3, Thread Synchronization]* Why use a queued spin lock in preference to a generic spin lock?

**21.17** *[Section 21.6.3, Thread Synchronization]* Explain how an executive resource can be used to solve the readers-and-writers problem.

**21.18** *[Section 21.7.1, Memory Organization]* Suppose an enterprising programmer modifies the Windows XP virtual memory manager so that it allocates space for all page table entries that a process may need as soon as the process is created. Assume that page table entries cannot be moved to secondary storage and that no page table entries are shared.

a. How much space would be needed to store all the page table entries for one process (on a 32-bit system)?

b. Windows XP stores each process's page table entries in system space, which is shared between all processes. If a system devoted all of its system space to page table entries, what is the maximum number of processes that could be active simultaneously?

**21.19** *[Section 21.7.1, Memory Organization]* Large pages need to be stored contiguously in main memory. There are several ways to do this: a system can designate a portion of main memory exclusively for large pages, a system can rearrange pages in main memory whenever a process requests a large page, or a system can do nothing and deny large-page memory allocation requests whenever necessary. Discuss the pros and cons of each of these three policies.

**21.20** *[Section 21.7.1, Memory Organization]* A **Windows XP** TLB entry contains the full 32-bit virtual address and the 32-bit physical address to which it corresponds. Associative memory is very expensive. Suggest a space-saving optimization for the TLB.

**21.21** *[Section 21.7.2, Memory Allocation]* When a process opens a file, Windows XP does not move the entire file into main memory. Instead, the system waits to see which portions of the file the process will access. The VMM moves only the accessed pages into main memory and creates PTEs for them. This creates extra overhead whenever the process accesses a new portion of an open file. What would happen if the operating system tried to save time by creating all PTEs at once?

**21.22** *[Section 21.7.2, Memory Allocation]* Windows XP eliminated must-succeed requests to make the system more stable. Suppose an enterprising programmer rewrote the operating system to accept must-succeed requests, but only when the system had enough main memory to fulfill the requests. What are the pitfalls of this policy?

**21.23** *[Section 21.7.3, Page Replacement]* Why does Windows XP enable pagefiles to be stored on separate disks?

**21.24** *[Section 21.7.3, Page Replacement]* Why does Windows XP zero pages?

**21.25** *[Section 21.7.3, Page Replacement]* Suppose process A requests a new page from disk. However, the Zeroed Page List and the Free Page List are empty. One page frame originally belonging to process B is in the Standby Page List, and one page frame belonging to process A is in the Modified Page List. Which page frame will the system take and what will happen to that page frame? What are the pros and cons of taking the other page frame?

**21.26** *[Section 21.8.1, File System Drivers]* The operating system uses different local file system drivers to access different storage devices. Would it make sense to develop a remote block file system driver and remote character file system driver to access different types of storage devices on remote computers?

**21.27** *[Section 21.8.2, NTFS]* In Windows XP, shortcuts (the icons that users put on their desktop) are implemented as soft links—if the file or program to which they are connected is moved, the shortcuts point to nothing. What are the advantages and disadvantages of replacing shortcuts with hard links?

**21.28** *[Section 21.8.2, NTFS] A* user wants to compress a 256KB file stored on an NTFS volume with 4KB clusters. Lempel-Ziv compression reduces the four 64KB compression units to 32KB, 31KB, 62KB, and 48KB. How much space does the compressed file take on the disk?

**21.29** *[Section 21.8.2, NTFS]* Windows XP performs file encryption using reparse points. The file is piped through the Encrypting File System filter, which does all of the encryption and decryption. Windows XP allows an application to ignore a reparse point and access the file directly. Does this compromise the security of encrypted data? Why or why not?

**21.30** *[Section 21.9.1, Device Drivers]* What are the advantages of servicing device I/O requests with a clearly defined driver stack rather than a single device driver?

**21.31** *[Section 21.9.1, Device Drivers]* What are some of the benefits that systems and users realize from Plug and Play?

**21.32** *[Section 21.9.2, Input/Output Processing]* We discussed four types of asynchronous I/O: polling, waiting on an event object that signals the completion of the I/O, alertable I/O and using I/O completion ports. For each technique, describe a situation for which it is useful; also, state a drawback for each technique.

**21.33** *[Section 21.9.3, Interrupt Handling]* Why should an ISR do as little processing as necessary and return quickly, saving most of the interrupt processing for a DPC?

**21.34** *[Section 21.11.2, Network Driver Architecture]* Suppose a new set of protocols were to replace TCP and IP as standard network protocols. How difficult would it be to incorporate support for these new protocols into Windows XP?

**21.35** *[Section 21.11.3, Network Protocols]* Winsock adds new functionality on top of BSD sockets, extending sockets' functionality, but hindering portability. Do you think this was a good design decision? Support your answer.

**21.36** *[Section 21.12.1, Symmetric Multiprocessing (SMP)]* Why does Windows XP attempt to schedule a thread on the same processor on which it recently executed?

**21.37** *[Section 21.13.1, Authentication]* Suppose a company network has only Windows XP machines. What are the advantages of allowing both NTLM and Kerberos authentication? What are the disadvantages?

**21.38** *[Section 21.13.2, Authorization]* When Windows XP compares an access token to a DACL (discretionary access control list), it scans the ACEs (access control entries) one by one and stops when it finds the first ACE whose SID (security identifier) matches an SID in the access token. Consider an administrator that wants to permit all authenticated users except interns to use a printer. The first ACE in the printer's DACL would deny the security group Interns and the second ACE would permit the security group Everyone. Assume that an intern's access token lists the security group Everyone first and the security group Interns second. What would happen if instead the system scanned the access token and stopped when it found an SID in the DACL?

**21.39** *[Section 21.13.3, Internet Connection Firewall]* The Windows XP ICF filters only inbound packets. What are the advantages of not checking outgoing packets? What are the disadvantages?

## Recommended Reading

A superb exposition of Windows NT-line internals can be found in *Inside Windows 2000,* 3d ed., by Mark Russinovich and David Solomon and published by Microsoft Press. These authors also discuss kernel enhancements from Windows 2000 to Windows XP in their article "Windows XP: Kernel Improvements Create a More Robust, Powerful, and Scalable OS," published in the December 2001 edition of *MSDN Magazine.* Helen Custer's *Inside Windows NT,* published by Microsoft Press, introduces many NT concepts still applicable for Windows XP. For the most updated reference on Windows XP, see the platform Software Development Kit (SDK) and Driver Development Kit (DDK) for Windows XP. Both are available from the *MSDN Library* at msdn.microsoft.com/library. Also, *Microsoft Windows XP Professional Resource Kit Documentation* by the Microsoft Corporation and published by Microsoft Press provides useful documentation about Windows XP. Some of the chapters of this book are available on *Microsoft Technet* at www.microsoft.com/technet/prodtechnol/winxppro/Default.asp. The *NT Insider* and *Windows & .NET Magazine* both provide excellent articles on Windows internals. Both of these magazines have archives of old editions. The *NT Insider* archives are available at www.osronline.com, and the *Windows & .NET Magazine* archives are available at www.wi nntmag.com.

## Works Cited

1. "Windows XP Captures More Than One-Third of O/S Market on Web," *StatMarket* 13, May 2003, <www.statmarket.com/cgi-bin/sm.cgi?sm&feature&week_stat>.
2. "Windows XP Product Information," Microsoft Corporation, July 29, 2003, <www.microsoft.com/windowsxp/evaluation/default.asp>.
3. Connolly, C, "First Look: Windows XP 64-Bit Edition for AMD64," *Game PC* September 5, 2003 <www.gamepc.com/labs/view_content.asp?id=amd64xp&page=l>.
4. Mirick, John, "William H. Gates III," Department of Computer Science at Virginia Tech, September 29, 1996, <ei.cs.vt.edu/-history/Gates.Mirick.html>.

5. Corrigan, Louis, "Paul Allen's Wide Wild Wired World," *The Motley Fool,* June 17, 1999 <www.fool.com/specials/1999/sp990617allen.htm>.
6. *Forbes, "Forbes'* World's Richest People 2003," 2003, <www.forbes.com/finance/lists/10/2003/LIR.j html?passLi stId=10&passYear=2003&passListType=Person&uniqueId=BH69&datatype=Person>.
7. "Bill Gates," *Microsoft,* September 2002, <www.microsoft.com/billgates/bio.asp>.
8. Krol, Louisa, and Leo Goldman, "Survival of the Richest," *Forbes.com,* March 17, 2003, <www.forbes.com/free_forbes/2003/0317/087.html>.
9. "MS-DOS," *Computer Knowledge,* 2001, <www.cknow.com/ckinfo/acro_m/msdos_1.shtml>.
10. Microsoft Corporation, "Bill Gates' Web Site," September 2003, <www.microsoft.com/billgates/bio.asp>.
11. Microsoft Corporation, "Bill Gates' Web Site," September 2003, <www.microsof t.com/billgates/bio.asp>.
12. Allison, D., "Interview with Bill Gates," 1993, <www.americanhi story.si.edu/csr/comphist/gates.htm>.
13. Allison, D., "Interview with Bill Gates," 1993, <www.americanhi story.si.edu/csr/comphist/gates.htm>.
14. Gates, W., *The Road Ahead,* New York: Viking Penguin, 1995, pp. 1, 12, 14-17, 48, 54.
15. Microsoft Corporation, "Bill Gates' Web Site," September 2003, <www.microsoft.com/billgates/bio.asp>.
16. Allison, D., "Interview with Bill Gates," 1993, <www.americanhistory.si .edu/csr/comphist/gates.htm>.
17. Gates, W, *The Road Ahead,* New York: Viking Penguin, 1995, pp. 1, 12, 14-17, 48, 54.
18. Allison, D., "Interview with Bill Gates," 1993, <www. ameri canhistory.si.edu/csr/comphist/gates.htm>.
19. Gates, W, *The Road Ahead,* New York: Viking Penguin, 1995, pp. 1, 12, 14-17, 48, 54.
20. Microsoft Corporation, "Bill Gates' Web Site," September 2003, <www.microsoft.com/billgates/bio.asp>.
21. Allison, D., "Interview with Bill Gates," 1993, <www.americanhi story.si.edu/csr/comphist/gates.htm>.
22. Gates, W, *The Road Ahead,* New York: Viking Penguin, 1995, pp. 1,12,14-17,48,54.
23. Allison, D., "Interview with Bill Gates," 1993, <www.americanhi story.si.edu/csr/comphist/gates.htm>.
24. Gates, W, *The Road Ahead,* New York: Viking Penguin, 1995, pp. 1,12,14-17,48,54.
25. Allison, D., "Interview with Bill Gates," 1993, <www. ameri canhistory.si.edu/csr/comphist/gates.htm>.
26. Allison, D., "Interview with Bill Gates," 1993, <www.americanhistory.si.edu/csr/comphist/gates.htm>.
27. Gates, W, *The Road Ahead,* New York: Viking Penguin, 1995, pp. 1, 12, 14-17, 48, 54.
28. Microsoft Corporation, "PressPass—Steve Ballmer," <www.microsoft.com/presspass/exec/steve/default.asp>.
29. Gates, W., *The Road Ahead,* New York: Viking Penguin, 1995, pp. 1, 12, 14-17, 48, 54.
30. Microsoft Corporation, "Bill Gates' Web Site," September 2003, <www.microsoft.com/billgates/bio.asp>.
31. "Microsoft DOS Version Features," *EMS,* <www.emsps.com/oldtools/msdosv.htm>.
32. "History of Microsoft Windows," *Wikipedia,* <www.wikipedia.org/wiki/History_of_Microsoft_Windows>.
33. "Windows Desktop Products History," Microsoft Corporation, 2003, <www.microsoft.com/windows/WinHistoryDesktop.mspx>.
34. "History of Microsoft Windows," *Wikipedia,* <www.wikipedia.org/wiki/History_of_Microsoft_Windows>.
35. Munro, Jay, "Windows XP Kernel Enhancements," *ExtremeTech,* June 8, 2001, <www.extremetech.com/print_article/0,3998,a=2473,00.asp>.
36. "The Foundations of Microsoft Windows NT System Architecture," *MSDN Library,* Microsoft Corporation, September 1997. <msdn.microsoft.com/ARCHIVE/en-us/dnarwbgen/html/msdn_ntfound.asp>.
37. "Microsoft Windows 2000 Overview," *Dytech Solutions,* Microsoft Corporation, 1999, <www.dytech.com.au/technologies/Win2000/Features/Features.asp>.
38. Russinovich, M., "Windows NT and VMS: The Rest of the Story," December 1998, <www.winnetmag.com/Articles/Index.cfm?ArticleID=4494>.
39. Custer, H, *Inside Windows NT,* Microsoft Press, 1992. Foreword by David Cutler.
40. Russinovich, M., "Windows NT and VMS: The Rest of the Story," December 1998, <www.winnetmag.com/Articles/Index.cfm?ArticleID=4494>.
41. Custer, H, *Inside Windows NT,* Microsoft Press, 1992. Foreword by David Cutler.
42. Custer, H, *Inside Windows NT,* Microsoft Press, 1992. Foreword by David Cutler.
43. Russinovich, M., "Windows NT and VMS: The Rest of the Story," December 1998, <www.winnetmag.com/Articles/Index.cfm?ArticleID=4494>.
44. Custer, H., *Inside Windows NT,* Microsoft Press, 1992. Foreword by David Cutler.
45. Custer, H, *Inside Windows NT,* Microsoft Press, 1992. Foreword by David Cutler.
46. Russinovich, M., "Windows NT and VMS: The Rest of the Story," December 1998, <www.winnetmag.com/Articles/Index.cfm?ArticleID=4494>.
47. Custer, H., *Inside Windows NT,* Microsoft Press, 1992. Foreword by David Cutler.
48. Russinovich, M., "Windows NT and VMS: The Rest of the Story." December 1998, <www.winnetmag.com/Articles/Index.cfm?ArticleID=4494>.
49. Custer, H, *Inside Windows NT,* Microsoft Press, 1992. Foreword by David Cutler.

50. Custer, H., *Inside Windows NT,* Microsoft Press, 1992. Foreword by David Cutler.
51. Russinovich, M., "Windows NT and VMS: The Rest of the Story," December 1998, <www.winnetmag.com/Articles/Index.cfm?ArticleID=4494>.
52. Custer, H., *Inside Windows NT,* Microsoft Press, 1992. Foreword by David Cutler.
53. Russinovich, M., "Windows NT and VMS: The Rest of the Story," December 1998, <www.winnetmag.com/Articles/Index.cfm?ArticleID=4494>.
54. J. Munro. "Windows XP Kernel Enhancements," *ExtremeTech,* June 8, 2001, <www.extremetech.com/print_article/0,3998,a=2473,00.asp>.
55. IBM, "Year 1987," <www-1.ibm.com/ibm/history/history/year_1987.html>.
56. Both, D, "A Short History of OS/2," December 18, 2002, <www.millennium-technology.com/History0fOS2.html>.
57. Necasek, M., "OS/2 1," <pages.prodigy.net/michaln/history/os210/index.html>.
58. Necasek, M., "OS/2 2," <pages.prodigy.net/michaln/history/os220/index.html>.
59. Both, D., "A Short History of OS/2," December 18, 2002, <www.millennium-technology.com/History0f0S2.html>.
60. "The Foundations of Microsoft Windows NT System Architecture," *MSDN Library,* Microsoft Corporation, September 1997, <msdn.microsoft.com/ARCHIVE/en-us/dnarwbgen/html/msdn_ntfound.asp>.
61. "Windows NT C2 Evaluations," *Microsoft TechNet,* Microsoft Corporation, December 2, 1999, <www.microsoft.com/technet/treeview/default.asp?url=/technet/secu ri ty/news/c2summ.asp>.
62. "Fast Boot/Fast Resume Design," Microsoft Corporation, June 12, 2002, <www.microsoft.com/whdc/hwdev/platform/performance/fastboot/default.mspx>.
63. "MS Windows NT Kernel-Mode User and GDI White Paper," *Microsoft TechNet,* viewed July 22, 2003, <www.microsoft.com/technet/prodtechnol/ntwrkstn/evaluate/featfunc/kernelwp.asp>.
64. "DDK Glossary-Hardware Abstraction Layer (HAL)," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/gloss/hh/gloss/glossary_01ix.asp>.
65. Walla, M., and R. Williams, "Windows .NET Structure and Architecture," *Windows IT Library,* April 2003, <www.windowsitlibrary.com/Content/717/02/1.html >.
66. Solomon, D, and M. Russinovich, *Inside Windows 2000,* 3rd ed., Redmond: Microsoft Press, 2000.
67. "MS Windows NT Kernel-Mode User and GDI White Paper," *Microsoft TechNet,* viewed July 22, 2003, <www.microsoft.com/technet/prodtechnol/ntwrkstn/evaluate/featfunc/kernelwp.asp>.
68. "Windows 2000 Architectural Tutorial," FindTutorials, viewed July 10, 2003, <tutorials.findtutorials.com/read/category/97/id/379>.
69. Walla, M., and R. Williams, "Windows .NET Structure and Architecture," *Windows IT Library,* April 2003, <www.windowsitlibrary.com/Content/717/02/1.html >.
70. S. Schreiber, *Undocumented Windows 2000 Secrets,* Boston: Addison-Wesley, 2001.
71. Walla, M., and R. Williams, "Windows .NET Structure and Architecture," *Windows IT Library,* April 2003, <www.windowsitlibrary.com/Content/717/02/1.html>.
72. "Windows XP 64-Bit Frequently Asked Questions," March 28, 2003, <www.microsoft.com/windowsxp/64bit/evaluation/faq.asp>.
73. "Dynamic-Link Libraries," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/dynamic_link_libraries.asp>.
74. "System Services Overview," *Microsoft TechNet,* viewed July 25, 2003, <www.microsoft.com/TechNet/prodtechnol/winxppro/proddocs/sys_srv_overview_01.asp>.
75. Foley, J., "Services Guide for Windows XP," *The Elder Geek,* viewed July 25, 2003 <www.theeldergeek.com/services_guide.htm#Services>.
76. "Registry," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/sysinfo/base/regi s-try.asp>.
77. "Chapter 23—Overview of the Windows NT Registry," *Microsoft TechNet,* viewed July 21, 2003, <www.microsoft.com/technet/prodtechnol/ntwrkstn/reskit/23_regov.asp>.
78. "Structure of the Registry," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/sysinfo/base/structure_of_the_registry.asp>.
79. "Predefined Keys," *MSDN Library,* February 2003, <msdn.microsoft.com/en-us/sysinfo/base/predefined_keys.asp>.
80. "Predefined Keys," *MSDN Library,* February 2003, <msdn.microsoft.com/en-us/sysinfo/base/predefined_keys.asp>.
81. "Opening, Creating, and Closing Keys," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/sysinfo/base/opening_creating_and_closing_keys.asp>.
82. "Registry Hives," *MSDN Library,* February 2003, <msdn.microsoft.com/1ibrary/en-us/sysinfo/base/regi stry_hives.asp>.
83. Russinovich, M., "Inside the Registry," *Windows & .NET Magazine,* May 1999 <www.win2000mag.com/Articles/Index.cfm?ArticleID=5195>.
84. "Object Management," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/objects_033b.asp>.
85. Russinovich, M., "Inside NT's Object Manager," *Windows & .NET Magazine,* October 1997, <www.winnetmag.com/Articles/Index.cfm?IssueID=24&ArticleID=299>.
86. "Object Categories," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/sysinfo/base/object_categori es.asp>.

87. "Object Handles," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/objects_5btz.asp>.
88. "Handle Inheritance," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/sysinfo/base/handle_i nheritance.asp>.
89. "Object Handles," *MSDN Library,* June 6, 2003 <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/objects_5btz.asp>.
90. "Object Names," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/objects_9kl3.asp>.
91. "Object Manager," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/sysinfo/base/object_manager.asp>.
92. Russinovich, M., "Inside NT's Object Manager," *Windows & .NET Magazine,* October 1997, <www.winnetmag.com/Articles/Index.cfm?IssueID=24&ArticleID=299>.
93. "Object Manager," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/sysinfo/base/object_manager.asp>.
94. "Life Cycle of an Object," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/objects_16av.asp>.
95. "IRQL," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/kl01_62b6.asp>.
96. "Managing Hardware Priorities," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_0kh3.asp>.
97. "Key Benefits of the I/O APIC," *Windows Hardware and Driver Central,* <www.microsoft.com/whdc/hwdev/platform/proc/IO-APIC.mspx>.
98. "Managing Hardware Priorities," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_0kh3.asp>.
99. "Windows DDK Glossary—Interrupt Dispatch Table (IDT)," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/gloss/hh/gloss/glossary_lqbd.asp>.
100. "Windows DDK Glossary—Interrupt Dispatch Table (IDT)," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/gloss/hh/gloss/glossary_lqbd.asp>.
101. "Managing Hardware Priorities," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_0kh3.asp>.
102. "Asynchronous Procedure Calls," *NT Insider,* Vol. 5, No. 1, January 1998,<www.osr.com/ntinsider/1998/apc.htm>.
103. "Asynchronous Procedure Calls," *NT Insider,* Vol. 5, No. 1, January 1998, <www.osr.com/ntinsider/1998/apc.htm>.
104. "QueueUserAPC," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/queueuserapc.asp>.
105. "Alterable I/O," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/alertable_i_o. asp>.
106. "Asynchronous Procedure Calls," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/asynchronous_procedure_calls.asp>.
107. Hadjidoukas, P. E., "A Device Driver for W2K Signals," *Windows Developer Network,* September 7, 2003, <www.windevnet.com/documents/s=7637/wdj0108a/0108a.htm?_temp=evgyijpt0a>.
108. "Asynchronous Procedure Calls," *NTInsider,Vol.* 5, No. 1, January 1998,<www.osr.com/_ntinsider/1998/apc.htm>.
109. "Introduction to DPCs," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_2zzb.asp>.
no. "Managing Hardware Priorities," *MSDN Library,* June 6, 2003. <msdn.mi crosoft.com/library/en-us/kmarch/hh/kmarch/intrupts_0kh3.asp>.
in. "Writing an ISR," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_4ron.asp>.
112. "Introduction to DPC Objects," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_0m1z.asp>.
113. Russinovich, M, "Inside NT's Interrupt Handling," *Windows & .NET Magazine,* November 1997, <www.winntmag.com/Articles/Print.cfm?ArticleID=298>.
114. "Stop 0x0000000A or IRQL_NOT_LESS_OR_EQUAL," *Microsoft TechNet,* viewed July 17, 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prmd_stp_hwpg.asp>.
115. "Guidelines for Writing DPC Routines," *MSDN Library,* <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_6zon.asp>.
116. Russinovich, M, "Inside NT's Interrupt Handling," *Windows & .NET Magazine,* November 1997, <www.winntmag.com/Articles/Print.cfm?ArticleID=298>.
117. "KeSetTargetProcessorDPC," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/kl05_9ovm.asp>.
118. "KeSetImportanceDPC," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/kl05_2pf6.asp>.
119. "I've Got Work To Do—Worker Threads and Work Queues," AT *Insider,* Vol. 5, No. 5, October 1998 (updated August 20, 2002) <www.osronline.com/article.cfm?i d=65>.
120. "Device-Dedicated Threads," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/synchro_9blj.asp>.
121. "Managing Hardware Priorities," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_0kh3.asp>.

122. "I've Got Work To Do—Worker Threads and Work Queues," *NT Insider,* Vol. 5, No. 5, October 1998 (updated August 20, 2002) <www.osronline.com/article.cfm?id=65>.
123. "System Worker Threads," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/en-us/kmarch/hh/kmarch/synchro_9ylz.asp>.
124. "Processes and Threads," *MSDN Library,* <msdn.microsoft.com/library/en-us/dllproc/base/about_processes_and_threads.asp>.
125. "Creating Processes," *MSDN Library,* <msdn.microsoft.com/library/en-us/dllproc/base/creating_processes.asp>.
126. "Event Objects," *MSDN Library,* <msdn.microsoft.com/library/en-us/dllproc/base/event_objects.asp>.
127. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed., Redmond: Microsoft Press, 2000.
128. Schreiber, S., *Undocumented Windows 2000 Secrets,* Boston: Addison-Wesley, 2001.
129. Schreiber, S., *Undocumented Windows 2000 Secrets,* Boston: Addison-Wesley, 2001, p. 416.
130. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed, Redmond: Microsoft Press, 2000, p. 291.
131. Schreiber, S., *Undocumented Windows 2000 Secrets,* Boston: Addison-Wesley, 2001, p. 429.
132. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed, Redmond: Microsoft Press, 2000, p. 291.
133. Schreiber, S., *Undocumented Windows 2000 Secrets,* Boston: Addison-Wesley, 2001, p. 416.
134. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed, Redmond: Microsoft Press, 2000, p. 319.
135. Schreiber, S., *Undocumented Windows 2000 Secrets,* Boston: Addison-Wesley, 2001, p. 419.
136. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed, Redmond: Microsoft Press, 2000, pp. 320-321.
137. Schreiber, S., *Undocumented Windows 2000 Secrets,* Boston: Addison-Wesley, 2001, p. 429.
138. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed, Redmond: Microsoft Press, 2000, p. 328.
139. "Thread Local Storage," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/thread_local_storage.asp>.
140. "Creating Processes," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/creating_processes.asp>.
141. "Inheritance," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/inheritance.aspx
142. "About Processes and Threads," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/about_processes_and_threads.asp>.
143. "Terminating a Process," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/terminating_a_process.asp>.
144- Richter, J., "Make Your Windows 2000 Processes Play Nice Together with Job Kernel Objects," *Microsoft Systems Journal,* March 1999, <www.microsoft.com/msj/0399/jobkernelobj/jobkernelobj.aspx>.
145. "JobObject_Basic_Limit_Information," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/jobobject_basic_limit_information_str.asp>.
146. "SetInformationJobObject," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/setinformationjobobject.asp>.
147. "Job Objects," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/job_objects.asp>.
148. Richter, J., "Make Your Windows 2000 Processes Play Nice Together with Job Kernel Objects," *Microsoft Systems Journal,* March 1999, <www.microsoft.com/msj/0399/jobkernelobj/jobkernelobj.aspx>.
149. "Fibers," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dlproc/base/fibers.aspx
150. "Convert Thread to Fiber," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/convertthreadtofiber.asp>.
151. "Fibers," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/fibers.asp>.
152. Zabatta, F, and K. Ying, "A Thread Performance Comparison: Windows NT and Solaris on a Symmetric Multiprocessor," *Proceedings of the Second USENIX Windows NT Symposium,* August 5,1998, pp. 57-66.
153. Benjamin, C, "The Fibers of Threads," *Compile Time (Linux Magazine),* May 2001, <www.linux-mag.com/2O01-05/compile_06.html>.
154. "Thread Pooling," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/thread_pooling.asp>.
155. "Queue User Worker Item," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/queueuserworkitem.aspx
156. Richter, J., "New Windows 2000 Pooling Functions Greatly Simplify Thread Management," *Microsoft Systems Journal,* April 1999, <www.microsoft.com/msj/0499/pooling/pooling.aspx>.
157. Richter, J., "New Windows 2000 Pooling Functions Greatly Simplify Thread Management," *Microsoft Systems Journal,* April 1999, <www.microsoft.com/msj/0499/pooling/pooling.aspx>.
158. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed, Redmond: Microsoft Press, 2000.
159. "Scheduling Priorities," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/scheduling_priorities.asp>.

i6o. "Win_32 Thread," *MSDN Library,* July 2003, <msdn.microsoft.com/library/en-us/wmisdk/wmi/win32_thread.asp>.

161. Solomon, D., and M. Russinovich, *Inside Windows, 2000,* 3rd ed, Redmond: Microsoft Press, 2000, pp. 348-349.

162. "Scheduling Priorities," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/scheduling_priorities. asp>.

163. "Scheduling Priorities," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/scheduling_priorities.asp>.

164. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed, Redmond: Microsoft Press, 2000, pp. 356-358.

165. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed, Redmond: Microsoft Press, 2000, pp. 338,347.

166. "Scheduling Priorities," *MSDN Library,* <msdn.microsoft.com/library/en-us/dllproc/base/scheduling_priorities.asp>.

167. "Priority Boosts," *MSDN Library,* <msdn.microsoft.com/library/en-us/dllproc/base/priority_boosts.asp>.

168. "Priority Inversion," *MSDN Library,* <msdn.microsoft.com/library/en-us/dl1proc/base/priority_inversion.asp>.

169. "Multiple Processors," *MSDN Library,* <msdn.microsoft.com/library/en-us/dllproc/base/multipie_processors.asp>.

170. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed, Redmond: Microsoft Press, 2000, p. 371.

171. "Multiple Processors," *MSDN Library,* <msdn.microsoft.com/library/en-us/dllproc/base/multipie_processors.asp>.

172. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed, Redmond: Microsoft Press, 2000, pp. 371-373.

173. "Introduction to Kernel Dispatcher Objects," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/synchro_7kfb.asp>.

174. "Interprocess Synchronization," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/interprocess_synchronization.asp>.

175. "Wait Functions," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dl1proc/base/wait_functions.asp>.

176. "Mutex Objects," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/mutex_objects.asp>.

177. "Event Objects," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/event_objects.asp>.

178. "Mutex Objects," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/mutex_objects.asp>.

179. "Semaphore Objects," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/semaphore_objects.asp>.

180. "Synchronicity: A Review of Synchronization Primitives," *NT Insider,* Vol. 9, No. 1, January 1, 2002, <www.osr.com/ntinsider/2002/synchronicity/synchronicity.htm>.

181. "Semaphore Objects," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/semaphore_objects.asp>.

182. "Waitable Timer Objects," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/waitable_timer_objects.asp>.

183. "Synchronization Objects," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/synchronization_objects.asp>.

184. "Synchronizing Access to Device Data," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_9sfb.asp>.

185. "Introduction to Spin Locks," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/1i brary/en-us/kmarch/hh/kmarch/synchro_8qsn.asp>.

186. "Queued Spin Locks," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/synchro_8ftz.asp>.

187. Friedman, M., and O. Pentakola, "Chapter 5: Multiprocessing," *Windows 2000 Performance Guide,* January 2002, <www.orei1ly.com/catalog/w2kperf/chapter/ch05.html>.

188. "Queued Spin Locks," *MSDN Library,* June 6, 2003. <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/synchro_8ftz.asp>.

189. "Fast Mutexes," *MSDN Library,* June 6, 2003. <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/synchro_3zmv.asp>.

190. "Synchronicity: A Review of Synchronization Primitives," *NT Insider,* Vol. 9, No. 1, January 2002, <www.osr.com/ntinsider/2002/synchronicity/synchronicity.htm>.

191. "Fast Mutexes," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/synchro_3zmv.asp>.

192. "Synchronicity: A Review of Synchronization Primitives," *NT Insider,* Vol. 9, No. 1, January 2002, <www.osr.com/ntinsider/2002/synchronicity/synchronicity.htm>.

193. "Critical Section Objects," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/critical_section_objects.asp>.

194. Prasad, S., "Windows NT Threads," *BYTE,* November 1995, <www.byte.com/art/9511/secll/art3.htm>.

!95- "Time Queues," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/timer_queues.asp>.

196. "Interlocked Variable Access," *MSDN Library,* February 2003. <msdn.microsoft.com/library/en-us/dllproc/base/interlocked_variable_access.asp>.

197. "Interlocking Singly Linked Lists," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/interlocked_singly_linked_lists.asp>.

**198.** "Introducing 64-Bit Windows," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/win64/win64/introducing_64_bit_windows.asp>.

199. "Memory Support and Windows Operating Systems," Microsoft Windows Platform Development, Microsoft Corporation, December 4, 2001, <www.microsoft.com/hwdev/platform/server/PAE/PAEmem.asp>.

200. "NT Memory, Storage, Design, and More," <www.windowsitlibrary.com/Content/113/01/1.html>.

201. R. Kath, "The Virtual-Memory Manager in Windows NT," *MSDN Library,* December 21, 1992, <msdn.microsoft.com/library/en-us/dngenlib/html/msdn_ntvmm.asp?frame=true>.

202. R. Kath, "The Virtual-Memory Manager in Windows NT," *MSDN Library,* December 21, 1992, <msdn.microsoft.com/library/en-us/dngenlib/html/msdn_ntvmm.asp?frame=true>.

203. "Memory Protection," *MSDN Library,* February 2003, <msdn.microsoft.com/library/default.asp?url=/library/en-us/memory/base/memory_protection.asp>.

**204.** "Large Page Support," *MSDN Library,* February 2001, <msdn.microsoft.com/library/en-us/memory/base/large_page_support.asp?frame=true>.

205. Russinovich, M., and D. Solomon, "Windows XP: Kernel Improvements Create a More Robust, Powerful, and Scalable OS," *MSDN Magazine,* December 2001, <msdn.microsoft.com/msdnmag/issues/01/12/XPKernel/print.asp>.

206. R. Kath, "The Virtual-Memory Manager in Windows NT," *MSDN Library,* December 21, 1992, <msdn.microsoft.com/library/en-us/dngenlib/html/msdn_ntvmm.asp?frame=true>.

207. "GetLargePageMinimum," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/memory/base/getlargepageminimum.asp>.

**208.** "Large Page Support," *MSDN Library,* February 2001, <msdn.microsoft.com/library/en-us/memory/base/large_page_support.asp?frame=true>.

209. GetLargePageMinimum," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/memory/base/getlargepageminimum.asp>.

**210.** "Large Page Support in the Linux Kernel," *LWN.net. 2002,* <lwn.net/Articles/6969/>.

211. Kath, R., "The Virtual-Memory Manager in Windows NT," *MSDN Library,* December 21, 1992, <msdn.microsoft.com/library/en-us/dngenlib/html/msdn_ntvmm.asp>.

212. Friedman, M. B., "Windows NT Page Replacement Policies," *Computer Measurement Group,* 1999, <www.demandtech.com/Resources/Papers/WinMemMgmt.pdf>.

**213.** "Allocating Virtual Memory," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/memory/base/allocating_virtual_memory.asp?frame=true>.

**214.** "Memory Management Enhancements," *MSDN Library,* February 13, 2003, <msdn.microsoft.com/library/en-us/appendix/hh/appendix/enhancements5_3oc3.asp?frame=true>.

**215.** "Bug Check 0x41: Must_Succeed_Pool_Empty," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/ddtools/hh/ddtools/bccodes_Obon.asp?frame=true>.

**216.** Munro, I, "Windows XP Kernel Enhancements," *ExtremeTech,* June 8, 2001, <www.extremetech.com/print_article/0,3998,a=2473,00.asp>.

**217.** Munro, J., "Windows XP Kernel Enhancements," *ExtremeTech,* June 8, 2001, <www.extremetech.com/print_article/0,3998,a=2473,00.asp>.

**218.** Russinovich, M., "Inside Memory Management, Part 2," *Windows & .NET Magazine,* September 1998, <www.winnetmag.com/Articles/Print.cfm?ArticleID=3774>.

**219.** Kath, R., "The Virtual-Memory Manager in Windows NT," *MSDN Library,* December 21, 1998, <msdn.microsoft.com/library/en-us/dngenlib/html/msdn_ntvtnm.asp?frame=true>.

**220.** "Windows NT Virtual Memory (Part II)," *OSR Open Systems Resources, Inc.,* 1999, <www.osr.com/ntinsider/1999/virtualmem2/virtualmem2.htm>.

**221.** Friedman, M., "Windows NT Page Replacement Policies," *Computer Measurement Group CMG,* 1999, <www.demandtech.com/Resources/Papers/WinMemMgmt.pdfx

**222.** "How Windows NT Provides 4 Gigabytes of Memory," *Microsoft Knowledge Base Article—99707,* February 20, 2002, <www.itpapers.com/cgi/PSummaryIT.pi?paperid=23661&scid=273>.

**223.** Munro, J., "Windows XP Kernel Enhancements," June 8, 2001, <www.extremetech.com/print_article/0,3998,a=2473,00.asp>.

**224.** LaBorde, D., "TWEAKS for Windows XP for Video Editing (v. 1.0)," *SlashCAM.com,* 2001, <www.siashcam.de/artikel/Tips/TWEAKS_for_Windows_XP_for_Video_Editing_v_l_0_.html>.

225. "Memory Management Enhancements," *MSDN Library,* <msdn.microsoft.com/library/en-us/appendix/hh/appendix/enhancements 5_3oc3.asp>.

**226.** "Memory Management Enhancements," *MSDN Library,* <msdn.microsoft.com/library/en-us/appendix/hh/appendix/enhancements 5_3oc3.asp>.

**227.** LaBorde, D., "Windows XP Tweaks/Optimizations for Windows XP for Video Editing Systems-Part II," *SlashCAM,* 2002, <www.siashcam.de/artikel/Tips/Windows_XP_TWEAKS_Optimization_for_Video_Editing_Systerns_PART_II.html>.

**228.** "Fast System Startup for PCs Running Windows XP," *Windows Platform Design Notes: Designing Hardware for the Microsoft® Windows® Family of Operating Systems,* Microsoft Corporation, January 31, 2002, <www.microsoft.com/hwdev/platform/performance/fastboot/fastboot-winxp.asp>.

229- Kath, R., "Managing Virtual Memory in Win23," *MSDN Library,* January 20, 1993, <msdn.microsoft.com/library/en-us/dngenlib/html/msdn_virtmm.asp?frame=true>.

230. Kath, R., "The Virtual-Memory Manager in Windows NT," *MSDN Library,* December 21, 1992, <msdn.microsfot.com/library/en-us/dngenlib/html/msdn_ntvmm.asp?frame=true>.

**231.** LaBorde, D., "TWEAKS for Windows XP for Video Editing (v. 1.0)," *Slashcam.com,* 2001, <www.slashcam.de/artikel/Tips/TWEAKS_for_Windows_XP_for_Video_Editing_v_l_0_.html>.

232. Friedman, M. B., "Windows NT Page Replacement Policies," *Computer Measurement Group CMG,* 1999, <www.demand-tech.com/Resources/Papers/WinMemMgmt.pdf>.

233. Friedman, M. B., "Windows NT Page Replacement Policies," *Computer Measurement Group CMG,* 1999, <www.demand-tech.com/Resources/Papers/WinMemMgmt.pdf>.

**234.** Kath, R., "The Virtual-Memory Manager in Windows NT," *MSDN Library,* December 21, 1992, <msdn.microsoft.com/library/en-us/dngenlib/html/msdn_ntvmm.asp?frame=true>.

**235.** Russinovich, M., "Inside Memory Management, Part 2," *Windows & .NET Magazine,* September 1998, <www.winnetmag.com/Articles/Index.cfm?ArticleID=3774&pg=2>.

**236.** Friedman, M. B., "Windows NT Page Replacement Policies," *Computer Measurement Group CMG,* 1999, <www.demand-tech.com/Resources/Papers/WinMemMgmt.pdf>.

**237.** Friedman, M. B., "Windows NT Page Replacement Policies," *Computer Measurement Group CMG,* 1999, <www.demand-tech.com/Resources/Papers/WinMemMgmt.pdf>.

**238.** Anderson, C, "Windows NT Paging Fundamentals," *Windows & .NET Magazine,* November 1997, <www.winnetmag.com/Articles/Print.cfm?ArticleID=595>.

**239.** Munro, J., "Windows XP Kernel Enhancements," *ExtremeTech,* June 8, 2001, <www.extremetech.com/print_article/0,3998,a=2473,00.asp>.

**240.** "When Should Code and Data Be Pageable?" *MSDN Library,* <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/memmgmt_7vhj.asp?frame=true>.

**241.** "Device Drivers and File System Drivers Defined," *MSDN Library,* February 13, 2003, <msdn.microsoft.com/library/en-us/gstart/hh/gstart/gs_basics_4w6f.asp>.

242. Custer, H., *Inside the Windows NT File System,* Redmond: Microsoft Press, 1994.

**243.** "File Systems," *Microsoft TechNet,* September 10, 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prkc_fil_buvl.asp>.

**244.** Custer, H., *Inside the Windows NT File System,* Redmond: Microsoft Press, 1994, p. 17.

**245.** Solomon, D, and M. Russinovich, *Inside Windows 2000,* 3rd ed., Redmond: Microsoft Press, 2000.

**246.** Richter, J., and L. F. Cabrera. "A File System for the 21st Century: Previewing the Windows NT 5.0 File System," *Microsoft Systems Journal,* November 1998, <www.microsoft.com/msj/1998/ntfs/ntfs.aspx>.

247. Russinovich, M., "Inside NTFS," *Windows & .NET Magazine,* January 1998, <www.win2000mag.com/Articles/Print.cfm?ArticleID=3455>.

248. Custer, H., *Inside the Windows NT File System,* Redmond: Microsoft Press, 1994, pp. 24-28.

249. Russinovich, M., "Inside NTFS," *Windows & .NET Magazine,* January 1998. <www.win200mag.com/Articles/Print.cfm?ArticleID=3455>.

250. Esposito, D., "A Programmer's Perspective on NTFS 2000, Part 1: Stream and Hard Link," *MSDN Library,* March 2000, <msdn.microsoft.com/library/en-us/dnfiles/html/ntfs5.asp>.

251. Richter, J., and L. F. Cabrera, "A File System for the 21st Century: Previewing the Windows NT 5.0 File System," *Microsoft Systems Journal,* November 1998, <www.microsoft.com/msj/1198/ntfs/ntfs.aspx>.

252. Richter, J., and L. F. Cabrera, "A File System for the 21st Century: Previewing the Windows NT 5.0 File System," *Microsoft Systems Journal,* November 1998, <www.microsoft.com/msj/1198/ntfs/ntfs.aspx>.

253. Esposito, D., "A Programmer's Perspective on NTFS 2000, Part 2: Encryption, Sparseness, and Reparse Points," *MSDN Library.* May 2000, <msdn.microsoft.com/library/en-us/dnfiles/html/ntfs2.asp>.

254. "NTFS," *MSDN Library,* February 2003. <msdn.microsoft.com/library/en-us/fileio/base/ntfs.asp>.

255. Phamdo, N., "Lossless Data Compression," *Data-Compression.com,* 2001, <www.data-compression.com/lossless.html>.

256. Richter, J., and L. F. Cabrera, "A File System for the 21st Century: Previewing the Windows NT 5.0 File System," *Microsoft Systems Journal,* November 1998, <www.microsoft.com/msj/1198/ntfs/ntfs.aspx>.

257. Richter, J., and L. F. Cabrera, "A File System for the 21st Century: Previewing the Windows NT 5.0 File System," *Microsoft Systems Journal,* November 1998, <www.microsoft.com/msj/1198/ntfs/ntfs.aspx>.

258. Esposito, D., "A Programmer's Perspective on NTFS 2000, Part 2: Encryption, Sparseness, and Reparse Points," *MSDN Library.* May 2000, <msdn.microsoft.com/library/en-us/dnfiles/html/ntfs2.asp>.

259. Richter, J., and L. F. Cabrera, "A File System for the 21st Century: Previewing the Windows NT 5.0 File System," *Microsoft Systems Journal,* November 1998, <www.microsoft.com/msj/1198/ntfs/ntfs.aspx>.

260. Richter, J., and L. F Cabrera, "A File System for the 21st Century: Previewing the Windows NT 5.0 File System," *Microsoft Systems Journal,* November 1998, <www.microsoft.com/msj/1198/ntfs/ntfs.aspx>.

261. "Microsoft Windows XP: What's New in Security for Windows XP Professional and Windows XP Home," *Microsoft Corpora-*

*tion,* July 2001. <www.microsoft.com/windowsxp/pro/techinfo/planning/security/whatsnew/WindowsXPSecurity.doo.

262. Richter, J., and L. F. Cabrera, "A File System for the 21st Century: Previewing the Windows NT 5.0 File System," *Microsoft Systems Journal,* November 1998, <www.microsoft.com/msj/1198/ntfs/ntfs.aspx>.

263. Esposito, D., "A Programmer's Perspective on NTFS 2000, Part 1: Stream and Hard Link," *MSDN Library,* March 2000, <msdn.microsoft.com/library/en-us/dnfiles/html/ntfs5.asp>.

264. Esposito, D., "A Programmer's Perspective on NTFS 2000, Part 2: Encryption, Sparseness, and Reparse Points," *MSDN Library,* May 2000, <msdn.microsoft.com/library/en-us/dnfiles/html/ntfs2.asp>.

265. Richter, J., and L. F. Cabrera, "A File System for the 21st Century: Previewing the Windows NT 5.0 File System," *Microsoft Systems Journal,* November 1998, <www.microsoft.com/msj/1198/ntfs/ntfs.aspx>.

266. Richter, J., and L. F. Cabrera, "A File System for the 21st Century: Previewing the Windows NT 5.0 File System," *Microsoft Systems Journal,* November 1998, <www.microsoft.com/msj/1198/ntfs/ntfs.aspx>.

267. Esposito, D., "A Programmer's Perspective on NTFS 2000, Part 2: Encryption, Sparseness, and Reparse Points," *MSDN Library,* May 2000, <msdn.microsoft.com/library/en-us/dnfiles/html/ntfs2.asp>.

268. Esposito, D., "A Programmer's Perspective on NTFS 2000, Part 2: Encryption, Sparseness, and Reparse Points," *MSDN Library,* May 2000, <msdn.microsoft.com/library/en-us/dnfiles/html/ntfs2.asp>.

269. Richter, J., and L. F. Cabrera, "A File System for the 21st Century: Previewing the Windows NT 5.0 File System," *Microsoft Systems Journal,* November 1998, <www.microsoft.com/msj/1198/ntfs/ntfs.aspx>.

270. "Overview of the Windows I/O Model," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/irps_0rhj.asp>.

271. "Plug and Play for Windows 2000 and Windows XP," *Windows Hardware and Driver Central,* December 4, 2001, <www.microsoft.com/whdc/hwdev/tech/pnp/pnpnt5_2.mspx>.

272. "Introduction to Device Objects," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/devobjts_4f qf.asp>.

273. "WDM Driver Layers: An Example," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/wdmintro_4uuf.asp>.

274. "Types of Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intro_8r6v.asp>.

**275-** "Device Extensions," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/devobjts_1gdj.asp>.

276. "Introduction to Driver Objects," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/drvcomps_6js7.asp>.

277. "Introduction to Standard Driver Routines," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/krnarch/hh/kmarch/drvcomps_24kn.asp>.

278. "Introduction to Plug and Play," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/plugplay_2tyf.asp>.

279. "PnP Components," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/plugplay_4bs7.asp>.

280. "PnP Driver Design Guidelines," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/plugplay_890n.asp>.

281. "Levels of Support for PnP," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/plugplay_78o7.asp>.

282. "Power Manager," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/pwrmgmt_30vb.asp>.

283. "System Power Policy," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/pwrmgmt_8baf.asp>.

284. "Driver Role in Power Management," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/pwrmgmt_47jb.asp>.

285. "Power IRPs for Individual Devices," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/pwrmgmt_6azr.asp>.

286. "Device Power States," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/pwrmgmt_20f b.asp>.

287. "Device Sleeping States," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/pwrmgmt_2i p3.asp>.

288. "System Power States," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/pwrmgmt_919j.asp>.

289. "Introduction to WDM," *MSDN Library,* <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/wdmintro_lep3.asp>.

290. "Bus Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/wdmintro_8nf r.asp>.

291. "Filter Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/wdmintro_7dpj.asp>.

292. "Introduction to Device Interface," *MSDN Library,* June 6,2003, <msdn.microsoft.com/library/en-us/install/hh/install/setup-cls_8vs7.asp>.

**293.** "Function Drivers," *MSDN Library,* June 6, 2003. <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/wdmintro_27f r.asp>.

**294.** "Types of Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intro_8r6v.asp>.

**295.** "Types of WDM Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/wdmi ntro_3ep3.asp>.

**296.** "Introduction to WDM," *MSDN Library,* <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/wdmintro_lep3.asp>.

**297.** "Introduction to WMI," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/wmi_5alz.asp>.

**298.** "How NT Describes I/O Requests," *NT Insider,* Vol. 5, No. 1, January 1998, <www.osr.com/ntinsider/1998/Requests/requests.htm>.

**299.** "I/O Status Blocks," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/irps_Oofb.asp>.

300. "I/O Stack Locations," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/irps_81gn.asp>.

**301.** "Starting a Device," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/plugplay_4otj.asp>.

**302.** "I/O Stack Locations," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/irps_81gn.asp>.

303. "Example I/O Request—The Details," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/irps_le3r.asp>.

**304.** "Registering an I/O Completion Routine," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/irps_6dd3.asp>.

**305.** "Example I/O Request—The Details," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/irps_le3r.asp>.

**306.** "Example I/O Request—The Details," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/irps_le3r.asp>.

**307.** "Queuing and Dequeuing IRPs," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/irps_4mqv.asp>.

**308.** "Synchronous and Asynchronous I/O," *MSDN Library,* February 2003 <msdn.microsoft.com/library/en-us/fileio/base/synchronous_and_asynchronous_i_o.asp>.

**309.** "Synchronization and Overlapped Input and Output," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dl1proc/base/synchronization_and_overlapped_input_and_output.asp>.

**310.** "Alertable I/O," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/alertable_i_o.asp>.

**311.** "I/O Completion Ports," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/i_o_completion_ports.asp>.

**312.** "Methods for Accessing Data Buffers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/iputoput_3m07.asp>.

**313.** "Using Buffered I/O," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/i putoput_lulj.asp>.

**314.** "Methods for Accessing Data Buffers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/iputoput_3m07.asp>.

**315.** "Using Buffered I/O," *MSDN Library,* June 6, 2003. <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/i putoput_lulj.asp>.

**316.** "Using Direct I/O with DMA," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/iputoput_0gx3.asp>.

**317.** "Using Neither Buffered nor Direct I/O," *MSDN Library,* June 6. 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/iputoput_3cbr.asp>.

**318.** "How NT Describes I/O Requests," *NT Insider,* Vol. 5, No. 1, January 1998, <www.osr.com/ntinsider/1998/Requests/requests.htm>.

**319.** "Methods for Accessing Data Buffers," *MSDN Library,* June 6. 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/iputoput_3m07.asp>.

**320.** "Example I/O Request—The Details," *MSDN Library,* June 6. 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/irps_le3r.asp>.

**321.** "InterruptService," *MSDN Library,* June 6, 2003. <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/drvrrtns_29ma.asp>.

**322.** "Registering an ISR," *MSDN Library,* June 6, 2003. <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_87qf.asp>.

**323.** "Introduction to Interrupt Objects," *MSDN Library,* June 6. 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_417r.asp>.

**324.** "Registering an ISR," *MSDN Library,* June 6, 2003. <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_87qf.asp>.

325. "Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed. Redmond: Microsoft Press, 2000, p. 92.

**326.** "Key Benefits of the I/O APIC," *Windows Hardware and Driver Central,* <www.microsoft.com/whdc/hwdev/platform/proc/IO-APIC.mspx>.

**327.** "Writing an ISR," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/intrupts_4ron.asp>.

328. "File Caching," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/file_caching.asp>.

329. "The NT Cache Manager," *NT Insider,* Vol. 3, No. 2, April 1996, <www.osr.com/ntinsider/1996/cacheman.htm>.

330. "The NT Cache Manager," *NT Insider,* Vol. 3, No. 2, April 1996. <www.osr.com/ntinsider/1996/cacheman.htm>.

331. "File Caching," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/file_caching.asp>.

332. "Life in the Fast I/O Lane," *NT Insider,* Vol. 3, No. 1, February 15, 1996, updated October 31, 2002, <www.osronline.com/article.cfm?id=166>.

333. "File Caching," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/file_caching.asp>.

334. "Mailslots," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/mailslots.asp>.

335. "About Pipes," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/about_pipes.asp>.

336. "Pipes," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/pipes.asp>.

337. "Named Pipes," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/named_pipes.asp>.

338. "Named Pipes Open Modes," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/named_pipe_open_modes.asp>.

339. "Anonymous Pipes," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/anonymous_pipes.asp>.

340. "Anonymous Pipe Operations," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/anonymous_pipe_operations.asp>.

341. "Anonymous Pipe Operations," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/anonymous_pi pe_operations.asp>.

342. "Named Pipes," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/named_pipes.asp>.

343. "CreateNamedPipe," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/create-namedpipe.asp>.

344- "Named Pipe Type, Read, and Wait Modes," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/named_pipe_type_read_and_wait_modes.asp>.

345. "Named Pipes Open Modes," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/named_pipe_open_modes.asp>.

346. "Named Pipe Type, Read, and Wait Modes," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/named_pipe_type„read_and_wait_modes.asp>.

347. "Synchronous and Overlapped Input and Output," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/synch ronous_and_overlapped_input_and_output.asp>.

348. "Mailslots," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/mailslots.asp>.

349. "About Mailslots," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/about_mailsiots.asp>.

350. "About Mailslots," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/about_mailslots.asp>.

351. "Common Windows Term Glossary," *Stanford Windows Infrastructure,* viewed July 10, 2003, <windows.stanford.edu/docs/glossary.htm>.

352. "About Mailslots," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/i pc/base/about_mailslots.asp>.

353. "CIFS/SMB Protocol Overview," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/cifs_smb_protocol_overview.asp>.

354. "Server and Client Functions," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/ipc/base/server_and_client_functions.asp>.

355. "File Mapping," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/file_mapping.asp?frame=true>.

356. "Sharing Files and Memory," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/sharing_files_and_memory.asp?frame=true>.

357. "File Mapping," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/file_mapping.asp?frame=true>.

358. Kath, R., "Managing Memory-Mapped Files in Win32," *MSDN Library,* February 9, 1993, <msdn.microsoft.com/library/en-us/dngenlib/html/msdn_manamemo.asp?frame=true>.

359. "Sharing Files and Memory," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/sharing_files_and_memory.asp?frame=true>.

360. Dabak, P.; M. Borate; and S. Phadke, "Local Procedure Call," *Windows IT Library,* October 1999, <www.windowsitlibrary.com/Content/356/08/1.html>.

361. Dabak, P.; M. Borate; and S. Phadke, "Local Procedure Call," *Windows IT Library,* October 1999, <www.windowsitlibrary.com/Content/356/08/l.htm1>.

362. Solomon, D., and M. Russinovich, *Inside Windows 2000,* 3rd ed, Redmond: Microsoft Press, 2000, p. 171.

363. Dabak, P.; M. Borate; and S. Phadke, "Local Procedure Call," *Windows IT Library,* October 1999, <www.windowsitlibrary.com/Content/356/08/1.html>.

364. "How RPC Works," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/how_rpc_works.asp>.

365. "DCE Glossary of Technical Terms," *Open Group,* 1996, <www.opengroup.org/dce/info/papers/dce-glossary.htm>.

366. "How RPC Works," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/how_rpc_works.asp>.

367. "Protocol Sequence Constants," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/protocol_sequence_constants.asp>.

368. "Asynchronous RPC," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/asynchronous_rpc.asp>.

369. "The IDL and ACF Files," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/the_idl_and_acf_files.asp>.

370. "Microsoft Interface Definition Language SDK Documentation," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/midl/midl/midl_start_page.asp>.

371. "The IDL Interface Header," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/the_idl_interface_header.asp>.

372. "The IDL Interface Body," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/the_idl_interface_body.asp>.

373. "Application Configuration File (ACF)," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/midl/midl/application_configuration_file_acf_.asp>.

374. "Registering Endpoints," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/registering_endpoints.asp>.

375. "Binding Handles," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/binding_handles.asp>.

376. "Client-Side Binding," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/client_side_binding.asp>.

377. "Selecting a Protocol Sequence," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/selecting_a_protocol_sequence.asp>.

378. "Types of Binding Handles," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/types_of_binding_handles.asp>.

379. "Explicit Binding Handles," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rpc/rpc/explicit_binding_handles.asp>.

380. Dabak, P.; M. Borate; and S. Phadke, "Local Procedure Call," *Windows IT Library,* October 1999, <www.windowsitlibrary.com/Content/356/08/1.html>.

381. Williams, S., and C. Kindel, "The Component Object Model," October 1994, <msdn.microsoft.com/library/en-us/dncomg/html/msdn_comppr.asp>.

382. Williams, S., and C. Kindel, "The Component Object Model," October 1994, <msdn.microsoft.com/library/en-us/dncomg/html/msdn_comppr.asp>.

383. "COM Objects and Interfaces," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/com/htm/com_Oalv.asp>.

384. S. Williams and C. Kindel, "The Component Object Model," October 1994, <msdn.microsoft.com/library/en-us/dncomg/html/msdn_comppr.asp>.

385. S. Williams and C. Kindel, "The Component Object Model," October 1994, <msdn.microsoft.com/library/en-us/dncomg/html/msdn_comppr.asp>.

386. "The COM Library," *MSDN Library,* February 2003. <msdn.microsoft.com/library/en-us/com/htm/com_1fuh.asp>.

387. "Registering COM Servers," *MSDN Library,* February 2003. <msdn.microsoft.com/library/en-us/com/htm/comext_05pv.asp>.

388. "QueryInterface: Navigating an Object," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/com/htm/com_02b8.asp>.

389. Williams, S., and C. Kindel, "The Component Object Model," October 1994, <msdn.microsoft.com/library/en-us/dncomg/html/msdn_comppr.asp>.

390. "DCOM Technical Overview," Microsoft Corporation, 1996'. <msdn.microsoft.com/library/en-us/dndcom/html/msdn_dcomtec.asp>.

391. "Processes, Threads and Apartments," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/com/htm/aptnthrd_8po3.asp>.

392. "Introducing COM+," *MSDN Library,* February 2003. <msdn.microsoft.com/library/en-us/cossdk/htm/betaintr_6d5r.asp>.

393. "What's New in COM+ 1.5," *MSDN Library* February 2003. <msdn.microsoft.com/library/en-us/cossdk/htm/whatsnewcomplus_350z.asp>.

394. "DCOM Technical Overview," Microsoft Corporation, 1996. <msdn.microsoft.com/library/en-us/dndcom/html/msdn_dcomtec.asp>.

**395-** "Introduction to ActiveX Controls," *MSDN Library,* July 9,2003, <msdn.microsoft.com/workshop/components/activex/intro.asp>.

**396.** "Clipboard," *MSDN Library,* 2003, <msdn.microsoft.com/library/en-us/winui/winui/windowsuserinterface/dataexchange/clipboard.asp>.

**397.** "About the Clipboard," *MDSN Library,* 2003, <msdn.microsoft.com/library/en-us/winui/winui/windowsuserinterface/dataexchange/clipboard/abouttheclipboard.asp>.

**398.** "Clipboard Formats," *MSDN Library,* 2003, <msdn.microsoft.com/library/en-us/winui/winui/windowsuserinterface/dataexchange/clipboard/clipboardformats.asp>.

**399.** "About the Clipboard," *MSDN Library,* 2003, <msdn.microsoft.com/library/en-us/winui/winui/windowsuserinterface/dataexchange/clipboard/abouttheclipboard.asp>.

**400.** Brockschmidt, K., "OLE Integration Technologies: A Technical Overview," Microsoft Corporation, October 1994, <msdn.microsoft.com/library/en-us/dnolegen/html/msdn_ddjole.asp>.

**401.** "Network Redirectors," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/network_redirectors.asp>.

**402.** "Description of a Network I/O Operation," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/description_of_a_network_i_o_operation.asp>.

**403.** "Network Redirectors," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/network_redirectors.asp>.

**404.** "Description of a Network I/O Operation," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/description_of_a_network_i_o_operation.asp>.

**405.** Zacker, C, "Multiple UNC Provider: NT's Traffic Cop," *Windows & .NET Magazine,* May 1998, <www.winnetmag.com/Articles/Index.cfm?ArticleID=3127>.

**406.** "CIFS/SMB Protocol Overview," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/cifs_smb_protocol_overview.asp>.

407. Leach, P., and D. Perry, "CIFS: A Common Internet File System," *Microsoft.com,* November 1996, <www.microsoft.com/mind/1196/cifs.asp>.

**408.** Munro, J., "Windows XP Kernel Enhancements," *ExtremeTech,* June 8, 2001, <www.extremetech.com/print_article/0,3998,a=2473,00.asp>.

**409.** "CIF Packet Exchange Scenario," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/fileio/base/cif_packet_exchange_scenario.asp>.

**410.** "Opportunistic Locks," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/opportunistic_locks.asp>.

411. "Breaking Opportunistic Locks," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/fileio/base/breaking_opportunistic_locks.asp>.

412. "NDIS Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/network/hh/network/102gen_0w2v.asp>.

**413.** "Benefits of Remote NDIS," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/network/hh/network/rndisover_44mf.asp>.

414. "Introduction to Intermediate Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/network/hh/network/301int_78mf.asp>.

**415.** "NDIS Miniport Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/network/hh/network/102gen_lv5d.asp>.

**416.** "NDIS Intermediate Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/network/hh/network/102gen_4mw7.asp>.

**417.** "NDIS Protocol Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/network/hh/network/102gen_67hj.asp>.

**418.** "TDI Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/network/hh/network/102gen_4y93.asp>.

**419.** "TCP/IP and Other Network Protocols," *Windows XP Professional Resource Kit,* viewed July 7, 2003, <www.microsoft.com/technetprodtechnol/winxppro/reskit/prcf_omn_gzcg.asp>.

420. "Network Protocols," *Windows XP Resource Kit,* viewed July 7, 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prch_cnn_lafg.asp>.

**421.** "Introduction to Intermediate Drivers," *MSDN Library,* June 6, 2003, <msdn.microsoft.com/library/en-us/network/hh/network/301int_78mf.asp>.

422. "TCP/IP and Other Network Protocols," *Windows XP Professional Resource Kit,* viewed July 7, 2003, <www.microsoft.com/tecr.net/prodtechnol /winxppro/reskit/prcc_tcp_tuoz.asp>.

423. "Defining TCP/IP," *Windows XP Professional Resource Kit,* viewed July 7, 2003, <www.microsoft.com/technet/treeview/default.asp?url=/technet/prodtechnol/winxppro/reski t/prcc_tcp_utip.asp>.

424. "Network Protocols," *Windows XP Resource Kit,* viewed July 7, 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prch_cnn_actp.asp>.

**425.** Hertel, C, "Understanding the Network Neighborhood," *Linux Magazine,* May 2001, <www.linux-mag.com/2001-05/smb_01.html>.

426. "Network Protocol Support in Windows," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/winsock/winsock/network_protocol_support_in_windows.asp>.

427. Hertel, C, "Understanding the Network Neighborhood," *Linux Magazine,* May 2001, <www.linux-mag.com/2001-05/smb_01.html>.

428. "About WinHTTP," *MSDN Library,* July 2003, <msdn.microsoft.com/library/en-us/winhttp/http/about_winhttp.asp>.

429. "About WinlNet," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/wininet/wininet/about_wininet.asp>.

430. "Porting WinlNET applications to WinHTTP," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/winhttp/http/wininet.asp>.

431. Hertel, C, "Understanding the Network Neighborhood," *Linux Magazine,* May 2001, <www.linux-mag.com/2001-05/smb_01.html>.

432. "Windows Sockets Background," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/vccore/html/_core_Windows_Sockets.3a_.Background.asp>.

433- "Windows Sockets 2 API," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/winsock/winsock/windows_sockets_2_api_2.asp>.

434. Lewandowski, S., "Interprocess Communication in UNIX and Windows NT," 1997, <www.cs.brown.edu/people/scl/files/IPCWinNTUNTX.pdf>.

435- "Windows Sockets Background," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/vccore/html/_core_Windows_Sockets.3a_.Background.asp>.

436. "Windows Sockets 2 API," *MSDN Library* February 2003, <msdn.microsoft.com/library/en-us/winsock/winsock/windows_sockets_2_api_2.asp>.

437. "Windows Sockets Functions," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/winsock/winsock/windows_sockets_functions_2.asp>.

438. "Active Directory," *MSDN Library,* July 2003, <msdn.microsoft.com/library/en-us/netdir/ad/active_directory.asp>.

439- "About Active Directory," *MSDN Library,* July 2003, <msdn.microsoft.com/library/en-us/netdir/ad/about_active_directory.asp>.

440. "Lightweight Directory Access Protocol," *MSDN Library,* July 2003, <msdn.microsoft.com/library/en-us/netdir/ldap/lightweight_directory_access_protocol_ldap_api.asp>.

441. "Active Directory Services Interface (ADSI): Frequently Asked Questions," *MSDN Library,* viewed July 7, 2003, <msdn.microsoft.com/library/en-us/dnactdir/html/msdn_adsifaq. asp>.

442. "Remote Access Service," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/rras/rras/ras_start_page.asp>.

**443-** "DCOM Technical Overview," Microsoft Corporation, 1996, <msdn.microsoft.com/library/en-us/dndcom/html/msdn_dcomtec.asp>.

**444.** Kaiser, J., "Windows XP and .NET: An Overview," *Microsoft TechNet,* July 2001, <www.microsoft.com/technet/prodtechnol/winxppro/evaluate/xpdotnet.asp>.

**445.** Ricciuti, M., "Strategy: Blueprint Shrouded in Mystery," *CNET News.com,* October 18, 2001, <news.com.com/2009-1001-274344.html>.

**446.** M. Ricciuti, "Strategy: Blueprint Shrouded in Mystery," *CNET News.com,* October 18, 2001, <news.com.com/2009-1001-274344.html>.

**447.** Ricciuti, M., "Strategy: Blueprint Shrouded in Mystery," *CNET News.com,* October 18, 2001, <news.com.com/2009-1001-274344.html>.

**448.** J. Kaiser, "Windows XP and .NET: An Overview," *Microsoft TechNet,* July 2001, <www.microsoft.com/technet/prodtechnol/winxppro/evaluate/xpdotnet.asp>.

**449.** M. Ricciuti, "Strategy: Blueprint Shrouded in Mystery," *CNET News.com,* October 18, 2001, <news.com.com/2009-1001-274344.html>.

**450.** "Multiple Processors," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/multipie_processors.asp>.

**451.** "Queued Spin Locks," *MSDN Library,* June 6, 2003. <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/synchro_8ftz.asp>.

**452.** Russinovich, M., and D. Solomon, "Windows XP: Kernel Improvements Create a More Robust, Powerful, and Scalable OS," *MSDN Magazine,* December 2001, <msdn.microsoft.com/msdnmag/issues/01/12/XPKernel/print.asp>.

**453.** "Introduction to Kernel Dispatcher Objects," *MSDN Library.* June 6, 2003, <msdn.microsoft.com/library/en-us/kmarch/hh/kmarch/synchro_7kfb.asp>.

**454.** Russinovich, M., and D. Solomon, "Windows XP: Kernel Improvements Create a More Robust, Powerful, and Scalable OS," *MSDN Magazine,* December 2001, <msdn.microsoft.com/msdnmag/issues/01/12/XPKernel/print.asp>.

**455.** "Overview of Windows XP 64-Bit Edition," *Microsoft TechNet,* viewed July 24, 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prka_fea_ayuw.asp>.

456. Connolly, C, "First Look: Windows XP 64-Bit Edition for AMD64," *Game PC* September 5, 2003 <www.gamepc.com/1abs/view_content.asp?id=amd64xp&page=l>.

**457.** "Overview of Windows XP 64-Bit Edition," *Microsoft TechNet.* viewed July 24, 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prka_fea_ayuw.asp>.

**458.** "Job Objects," *MSDN Library,* February 2003. <msdn.microsoft.com/library/en-us/dllproc/base/job_objects.asp>.

**459.** "I/O Completion Ports," *MSDN Library,* February 2003. <msdn.microsoft.com/library/en-us/fileio/base/i_o_completion_ports.asp>.

460. "Thread Pooling," *MSDN Library,* February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/thread_pooling.asp>.

461. Fincher, X, "Getting to Know Windows NT Embedded and Windows XP Embedded," *MSDN Library,* November 20, 2001, <msdn.microsoft.com/library/en-us/dnembedded/html/embeddedll202001.asp>.

462. "About Windows XP Embedded," *MSDN Library,* June 2, 2003, <msdn.microsoft.com/library/en-us/xpehelp/html/xeconAboutWindowsXPEmbeddedTop.asp>.

463. Fincher, X, "Getting to Know Windows NT Embedded and Windows XP Embedded," *MSDN Library,* November 20, 2001, <msdn.microsoft.com/library/en-us/dnembedded/html/embeddedll202001.asp>.

464. Cherepov, M., et al., "Hard Real-Time with Venturcom RTX on Microsoft Windows XP and Windows XP Embedded," *MSDN Library,* June 2002, <msdn.microsoft.com/library/en-us/dnxpembed/html/hardrealtime.asp>.

465. "Components Used in Interactive Logon," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prdp_log_axsg.asp>.

466. "Interactive Logons Using Kerberos Authentication," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskig/prdp_log_brso.asp>.

467. "Components Used in Interactive Logon," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prdp_log_axsg.asp>.

468. "Protocol Selection," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reski t/prdp_log_axsg.asp>

469. "Components Used in Interactive Logon," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prdp_log_axsg.asp>.

470. "What's New in Security for Windows XP Professional and Windows XP Home Edition," *Microsoft Corporation,* July 2001, <www.microsoft.com/windowsxp/pro/techinfo/planning/security/whatsnew/WindowsXPSecurity.doo.

471. "Security Principals," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prdp_log_jrut.asp>.

472. "Security Groups," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/treeview/default.asp?url=/technet/prodtechnol/winxppro/reskit/prdp_log_jrut.asp>.

**473-** "Security Identifiers," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prdp_log_zhi u.asp>.

**474.** "Security Identifiers," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prdp_l og_zhiu.asp>.

**475.** "What's New in Security for Windows XP Professional and Windows XP Home Edition," Microsoft Corporation, July 2001, <www.microsoft.com/windowsxp/pro/techinfo/planning/security/whatsnew/WindowsXPSecurity.doo.

**476.** "Important Terms," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prdd_sec_zrve.asp>.

**477.** "User-Based Authorization," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prdd_sec_qjin.asp>.

**478.** "New in Windows XP Professional," *Microsoft TechNet,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prdp_log_oeec.asp>.

**479.** "Internet Connection Firewalls," *MSDN Library,* 2003, <www.microsoft.com/technet/prodtechnol/winxppro/reskit/prcg_cnd_brph.asp>.

**480.** "What's New in Security for Windows XP Professional and Windows XP Home Edition," Microsoft Corporation, July 2001, <www.microsoft.com/windowsxp/pro/techinfo/planning/security/whatsnew/WindowsXPSecurity.doo.

**481.** Wong, D., "Windows ICF: Can't Live With It, Can't Live Without It," *SecurityFocus,* August 22, 2002, <www.securityfocus.com/infocus/1620>.

**482.** Wong, D., "Windows ICF: Can't Live With It, Can't Live Without It," *SecurityFocus,* August 22, 2002, <www.securityfocus.com/infocus/1620>.

**483.** "What's New in Security for Windows XP Professional and Windows XP Home Edition," Microsoft Corporation, July 2001, <www.microsoft.com/windowsxp/pro/techinfo/planning/security/whatsnew/WindowsXPSecurity.doo.

**484.** "What's New in Security for Windows XP Professional and Windows XP Home Edition," Microsoft Corporation, July 2001. <www.microsoft.com/windowsxp/pro/techinfo/planning/security/whatsnew/WindowsXPSecurity.doo.

**485.** "Top 10 Reasons to Get Windows XP for Home PC Security," Microsoft Corporation, 2003, <www.microsoft.com/WindowsXP/security/top10.asp>.

# Glossary

## *Numerics*

**2-D mesh network**—Multiprocessor interconnection scheme that arranges nodes in an *m* x *n* rectangle.

3DES-See Triple DES.

**4-connected 2-D mesh network**—2-D mesh network in which nodes are connected with the nodes directly to the north, south, east and west.

## A

**abandoned mutex** (Windows XP)—Mutex that was not released before the termination of the thread that held it.

**abort**—Action that terminates a process prematurely. Also, in the IA-32 specification, an error from which a process cannot recover.

**absolute loading**—Loading technique in which the loader places the program in memory at the address specified by the programmer or compiler.

**absolute path**—Path beginning at the root directory.

**absolute performance measure**—Measure of the efficiency with which a computer system meets its goals, described by an absolute quantity such as the amount of time in which a system executes a certain benchmark. This contrasts with relative performance measures such as ease of use, which only can be used to make comparisons between systems.

**Accelerated Graphics Port (AGP)**-Popular bus architecture used for connecting graphics devices; AGPs typically provide 260MB/s of bandwidth.

**access control attribute** (Linux)—Specifies the access rights for processes attempting to access a particular resource.

**access control entry (ACE)** (Windows XP)-Entry in a discretionary access control list (DACL) for a specific resource that contains the security identifier (SID) of a security principal and the type of access (if any) the security principal has to that particular resource.

**access control list**—List that stores one entry for each access right granted to a subject for an object. An access control list consumes less space than an access control matrix.

**Access Control List (ACL)** (Multics)—Multics' discretionary access control implementation.

**access control matrix**-Matrix that lists system's subjects in the rows and the objects to which they require access in the columns. Each cell in the matrix specifies the actions that a subject (defined by the row) can perform on an object (defined by the column). Access control matrices typically are not implemented because they are sparsely populated.

**access control mode**—Set of privileges (e.g., read, write, execute and/or append) that determine how a page or segment of memory can be accessed.

**Access Isolation Mechanism (AIM)** (Multics)—Multics' mandatory access control implementation.

**access method**—Technique a file system uses to access file data. See also queued access methods and basic access methods.

**access right**—Defines how various subjects can access various objects. Subjects may be users, processes, programs or other entities-Objects are information-holding entities; they may be physical objects that correspond to disks, processors or main memory or abstract objects that correspond to data structures, processes or services. Subjects are also considered to be objects of the system; one subject may have rights to access another.

**access token** (Windows XP)—Data structure that stores security information, such as the security identifier (SID) and group memberships, about a security principal.

**access transparency**—Hides the details of networking protocols that enable communication between computers in a distributed system.

**accessed bit**—See referenced bit.

**accessibility** (file)—File property that places restrictions on which users can access file data.

**acknowledgement segment (ACK)-In** TCP, a segment that is sent to the source host to indicate that the destination host has received a segment. If a source host does not receive and ACK for a segment, it will retransmit that segment. This guarantees that each transmitted segment is received.

**acquire operation**—In several coherence strategies, an operation indy-eating that a process is about to access shared memory.

**Active Directory** (Windows XP)-Network service that provides directory services for shared objects (e.g., files, printers, services, etc.) on a network.

**active list** (Linux)—Scheduler structure that contains processes that will control the processor at least once during the current epoch.

**active page** (Linux)—Page of memory that will not be replaced the next time pages are selected for replacement.

***active* state** (Linux)—Task state describing tasks that can compete for execution on a processor during the current epoch.

**ActiveX Controls**—Self-registering COM objects useful for embedding in Web pages.

**activity** (file)—Percentage of a file's records accessed during a given period of time.

**actuator**—See disk arm.

**ad hoc network**—Network characterized as being spontaneous—any number of wireless or wired devices may be connected to it at any time.

**Ada**—Concurrent, procedural programming language developed by the DoD during the 1970s and 1980s.

**adaptive lock**—Mutual exclusion lock that allows processes to switch between using a spin lock or a blocking lock, depending on the current condition of the system.

**adaptive mechanism**—Control entity that adjusts a system in response to its changing behavior.

**add-on card**—Device that extends the functionality of a computer (e.g., sound and video cards).

**address binding**—Assignment of memory addresses to program data and instructions.

**address bus**—Part of a bus that specifies the memory location from or to which data is to be transferred.

**address space**—Set of memory locations a process can reference.

**address translation map**—Table that assists in the mapping of virtual addresses to their corresponding real memory addresses.

**admission scheduling**—See high-level scheduling.

**Advanced Configuration and Power Interface (ACPI)**—Power management specification supported by many operating systems that allows a system to turn off some or all of its devices without loss of work.

**Advanced Encryption Standard (AES)**—Standard for symmetric encryption that uses Rijndael as the encryption method. AES has replaced the Data Encryption Standard (DES) because AES provides enhanced security.

**Advanced Research Projects Agency (ARPA)**—Government agency under the Department of Defense that laid the groundwork for the Internet; it is now called the Defense Advanced Research Projects Agency (DARPA).

**advertisement** (in JXTA)—XML document formatted according to JXTA specifications that is used by a peer to advertise itself and notify others of its existence.

**advisable process lock (APL)**—Locking mechanism in which an acquirer estimates how long it will hold the lock; other processes can use this estimate to determine whether to block or spin when waiting for the lock.

**affinity mask** (Windows XP)—List of processors on which a process's threads are allowed to execute.

**agent level** (in JMX)—JMX level that provides services for exposing the managed resources.

**aging of priorities**—Method of preventing indefinite postponement by increasing a process's priority gradually as it waits.

**air gap technology**—Network security solution that complements a firewall. It secures private data from external users accessing the internal network.

**alertable** I/O (Windows XP)—Type of asynchronous I/O in which the system notifies the requesting thread of the completion of the I/O with an APC.

**alertable *wait* state** (Windows XP)—State in which a thread cannot execute until it is awakened either by an APC entering the thread's APC queue or by receiving a handle to the object (or objects) for which the thread is waiting.

**alternate data stream**—File attribute that stores the contents of an NTFS file that are not in the default data stream. NTFS files may have multiple alternate data streams.

**analytic model**—Mathematical representation of a computer system or component of a computer system for the purpose of estimating its performances quickly and relatively accurately.

**Andrew File System (AFS)**—Scalable distributed file system that would grow to support a large community while being secure. AFS is a global file system that appears as a branch of a traditional UNIX file system at each workstation. AFS is completely location transparent and provides a high degree of availability.

**anonymous pipe** (Windows XP)—Unnamed pipe that can be used only for synchronous one-way communication between local processes.

**anticipatory buffering**—Technique that allows processing and I/O operations to be overlapped by buffering more than one record at a time in main memory.

**anticipatory movement**—Movement of the disk arm during disk arm anticipation.

**anticipatory paging**—Technique that preloads a process's nonresident pages that are likely to be referenced in the near future. Such strategies attempt to reduce the number of page faults a process experiences.

**antivirus software**—Program that attempts to identify, remove and otherwise protect a system from viruses.

**anycast**—Type of IPv6 address that enables a datagram to be sent to any host within a group of hosts.

**Apache Software Foundation**—Provides open-source software, such as Tomcat, the official reference implementation of JSP and servlet specifications.

**apartment model**—COM object threading model in which only one thread acts as a server for each COM object.

**APC IRQL** (Windows XP)-IRQL at which APCs execute and incoming asynchronous procedure calls (APCs) are masked.

**append access**—Access right that enables a process to write additional information at the end of a segment but not to modify its existing contents; see also execute access, read access and write access.

**append-only file attribute** (Linux)—File attribute that limits users to appending data to existing file contents.

**application base**—Combination of the hardware and the operating system environment in which applications are developed. It is diffi-

cult for users and application developers to convert from an established application base to another.

**application configuration file (ACF)** (Windows XP)— File that specifies platform-specific attributes (e.g., how data should be formatted) for an RPC function call.

**application layer** (in grid computing)—Contains applications that use lower-level layers to access the distributed resources.

**application layer** (in OSI)—Interacts with the applications and provides various network services, such as file transfer and e-mail.

**application layer** (in TCP/IP)—Protocols in this layer allow applications on remote hosts to communicate with each other. The application layer in TCP/IP performs the functionality of the top three layers of OSI—the application, presentation and session layers.

**application-level gateway**—Hardware or software that protects the network against the data contained in packets. If the message contains a virus, the gateway can block it from being sent to the intended receiver.

**application programming**—Software development that entails writing code that requests services and resources from the operating system to perform tasks (e.g., text editing, loading Web pages or payroll processing).

**application programming interface (API)**—Set of functions that allows an application to request services from a lower level of the system (e.g., the operating system or a library module).

**application vector**—Vector that contains the relative demand on operating system primitives by a particular application, used in an application-specific performance evaluation.

**architecture-specific code**—Code that specifies instructions unique to a particular architecture.

**Arithmetic and Logic Unit (ALU)**—Component of a processor that performs basic arithmetic and logic operations.

**ARPAnet**—Predecessor to the Internet that enabled researchers to network their computers. ARPAnet's chief benefit proved to be quick and easy communication via what came to be known as electronic mail (e-mail).

**array processor**—SIMD (single-instruction-stream, multiple-data-stream) system consisting of many (possibly tens of thousands) simple processing units, each executing the same instruction in parallel on many data elements.

**arrival rate**—Rate at which new requests are made for a resource.

**artificial contiguity**—Technique employed by virtual memory systems to provide the illusion that a program's instructions and data are stored contiguously when pieces of them may be spread throughout main memory; this simplifies programming.

**ASCII (American Standard Code for Information Interchange)**—Character set, popular in personal computers and in data communication systems, that stores characters as 8-bit bytes.

**assembler**—Translator program that converts assembly-language programs to machine language.

**assembly language**—Low-level language that represents basic computer operations as English-like abbreviations.

**associative mapping**—Content-addressed associative memory that assists in the mapping of virtual addresses to their corresponding real memory addresses; all entries of the associative memory are searched simultaneously.

**associative memory**—Memory that is searched by content, not by location; fast associative memories can help implement high-speed dynamic address translation mechanisms.

**asynchronous cancellation** (POSIX)—Cancellation mode in which a thread is terminated immediately upon receiving the cancellation signal.

**asynchronous concurrent threads**—Threads that exist simultaneously but operate independently of one another and that occasionally communicate and synchronize to perform cooperative tasks.

**asynchronous procedure call (APC)** (Windows XP)—Procedure calls that threads or the system can queue for execution by a specific thread.

**asynchronous real-time process**—Real-time process that executes in response to events.

**asynchronous signal**—Signal generated for reasons unrelated to the current instruction of the running thread.

**asynchronous transmission**—Transferring data from one device to another that operates independently via a buffer to eliminate the need for blocking; the sender can perform other work once the data arrives in the buffer, even if the receiver has not yet read the data.

**atomic broadcast**—Guarantees that all messages in a system are received in the same order at each process. Also known as totally ordered or agreed broadcast.

**atomic operation**—Operation performed without interruption.

**atomic transaction**—Group of operations that have no effect on the state of the system unless they complete in their entirety.

**attenuation**—Deterioration of a signal due to physical characteristics of the medium.

**attraction memory (AM)**—Main memory in a COMA (cache-only memory architecture) multiprocessor, which is organized as a cache.

**attribute** (of an object)—See property.

**authentication** (secure transaction)—One of the five fundamental requirements for a successful, secure transaction. Authentication deals with how the sender and receiver of a message verify their identities to each other.

**authentication header (AH)** (IPSec)—Information that verifies the identity of a packet's sender and proves that a packet's data was not modified in transit.

**authentication server scripts**—Single sign-on implementation that authenticates users via a central server, which establishes connections between the user and the applications the user wishes to access.

**authorization** (secure transaction)—One of the five fundamental requirements for a successful, secure transaction. Authorization deals with how to manage access to protected resources on the basis of user credentials.

**auto-reset timer object** (Windows XP)—Timer object that remains *signaled* only until one thread finishes waiting on the object.

**automatic binding handle** (Windows XP)—RPC binding handle in which the client process simply calls a remote function, and the stub manages all communication tasks.

**auxiliary storage**—See secondary storage.

**auxiliary storage management**—Component of file systems concerned with allocating space for files on secondary storage devices.

**available volume storage group (AVSG)** (in Coda)—Members of the VSG with which the client can communicate.

## B

**back-door program**—Resident virus that allows an attacker complete, undetected access to the victim's computer resources.

**backup**—Creation of redundant copies of information.

**Bad Page List** (Windows XP)—List of page frames that have generated hardware errors; the system does not store pages on these page frames.

**balance set manager** (Windows XP)—Executive component responsible for adjusting working set minimums and maximums and trimming working sets.

**bandwidth**—Information-carrying capacity of a communications line. Also, a the amount of data transferred over a unit of time.

**base priority class** (Windows XP)—Process attribute that determines a narrow range of base priorities the process's threads can have.

**base priority level** (Windows XP)—Thread attribute that describes the thread's base priority within a base priority class.

**base register**—Register containing the lowest memory address a process may reference.

**baseline network**—Type of multistage network.

**basic access method**—File access method in which the operating system responds immediately to user I/O demands. It is used when the sequence in which records are to be processed cannot be anticipated, particularly with direct accessing.

**basic input/output system (BIOS)**—Low-level software instructions that control basic hardware initialization and management.

**batch process**—Process that executes without user interaction.

**behavior** (of an object)—See method.

**Belady's anomaly**—See FIFO anomaly.

**benchmark**—Real program that an evaluator executes on the system being evaluated to determine how efficiently the system executes that program; benchmarks are used to compare systems.

**Beowulf cluster**—Linux clustering solution, which is a high-performance cluster. A Beowulf cluster may contain several nodes or several hundred. Theoretically, all nodes have Linux installed as their operating system and are interconnected with high-speed Ethernet. Usually, all the nodes in the cluster are connected within a single room to form a supercomputer.

**Berkeley Software Distribution (BSD) UNIX**-UNIX version modified and released by a team led by Bill Joy at the University of California at Berkeley. BSD UNIX is the parent of several UNIX variations.

**best-fit memory placement strategy**—Memory placement strategy that places an incoming job in the smallest hole in memory that can hold the job.

**bidding algorithm**—Dynamic load balancing algorithm in which processors with smaller loads "bid" for jobs on overloaded processors; the bid value depends on the load of the bidding processor and the distance between the underloaded and overloaded processors.

**big kernel lock (BKL)** (Linux)—Global spin lock that served as an early implementation of SMP support in the Linux kernel.

**binary buddy algorithm** (Linux)—Algorithm that Linux uses to allocate physical page frames. The algorithm maintains a list of groups of contiguous pages; the number in each group is a power of two. This facilitates memory allocation for processes and devices that require access to contiguous physical memory.

**binary relation**—Relation (in a relational database) of degree 2.

**binary semaphore**—Semaphore whose value can be no greater than one, typically used to allocate a single resource.

**binding** (Windows XP)—Connection between the client and the server used for network communication (e.g., for an RPC).

**binding handle** (Windows XP)—Data structure that stores the connection information associated with a binding between a client and a server.

**bio structure** (Linux)—Structure that simplifies block I/O operations by mapping I/O requests to pages.

**biometrics**—Technique that uses an individual's physical characteristics, such as fingerprints, eyeball iris scans or face scans, to identify the user.

**BIPS (billion instructions per second)**—Unit commonly used to categorize the performance of a particular computer; a rating of one BIPS means a processor can execute one billion instructions per second.

**bisection width**—Minimum number of links that need to be severed to divide a network into two unconnected halves.

**bit pattern**—Lowest level of the data hierarchy. A bit pattern is a group of bits that represent virtually all data items of interest in computer systems.

**bitmap**—Free space management technique that maintains one bit for each block in memory, where the *i*th bit corresponds to the *i*th block in memory. Bitmaps enable a file system to more easily allocate contiguous blocks but can require substantial execution time to locate a free block.

**block**—Fixed-size unit of contiguous data, typically much larger than a byte. Placing contiguous data records in blocks enables the system to reduce the number of I/O operations required to retrieve them.

**block allocation**—Technique that enables the file system to manage secondary storage more efficiently and reduce file traversal overhead by allocating extents (blocks of contiguous sectors) to files.

**block allocation bitmap** (Linux)—Bitmap that tracks the usage of blocks in each block group.

**block cipher**—Encryption technique that divides a message into fixed-size groups of bits to which an encryption algorithm is applied.

**block device**—Device such as a disk that transfers data in fixed-size groups of bytes, as opposed to a character device, which transfers data one byte at a time.

**block group** (Linux)—Collection of contiguous blocks managed by group-wide data structures so that related data blocks, inodes and other file system metadata are contiguous on disk.

**block map table**—Table containing entries that map each of a process's virtual blocks to a corresponding block in main memory (if there is one). Blocks in a virtual memory system are either segments or pages.

**block map table origin register**—Register that stores the address in main memory of a process's block map table; this high-speed register facilitates rapid virtual address translation.

**block mapping**—Mechanism that, a virtual memory system, reduces the number of mappings between virtual memory addresses and real memory addresses by mapping blocks in virtual memory to blocks in main memory.

**blocked list**—Kernel data structure that contains pointers to all *blocked* processes. This list is not maintained in any particular priority order.

**blocked record**—Record that may contain several logical records for each physical record.

***blocked* state**—Process (or thread) state in which the process (or thread) is waiting for the completion of some event, such as an I/O completion, and cannot use a processor even if one is available.

**blocking**—Grouping of contiguous records into larger blocks that can be read using a single I/O operation. This technique reduces access times by retrieving many records with a single I/O operation.

**boom**—See disk arm.

**boot sector**—Specified location on a disk in which the initial operating system instructions are stored; the BIOS instructs the hardware to load these initial instructions when the computer is turned on.

**boot sector virus**—Virus that infects the boot sector of the computer's hard disk, allowing it to load with the operating system and take control of the system.

**bootstrapping**—Process of loading initial operating system components into system memory so that they can load the rest of the operating system.

***born* state**—Thread state in which a new thread begins its life cycle.

**bottleneck**—Condition that occurs when a resource receives requests faster than it can process them, which can slow process execution and reduce resource utilization. Hard disks are a bottleneck in most systems.

**bottom half of an interrupt handler** (Linux)—Portion of interrupt-handling code that can be preempted.

**bounce buffer** (Linux)—Region of memory that allows the kernel to map data from the high memory zone into memory that it can directly reference. This is necessary when the system's physical address space is larger than kernel's virtual address space.

**boundary register**—Register for single-user operating systems that was used for memory protection by separating user memory space from kernel memory space.

**bounded buffer**—See circular buffer.

**bounds register**—Register that stores information regarding the range of memory addresses accessible to a process.

**branch penalty**—Performance loss in pipelined architectures associated with a branch instruction; this occurs when a processor cannot begin processing the instruction after the branch until the processor knows the outcome of the branch. The branch penalty can be reduced by using delayed branching, branch prediction or branch predication.

**branch predication**—Technique used in EPIC processors whereby a processor executes all possible instructions that could follow a branch in parallel and uses only the result of the correct branch once the predicate (i.e., the branch comparison) is resolved.

**branch prediction**—Technique whereby a processor uses heuristics to determine the most probable result of a branch in code; when the processor predicts correctly, performance increases, because the processor can continue to execute instructions immediately after the branch.

**brute-force cracking**—Technique to compromise a system simply by attempting all possible passwords or by using every possible decryption key to decrypt a message.

**buffer**—Temporary storage area that holds data during I/O between devices operating at different speeds. Buffers enable a faster device to produce data at its full speed (until the buffer fills) while waiting for the slower device to consume the data.

**buffer overflow**—Attack that sends input that is larger than the space allocated for it. If the input is properly coded and the system's stack is executable, buffer overflows can enable an attacker to execute malicious code.

**bullet server**—Standard file server used in the Amoeba file system.

**burping the memory**—See memory compaction.

**bus**—Collection of traces that form a high-speed communication channel for transporting information between different devices on a mainboard.

**bus driver**—WDM (Windows Driver Model) driver that provides some generic functions for devices on a bus, enumerates those devices and handles Plug and Play I/O requests.

**bus mastering**—DMA transfer in which a device assumes control of the bus (preventing others from accessing the bus simultaneously) to access memory.

**bus network**—Network in which the nodes are connected by a single bus link (also known as a linear network).

**bus snooping**—Coherence protocol in which processors "snoop" the shared bus to determine whether a requested write is to a data item in that processor's cache or, if applicable, local memory.

**Business Application Performance Corporation (BAPCo)** — Organization that develops standard benchmarks, such as the popular SYSMark for processors.

**business-critical system** —System that must function properly, but whose failure of which leads to reduced productivity and profitability; not as crucial as a mission-critical system, where failure could put human lives could be at risk.

**busy waiting**—Form of waiting where a thread continuously tests a condition that will let the thread proceed eventually; while busy waiting, a thread uses processor time.

**byte**—Second-lowest level in the data hierarchy. A byte is typically 8 bits.

**bytecode**—Intermediate code that is intended for virtual machines (e.g., Java bytecode runs on the Java Virtual Machine).

## C

**C**—Procedural programming language developed by Dennis Ritchie that was used to create UNIX.

**C-threads**—Threads supported natively in the Mach micro-kernel (on which Macintosh OS X is built).

**C#**—Object-oriented programming language developed by Microsoft that provides access to .NET libraries.

**C++** —Object-oriented extension of C developed by Bjarne Stroustup.

**cache coherence**—Property of a system in which any data item read from a cache has the value equal to the last write to that data item.

**cache-coherent NUMA (CC-NUMA)** —NUMA multiprocessor that maintains cache coherence, usually through a home-based approach.

**cache-cold process**—Process that contains little, if any, of its data or instructions in the cache of the processor to which it will be dispatched.

**cache hit**—Request for data that is present in the cache.

**cache-hot process**—Process that contains most, if not all, of its data and instructions in the cache of the processor to which it will be dispatched.

**cache line**—Entry in a cache.

**cache manager** (Windows XP)—Executive component that handles file cache management.

**cache memory**—Small, expensive, high-speed memory that holds copies of programs and data to decrease memory access times.

**cache miss**—Request for data that is not present in the cache.

**cache miss latency**—Extra time required to access data that does not reside in the cache.

**cache-only memory architecture (COMA) multiprocessor**—Multiprocessor architecture in which nodes consist of a processor, cache and memory module; main memory is organized as a large cache.

**cache snooping**—Bus snooping used to ensure cache coherency.

**callback** (in AFS) —Sent by the server to notify the client that the cached file is modified.

**cancellation of a thread**—Thread operation that terminates the target thread. Three modes of cancellation are disabled, deferred and asynchronous cancellation.

**capability** — Security mechanism that assigns access rights to a subject (e.g., a process) by granting it a token for an object (i.e., a resource). This enables administrators to specify and enforce fine-grained access control. The token is analogous to a ticket the bearer may use to gain access to a sporting event.

**capacity**—Measure of the maximum throughput a system can attain, assuming that whenever the system is ready to accept more jobs, another job is immediately available.

**Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)**—Protocol used in 802.11 wireless communication. Devices must send a Request To Send (RTS) and receive a Clear To Send (CTS) from the destination host before transmitting.

**Carrier Sense Multiple Access with Collision Detection (CSMA/CD)**—Protocol used in Ethernet that enables transceivers to test a shared medium to see if it is available before transmitting data. If a collision is detected, transceivers continue transmitting data for a period of time to ensure that all transceivers recognize the collision.

**causal broadcast**—Ensures that when a message is sent from one process to all other processes, any given process will receive the message before it receives a response to the message from a different process.

**causal ordering**—Ensures that all processes recognize that a causally dependent event must occur only after the event on which it is dependent.

**causally dependent**—Event B is causally dependent on event A if event B may occur only if event A occurs.

**cell** (in AFS-3)—Unit in AFS-3, which preserves namespace continuity while allowing different systems administrators to oversee each cell.

**central deadlock detection**—A strategy in distributed deadlock detection, in which one site is dedicated to monitoring the TWFG of the entire system. Whenever a process requests or releases a resource, it informs the central site. The site continuously checks the global TWFG for cycles.

**central migration server**—Workstation in a Sprite distributed operating system that keeps information about idle workstations.

**central processing unit (CPU)**—Processor responsible for the general computations in a computer.

**centralized P2P application**—Uses a server that connects to each peer.

**certificate authority (CA)**—Financial institution or other trusted third party, such as VeriSign, that issues digital certificates.

**certificate authority hierarchy** — Chain of certificate authorities, beginning with the root certificate authority, that authenticates certificates and CAs.

**certificate repositories**—Locations where digital certificates are stored.

**certificate revocation list (CRL)** —List of cancelled and revoked certificates. A certificate is cancelled/revoked if a private key is compromised before its expiration date.

**chaining**—Indexed noncontiguous allocation technique that reserves the last few entries of an index block to store pointers to more index blocks, which in turn point to data blocks. Chaining enables index blocks to reference large files by storing references to its data across several blocks.

**chaining** (hash tables)—Technique that resolves collisions in a hash table by placing each unique item in a data structure (typically a linked list). The position in the hash table at which the collision occurred contains a pointer to that data structure.

**character**—In the data hierarchy, a fixed-length pattern of bits, typically 8,16 or 32 bits.

**character device**—Device such as a keyboard or mouse that transfers data one byte at a time, as opposed to a block device, which transfers data in fixed-size groups of bytes.

**character set**—Collection of characters. Popular character sets include ASCII, EBCDIC and Unicode.

**checkpoint** (transaction logging)—Marker indicating which transactions in a log have been transferred to permanent storage. The system need only reapply the transactions from the latest checkpoint to determine the state of the file system, which is faster than reapplying all transactions starting at the beginning of the log.

**checkpoint** (deadlock recovery)—Record of a state of a system so that it can be restored later if a process must be prematurely terminated (e.g., to perform deadlock recovery).

**checkpoint/rollback**—Method of deadlock and system recovery that undoes every action (or transaction) of a terminated process since the process's last checkpoint.

**checksum**—Result of a calculation on the bits of a message. The checksum calculated at the receiver is compared to the checksum calculated by the sender (which is embedded in the control information). If the checksums do not match, the message has been corrupted.

**child process**—Process that has been spawned from a parent process. A child process is one level lower in the process hierarchy than its parent process. In UNIX systems, child processes are created using the fork system call.

**chipset**—Collection of controllers, coprocessors, buses and other hardware specific to the mainboard that determine the hardware capabilities of a system.

**cipher**—Mathematical algorithm for encrypting messages. Also called cryptosystem.

**ciphertext**—Encrypted data.

**circular buffer**—In the producer/consumer relationship, a fixed-size region of shared memory that stores multiple values produced by a producer. If the producer occasionally produces values faster than the consumer, a circular buffer reduces the time the producer spends waiting for a consumer to consume the values, when compared to a buffer that stores a single value. If the consumer temporarily consumes values faster than the producer, a circular buffer can similarly reduce the time a consumer spends waiting for the producer to produce values.

**circular LOOK (C-LOOK) disk scheduling**—Disk scheduling strategy that moves the arm in one direction, servicing requests on a shortest-seek basis. When there are no more requests on a current sweep, the read-write head moves to the request closest to the cylinder opposite its current location (without servicing requests in between) and begins the next sweep. The C-LOOK policy is characterized by potentially lower variance of response times compared to LOOK; it offers high throughput (although lower than that of LOOK).

**circular SCAN (C-SCAN) disk scheduling**—Disk scheduling strategy that moves the arm in one direction, servicing requests on a shortest-seek basis. When the arm has completed its sweep, it jumps (without servicing requests) to the cylinder opposite its current location, then resumes its inward sweep, processing requests. C-SCAN maintains high levels of throughput while further limiting variance of response times by avoiding the discrimination against the innermost and outermost cylinders.

**circular wait**—Condition for deadlock that occurs when two or more processes are locked in a "circular chain," in which each process in the chain is waiting for one or more resources that the next process in the chain is holding.

**circular-wait necessary condition for deadlock**—One of the four necessary conditions for deadlock; states that if a deadlock exists, there will be two or more processes in a circular chain such that each process is waiting for a resource held by the next process in the chain.

**class**—Type of an object. Determines an object's methods and attributes.

**class ID (CLSID)**—Globally unique ID given to a COM object class.

**class/mini class driver pair** (Windows XP)—Pair of device drivers that act as a function driver for a device; the class driver provides generic processing for the device's particular device class, and the miniclass driver provides processing for the specific device.

**Clear to Send (CTS)**—Message that a receiver broadcasts in the CSMA/CA protocol to indicate that the medium is free. A CTS message is sent in response to a Request to Send (RTS).

**client**—Process that requests a service from another process (a server). The machine on which the client process runs is also called a client.

**client caching**—Clients keep local copies of files and flush them to the server after having modified the files.

**client modification log (CML)** (in Coda)—Log which is updated to reflect file changes on disk.

**client stub**—Stub at the client side that prepares outbound data for transmission and translates incoming data so that it may be correctly interpreted.

**client/server model**—Popular networking paradigm in which processes that need various services performed (clients) transmit their requests to processes that provide these services (servers). The server processes the request and returns the result to the client. The client and the server are typically on different machines on the network.

**clipboard** (Windows XP) —Central repository of data that is accessible to all processes, typically used with copy, cut and paste commands

**clock IRQL** (Windows XP) — IRQL at which clock interrupts occur and at which APC, DPC/dispatch, DIRQL and profile-level interrupts are masked.

**clock page-replacement strategy—**Variation ot the second-chance page-replacement strategy that arranges the pages in a circular list instead of a linear list. A list pointer moves around the circular list, much as the hand of a clock rotates, and replaces the page nearest the pointer (in circular order) that has its referenced bit turned off.

**clocktick—**See cycle.

**close** (file) — Operation that prevents further reference to a file until it is reopened.

**cluster—**Set of nodes that forms what appears to be a single parallel machine.

**cluster** (NTFS) — Basic unit of disk storage in an NTFS volume, consisting of a number of contiguous sectors; a system's cluster size can range from 512 bytes to 64KB, but is typically 2KB, 4KB or 8KB.

**clustering—**Interconnection of nodes within a high-speed LAN so that they function as a single parallel computer.

**CMS Batch Facility** (VM)—VM Component that allows the user to run longer jobs in a separate virtual machine so that the user can continue interactive work.

**coalescing memory holes—**Process of merging adjacent holes in memory in variable-partition multiprogramming systems. This helps create the largest possible holes available for incoming programs and data.

**coarse-grained strip** (RAID) —Strip size that enables average files to be stored in a small number of strips. In this case, some requests can be serviced by only a portion of the disks in the array, so it is more likely that multiple requests can be serviced simultaneously. If requests are small, they are serviced by one disk at a time, which reduces their average transfer rates.

**Coda Optimistic Protocol—**Protocol used by Coda clients to write a copy of the file to each of the members of the AVSG, which provides a consistent view of a file within an AVSG.

code **freeze** (Linux)—Point at which no new code should be added to the kernel unless the code fixes a known bug.

**code generator—**Part of a compiler responsible for producing object code from a higher-level language.

**collective layer** (in grid computing) — Layer responsible for coordinating distributed resources, such as scheduling a task to analyze data received from a scientific device.

**collision** (CSMA/CD protocol) —Simultaneous transmission in CSMA/CD protocol.

**collision** (hash tables)—Event that occurs when a hash function maps two different items to the same position in a hash table. Some hash tables use chaining to resolve collisions.

**COM+—**Extension of COM (Microsoft's Common Object Model) that handles advanced resource management tasks, such as providing support for transaction processing and using thread and object pools.

**commit memory** (Windows XP)—Necessary stage before a process can access memory. The VMM ensures that there is enough space in a pagefile for the memory and creates page table entries (PTEs) in main memory for the committed pages.

**committed transaction—**Transaction that has completed successfully.

**COmmon Business Oriented Language (COBOL)—**Procedural programming language developed in the late 1950s that was designed for writing business software that manipulates large volumes of data.

**Common Internet File System (CIFS)—**Native file sharing protocol of Windows XP.

**Common Object Request Broker Architecture (CORBA)** —Conceived in the early 1990s by the Object Management Group (OMG), CORBA is a standard specification of distributed systems architecture that has gained wide acceptance.

**communication deadlock—**One of the two types of distributed deadlock, which is a circular waiting for communication signals.

**compact disk (CD)—**Digital storage medium in which data is stored as a series of microscopic pits on a flat surface.

**compile—**Translate high-level-language source code into machine code.

**compiler—**Application that translates high-level-language source code into machine code.

**complex instruction set computing (CISC)** —Processor-design philosophy, emphasizing expanded instruction sets that incorporate single instructions that perform several operations.

**component** (Windows XP)—Functional unit of Windows XP. Components range from user-mode applications such as Notepad to portions of kernel space such as the CD-ROM interface and power management tools. Dividing Windows XP into components facilitates operating system development for embedded systems.

**Component Object Model (COM)—**Microsoft-developed software architecture that allows interoperability between diverse components through the standard manipulation of object interfaces.

**compression unit** (NTFS) —Sixteen clusters. Windows XP compresses and encrypts files one compression unit at a time.

**compute-bound**—See processor-bound.

**concurrent—**The description of a process or thread that exists in a system simultaneously with other processes and/or threads.

**concurrent** (happens-before relation)—Two events are concurrent if it cannot be determined which occurred earlier by following the happens-before relation.

**concurrent program execution—**Technique whereby processor time is shared among multiple active processes. On a uniprocessor system, concurrent processes cannot execute simultaneously; on a multiprocessor system, they can.

**concurrent write sharing—**Occurs when two clients modify cached copies of the same file.

**condition variable—**Variable that contains a value and an associated queue. When a thread waits on a condition variable inside a monitor, it exits the monitor and is placed in the condition variable's queue. Threads wait in the queue until signaled by another thread.

**configurable lock** — See adaptive lock

**configuration manager** (Windows XP) — Executive component that manages the registry.

**congestion control**—Means by which TCP restricts the number of segments sent by a single host in response to network congestion.

**connection-oriented transport**—Method of implementing the transport layer in which hosts send control information to govern the session. Handshaking is used to set up the connection. The connection guarantees that all data will arrive and in the correct order.

**connectionless transport**—Method of implementing the transport layer in which there is no guarantee that data will arrive in order or at all.

**connectivity layer** (in grid computing)—Layer that carries out reliable and secure transactions between distributed resources.

**consumer thread**—Thread whose purpose is to read and process data from a shared object.

**contention** — In multiprocessing, a situation in which several processors compete for the use of a shared resource.

**context switching** — Action performed by the operating system to remove a process from a processor and replace it with another. The operating system must save the state of the process that it replaces. Similarly, it must restore the state of the process being dispatched to the processor.

**contiguous memory allocation** — Method of assigning memory such that all of the addresses in the process's entire address space are adjacent to one another.

**control information** — Data in the form of headers and/or trailers that allows protocols of the same layer on different machines to communicate. Control information might include the addresses of the source and destination hosts and the type of data or size of the data that is being sent.

**Control Program (CP)** (VM) — Component of VM that runs the physical machine and creates the environment for the virtual machine.

**controller** — Hardware component that manages access to a bus by devices.

**Conversational Monitor System (CMS)** (VM)—Component of VM that is an interactive application development environment.

**cooperative multitasking**—Process scheduling technique in which processes execute on a processor until they voluntarily relinquish control of it.

**coprocessor**—Processor, such as a graphics or digital signal processor, designed to efficiently execute a limited set of special-purpose instructions (e.g., 3D transformations).

**copy** (file) — Operation that creates another version of a file with a new name.

**copy-on-reference migration**—Process migration technique in which only a process's dirty pages are migrated with the process, and the process can request clean pages either from the sending node or from secondary storage.

**copy-on-write**—Mechanism that improves process creation efficiency by sharing mapping information between parent and child until a process modifies a page, at which point a new copy of the page is created and allocated to that process. This can incur substantial overhead if the parent or child modifies many of the shared pages.

**core file** (Linux)—File that contains the execution state of a process, typically used for debugging purposes after a process encounters a fatal exception.

**coschediiling**—Job-aware process scheduling algorithm that attempts to execute processes from the same job concurrently by placing them in adjacent global-run-queue locations.

**cost of an interconnection scheme**—Total number of links in a network.

**counting semaphore** — Semaphore whose value may be greater than one, typically used to allocate resources from a pool of identical resources.

**CP/CMS**—Timesharing operating system developed by IBM in the 1960s.

**cracker**—Malicious individual that is usually interested in breaking into a system to disable services or steal data.

**create** (file) — Operation that builds a new file.

**credential**—Combination of user identity (e.g., username) and proof of identify (e.g., password).

**critical region** — See critical section.

**critical section**—Section of code that performs operations on a shared resource (e.g., writing data to a shared variable). To ensure program correctness, at most one thread can simultaneously execute in its critical section.

**critical section object** (Windows XP)—Synchronization object, which can be employed only by threads within a single process, that allows only one thread to own a resource at a time.

**crossbar-switch matrix**—Processor interconnection scheme that maintains a separate path from every sender node to every receiver node.

**cryptanalytic attack**—Technique that attempts to decrypt ciphertext without possession of the decryption key. The most common such attacks are those in which the encryption algorithm is analyzed to find relations between bits of the encryption key and bits of the ciphertext.

**Cryptographic API** (Linux)—Kernel interface through which applications and services (e.g., file systems) can encrypt and decrypt data.

**cryptography**—Study of encoding and decoding data so that it can be interpreted only by the intended recipients.

**cryptosystem**—Mathematical algorithm for encrypting messages. Also called a cipher.

**CTSS**—Timesharing operating system developed at MIT in the 1960s.

**cycle** (clock)—One complete oscillation of an electrical signal. The number of cycles that occur per second determines a device's frequency (e.g., processors, memory and buses) and can be used by the system to measure time.

**cycle stealing**—Method that gives channels priority over a processor when accessing the bus to prevent signals from channels and processors from colliding.

**cylinder**—Set of tracks that can be accessed by the read/write heads for a specific position of the disk arm.

## D

**daemon** (Linux)—Process that runs periodically to perform system services.

**data bus**—Bus that transfers data to or from locations in memory that are specified by the address bus.

**data compression**—Technique that decreases the size of a data record by replacing repetitive patterns with shorter bit strings. This can reduce seeks, and transmission times, but may require substantial processor time to compress the data for storage on the disk, and to decompress the data to make it available to applications.

**data definition language (DDL)**—Type of language that specifies the organization of data in a database.

**data-dependent application**—Application that relies on a particular file system's organization and access techniques.

**Data Encryption Standard (DES)**—Symmetric encryption algorithm that uses a 56-bit key and encrypts data in 64-bit blocks. For many years, DES was the encryption standard set by the U.S. government and the American National Standards Institute (ANSI). However, due to advances in computing power, DES is no longer considered secure—in the late 1990s, specialized DES cracker machines were built that recovered DES keys after a period of several hours.

**data hierarchy**—Classification that groups different numbers of bits to extract meaningful data. Bit patterns, bytes and words contain small numbers of bits that are interpreted by hardware and low-level software. Fields, records and files may contain large numbers of bits that are interpreted by operating systems and user applications.

**data independence**—Property of applications that do not rely on a particular file organization technique or access technique.

**data link layer** (in OSI)—At the sender, converts the data representation from the network layer into bits to be transmitted over the physical layer. At the receiver, converts the bits into the data representation for the network layer.

**data manipulation language (DML)**—Type of language that enables data modification.

**data regeneration** (RAID)—Reconstruction of lost data (due to disk errors or failures) in RAID systems.

**data region**—Section of a process's address space that contains data (as opposed to instructions). This region is modifiable.

**data stream** (NTFS)—File attribute that stores the file's content or metadata; a file can have multiple data streams.

**data striping** (RAID)—Technique in RAID systems that divides contiguous data into fixed-size strips that can be placed on different disks. This enables multiple disks to service requests for data.

**database**—Centrally controlled collection of data that is stored in a standardized format and can be searched based on logical relations between data. Databases organize data according to content as opposed to pathname, which tends to reduce or eliminate redundant information.

**database language**—Language that provides for organizing, modifying and querying of structured data.

**database management system (DBMS)**—Software that controls database organization and operations.

**database system**—A particular set of data, the storage devices on which it resides and the software that controls its storage and retrieval (called a database management system or DBMS).

**datagram**—Piece of data transferred using UDP or **IP.**

**datagram socket**—Socket that uses the UDP protocol to transmit data.

**dcache (directory entry cache)** (Linux)—Cache that stores directory entries (dentries), which enables the kernel to quickly map file descriptors to their corresponding inodes.

**deactivated process** (Linux)—Process that has been removed from the run queues and can therefore no longer contend for processor time.

***dead* state**—Thread state entered after a thread completes its task or otherwise terminates.

**deadline rate-monotonic scheduling**—Scheduling policy in real-time systems that meets a periodic process's deadline that does not equal its period.

**deadline scheduler** (Linux)—Disk scheduling algorithm that eliminates indefinite postponement by assigning deadlines by which I/O requests are serviced.

**deadline scheduling**—Scheduling a process or thread to complete by a definite time; the priority of the process or thread may need to be increased as its completion deadline approaches.

**deadlock**—Situation in which a process or thread is waiting for an event that will never occur and therefore cannot continue execution.

**deadlock avoidance**—Strategy that eliminates deadlock by allowing a system to approach deadlock, but ensuring that deadlock never occurs. Avoidance algorithms can achieve higher performance than deadlock prevention algorithms. (See also Dijktra's Banker's Algorithm.)

**deadlock detection**—Process of determining whether or not a system is deadlocked. Once detected, a deadlock can be removed from a system, typically resulting in loss of work.

**deadlock prevention**—Process of disallowing deadlock by eliminating one of the four necessary conditions for deadlock.

**deadlock recovery**—Process of removing a deadlock from a system. This can involve suspending a process temporarily (and preserving its work) or sometimes killing a process (thereby losing its work) and restarting it.

**deadly embrace**—See deadlock.

**decentralized peer-to-peer application**—Also called a pure peer-to-peer application. It does not have a server and therefore does not suffer from the same deficiencies as applications that depend on servers.

**decryption**—Technique that reverses data encryption so that data can be read in its original form.

**dedicated resource**—Resource that may be used by only one process at a time. Also known as a serially reusable resource.

**default action for a signal handler** (Linux)—Predefined signal handler that is executed in response to a signal when a process does not specify a corresponding signal handler.

**default data stream** (NTFS)—File attribute that stores the primary contents of an NTFS file. When an NTFS file is copied to a file system that does not support multiple data streams, only the default data stream is preserved.

**deferred cancellation** (POSfX)—Cancellation mode in which a thread is terminated only after explicitly checking that it has received a cancellation signal.

**deferred procedure call (DPC)** (Windows XP)—Software interrupt that executes at the DPC/dispatch IRQL and executes in the context of the currently executing thread.

**defragmentation—**Moving parts of files so that they are located in contiguous blocks on disk. This can reduce access times when reading from or writing to files sequentially.

**degree—**Number of attributes in a relation in a relational database.

**degree of a node** — Number of other nodes with which a node is directly connected.

**degree of multiprogramming—**Total number of processes in main memory at a given time.

**Dekker's Algorithm—**Algorithm that ensures mutual exclusion between two threads and prevents both indefinite postponement and deadlock.

**delayed blocking—**Technique whereby a process spins on a lock for a fixed amount of time before it blocks; the rationale is that if the process does not obtain the lock quickly, it will probably have to wait a long time, so it should block.

**delayed branching—**Optimization technique for pipelined processors in which a compiler places directly after a branch an instruction that must be executed whether or not the branch is taken; the processor begins executing this instruction while determining the outcome of the branch.

**delayed consistency**—Memory coherence strategy in which processors send update information after a release, but a receiving node does not apply this information until it performs an acquire operation on the memory.

**delegation** (in NFS)—Allows the server to temporarily transfer the control of a file to a client. When the server grants a read delegation of a particular file, then no other clients can write to that file, but they can read it. When the server grants a write delegation of a particular file to the client, then no other clients can read or write that file.

**delete** (file)—Operation that removes a data item from a file.

**demand fetch strategy**—Method of bringing program parts or data into main memory as they are requested by a process.

**demand paging—**Technique that loads a process's nonresident pages into memory only when the process explicitly references the pages.

**denial-of-service (DoS) attack—**Attack that prevents a system from properly servicing legitimate requests. In many DoS attacks, unauthorized traffic saturates a network's resources, restricting access for legitimate users. Typically, the attack is performed by flooding servers with data packets.

**dentry (directory entry)** (Linux)—Structure that maps a file to an inode.

**derived speed—**Actual speed of a device as determined by the front-side bus speed and clock multipliers or dividers.

**desktop environment—**GUI layer above a window manager that provides tools, applications and other software to improve system usability.

**destroy** (file) — Operation that removes a file from the file system.

**device class** — Group of devices that perform similar functions.

**device driver—**Software through which the kernel interacts with hardware devices. Device drivers are intimately familiar with the specifics of the devices they manage —such as the arrangement of data on those devices—and they deal with device-specific operations such as reading data, writing data and opening and closing a DVD drive's tray. Drivers are modular, so they can be added and removed as a system's hardware changes, enabling users to add new types of devices easily; in this way they contribute to a system's extensibility.

**device extension** (Windows XP)—Portion of the nonpaged pool that stores information a driver needs to process I/O requests for a particular device.

**device independence—**Property of files that can be referenced by an application using a symbolic name instead of a name indicating the device on which it resides.

**device IRQL (DIRQL)** (Windows XP)-IRQL at which devices interrupt and at which APC, DPC/dispatch and lower DIRQLs are masked; the number of DIRQLs is architecture dependent.

**device object** (Windows XP)—Object that a device driver uses to store information about a physical or logical device.

**device special file** (Linux)—Entry in the /dev directory that provides access to a particular device.

**Dhrystone**—Classic synthetic program that measures how effectively an architecture runs systems programs.

**digital certificate—**Digital document that identifies a user or organization and is issued by a certificate authority. A digital certificate includes the name of the subject (the organization or individual being certified), the subject's public key, a serial number (to uniquely identify the certificate), an expiration date, the signature of the trusted certificate authority and any other relevant information.

**digital envelope—**Technique that protects message privacy by sending a package including a message encrypted using a secret key and the secret key encrypted using public-key encryption.

**digital notary service**—See time-stamping agency.

**digital signature**—Electronic equivalent of a written signature. To create a digital signature, a sender first applies a hash function to the original plaintext message. Next, the sender uses the sender's private key to encrypt the message digest (the hash value). This step

creates a digital signature and validates the sender's identity, because only the owner of that private key could encrypt the message.

**Digital Signature Algorithm (DSA)**—U.S. government's digital authentication standard.

**digital watermark**—Popular application of steganography that hides information in unused (or rarely used) portions of a file.

**Dijkstra's algorithm**—Efficient algorithm to find the shortest paths in a weighted graph.

**Dijkstra's Banker's Algorithm**—Deadlock avoidance algorithm that controls resource allocation based on the amount of resources owned by the system, the amount of resources owned by each process and the maximum amount of resources that the process will request during execution. Allows resources to be assigned to processes only when the allocation results in a safe state. (See also safe state and unsafe state.)

**Dining Philosophers**—Classic problem introduced by Dijkstra that illustrates the problems inherent in concurrent programming, including deadlock and indefinite postponement. The problem requires the programmer to ensure that a set of *n* philosophers at a table containing *n* forks, who alternate between eating and thinking, do not starve while attempting to acquire the two adjacent forks necessary to eat.

**direct file organization**—File organization technique in which a record is directly (randomly) accessed by its physical address on a direct access storage device (DASD).

**direct I/O**—Technique that performs I/O without using the kernel's buffer cache. This leads to more efficient memory utilization in database applications, which typically maintain their own buffer cache.

**direct mapping**—Address translation mechanism that assists in the mapping of virtual addresses to their corresponding real addresses, using an index into a table stored in location-addressed memory.

**direct memory access (DMA)**—Method of transferring data from a device to main memory via a controller that requires interrupting the processor only when the transfer completes. I/O transfer via DMA is more efficient than programmed I/O or interrupt-driven I/O because the processor does not need to supervise the transfer of each byte or word of data.

**directory**—File storing references to other files. Directory entries often include a file's name, type, and size.

**directory junction** (Windows XP)—Directory that refers to another directory within the same volume, used to make navigating the file system easier.

**dirty bit**—Page table entry bit that specifies whether the page has been modified (also known as the modified bit).

**dirty eager migration**—Process migration method in which only a process's dirty pages are migrated with the process; clean pages must be accessed from secondary storage.

**disable (mask) interrupts**—When a type of interrupt is disabled (masked), interrupts of that type are not delivered to the process that has disabled (masked) the interrupts. The interrupts are either queued to be delivered later or dropped by the processor. This technique can be used to temporarily ignore interrupts, allowing a thread on a uniprocessor system to execute its critical section atomically.

**disabled cancellation** (POSIX) —Cancellation mode in which a thread does not receive pending cancellation signals.

**discovery** (in Jini)—Process of finding the lookup services and obtaining references to them.

**discretionary access control (DAC)**—Access control model in which the creator of an object controls the permissions for that object.

**discretionary access control list (DACL)** (Windows XP) — Ordered list that tells Windows XP which security principals may access a particular resource and what actions those principals may perform on the resource.

**disk arm**—Moving-head disk component that moves read/write heads linearly, parallel to disk surfaces.

**disk arm anticipation**—Moving the disk arm to a location that will minimize the next seek. Disk arm anticipation can be useful in environments where process disk-request patterns exhibit locality and when the load is light enough that there is sufficient time to move the disk arm between disk requests without degrading performance.

**disk cache buffer**—A region of main memory that the operating system reserves for disk data. In one context, the reserved memory acts as a cache, allowing processes quick access to data that would otherwise need to be fetched from disk. The reserved memory also acts as a buffer, allowing the operating system to delay writing the data to improve I/O performance by batching multiple writes into a small number of requests.

**disk mirroring** (RAID)—Data redundancy technique in RAID that maintains a copy of each disk's contents on a separate disk. This technique provides high reliability and simplifies data regeneration but incurs substantial storage overhead, which increases cost.

**disk reorganization**—Technique that moves file data on disk to improve its access time. One such technique is defragmentation, which attempts to place sequential file data contiguously on disk. Another technique attempts to place frequently requested data on tracks that result in low average seek times.

**disk scheduler** —Operating system component that determines the order in which disk I/O requests are serviced to improve performance.

**disk scheduling**—Technique that orders disk requests to maximize throughput and minimize response times and the variance of response times. Disk scheduling strategies improve performance by reducing seek times and rotational latencies.

**dispatcher** —Operating system component that assigns the first process on the ready list to a processor.

**dispatcher** (Windows XP)—Thread scheduling code dispersed throughout the microkernel.

**dispatcher object** (Windows XP) —Object, such as a mutex, semaphore, event or waitable timer, that kernel and user threads can use for synchronization purposes.

**dispatching**—Act of assigning a processor to a process.

**dispersion**—Measure of the variance of a random variable.

**displacement**—Distance of an address from the start of a block, page or segment, also called offset.

**Distributed Component Object Model (DCOM)**—Distributed systems extension of Microsoft's COM.

**distributed computing**—Using multiple independent computers to perform a common task.

**distributed database**—Database that is spread throughout the computer systems of a network.

**distributed deadlock**—Similar to deadlock in a uniprocessor system, except that the processes concerned are spread over different computers.

**distributed deadlock detection strategy**—Technique used to find deadlock in a distributed system.

**distributed denial-of-service attack**—Attack that prevents a system from servicing requests properly by initiating packet flooding from separate computers sending traffic in concert.

**distributed file system**—File system that is spread throughout the computer systems of a network.

**distributed operating system**—Provides all of the same services as a traditional operating system, but must provide adequate transparency such that objects in the system are unaware of the computers that provide the service.

**distributed search**—Searching technology used in peer-to-peer applications to make networks more robust by removing single points of failure, such as servers. In a distributed search, if a peer cannot answer the client's request, the peer forwards the request to its directly connected peers, and the search is distributed to the entire peer-to-peer network.

**distributed system**—Collection of remote computers that cooperate via a network to perform a common task.

**distribution** (Linux)—Software package containing the Linux kernel, user applications and/or tools that simplify the installation process.

**distribution of response times**—Set of values describing the response times for jobs in a system and the relative frequencies with which those values occur.

**DMA memory** (Linux)—Region of physical memory between zero and 16MB that is typically reserved for kernel bootstrapping code and legacy DMA devices.

**DNS (domain name system) attack**—Attack that modifies the address to which network traffic for a particular Web site is sent. Such attacks can be used to redirect users of a particular Web site to another, potentially malicious Web site.

**domain**—Set of possible values for attributes in a relational database system.

**domain** (in Sprite file system)—Unit that represents a portion of the global file hierarchy and is stored at one server.

**domain** (Windows XP)—Set of computers that share common resources.

**domain controller** (Windows XP)—Computer responsible for security on a network.

**Domain Name System (DNS)**—System on the Internet used to translate a machine's name to an IP address.

**double data rate (DDR)**—Chipset feature that enables a frontside bus to effectively operate at twice its clock speed by performing two memory transfers per clock cycle. This feature must be supported by the system's chipset and RAM.

**doubly indirect pointer**—Inode pointer that locates a block of (singly) indirect pointers.

**DPC/dispatch IRQL** (Windows XP)-IRQL at which DPCs and the thread scheduler execute and at which APC and incoming DPC/dispatch level interrupts are masked.

**drafting algorithm**—Dynamic load balancing algorithm that classifies each processor's load as low, normal or high; each processor maintains a table of the other processors' loads, and the system uses a receiver-initiated policy to exchange processes.

**driver object** (Windows XP)—Object used to describe device drivers; it stores pointers to standard driver routines and to the device objects for the devices that the driver services.

**driver stack**—Group of related device drivers that cooperate to handle the I/O requests for a particular device.

**dynamic address translation (DAT)**—Mechanism that converts virtual addresses to physical addresses during execution; this is done at extremely high speed to avoid slowing execution.

**dynamic-linked library (DLL)** (Windows XP)—Module that provides data or functions to which processes or other DLLs link at execution time.

**dynamic linking**—Linking mechanism that resolves references to external functions when the process first makes a call to the function. This can reduce linking overhead because external functions that are never called while the process executes are not linked.

**dynamic load balancing**—Technique that attempts to distribute processing responsibility equally by changing the number of processors assigned to a job throughout the job's life.

**dynamic loading**—Method for loading that specifies memory addresses at runtime.

**dynamic partitioning**—Job-aware process scheduling algorithm that divides processors in the system evenly among jobs, except that no single job can be allocated more processors than runnable processes; this algorithm maximizes processor affinity.

**dynamic priority class** (Windows XP)—Priority class that encompasses the five base priority classes within which a thread's priority can change during system execution; these classes are idle, below normal, normal, above normal and high.

**dynamic RAM (DRAM)**—RAM that must be continuously read by a refresh circuit to keep the contents in memory.

**dynamic real-time scheduling algorithm**—Scheduling algorithm that uses deadlines to assign priorities to processes throughout execution.

## *E*

**eager migration**—Process migration strategy that transfers the entire address space of a process during the initial phases of migration to